

Master Security

Security in Spring Boot & Microservices

Why Security for your spring boot app?

Security is needed in a Spring Boot app to:

- Protect sensitive data.
- Prevent unauthorized access to critical features.
- Defend against common attacks (CSRF, XSS, etc.).
- Ensure compliance with industry regulations.

What is Spring Security?

Spring Security is a customizable framework for managing **authentication**, **authorization**, and protection against common security threats (like DDoS i.e. Distributed Denial of Service Attack) in Spring applications.

Think of application level security like **protecting your home**—you could leave the door open, hide a spare key, or go all out with **locks, cameras, and alarms**.

In the same way, Spring Security gives you the tools to decide how **tight or relaxed** your application's security should be.

Spring Security helps you write less code for securing your app by offering built-in features and avoiding repetitive logic. Like the rest of Spring, it **reduces boilerplate** while still giving you the flexibility to customize everything as needed.

Key Spring Security Concepts

Authentication

Authorization

Servlet Filters

Authentication

Imagine you're trying to enter a private club. To get in, you need to prove who you are. Here's how it works:

- **The Club Entrance** (Your Website/Application): This is the door or gate where you want to get into (e.g., your website or app).
- **The Key** (Your Username & Password): To enter the club, you need a key. But not just any key! It's a special key that only you should have. In the world of websites, this "key" is like your username (your identity) and password (your secret code).



Authentication

- **The Bouncer (Authentication System):** At the door, there's a bouncer (the authentication system). The bouncer's job is to check if the key you have matches the one they have on record.
- If the key is correct (you entered the right username and password), the bouncer says, "Welcome in! You're authenticated!" and you're allowed to enter the club.
- What Happens if You Don't Have the Right Key?
- If you try to enter with a wrong key (incorrect username or password), the bouncer stops you and says, "Sorry, you're not on the list." You won't be allowed to enter.



Authentication

In the Context of Web Applications:

- The club entrance is the website or application you're trying to use.
- The key is your username and password (or sometimes even a fingerprint or face scan in more secure cases).
- The bouncer is the system that checks if your credentials are valid (authentication process).
- So, **authentication** is like the process of proving you are who you say you are using the key (your credentials). If your credentials are correct, you're allowed in. If not, you get stopped at the door.
- In short: **Authentication = Proving your identity.**



Authorization

- Authorization is about **what you're allowed to do or where you're allowed to go once you've proved your identity.**
- Analogy –
- Now, let's say you're inside the private club you've just entered after passing the bouncer (authentication). But inside the club, there are different rooms—some are for VIPs, some are for regular guests, and some are for staff only. You can't just wander into any room; **you need permission based on who you are.**

Authorization

The Wristband (Your Role or Permission):

- After you've entered the club (authenticated), you're given a wristband (this could represent your role or permissions).
 - If you're a VIP, you get a VIP wristband.
 - If you're just a regular guest, you get a guest wristband.
 - If you're a staff member, you get a staff wristband.
- The wristband is a way of telling the club staff which rooms you're allowed to enter.



Authorization

The Club Staff (Authorization System):

- When you try to enter a particular room, the staff checks your wristband to see if you have permission to enter.
 - VIP Room: If you have a VIP wristband, the staff lets you in.
 - Staff Room: If you're a regular guest, the staff stops you because your wristband doesn't give you access.
 - General Seating Area: If you're a guest or a VIP, you can enter, since it's open to most people.

What Happens if You Don't Have Permission?:

- If you try to enter a restricted room (a room you don't have permission for), the staff will stop you and say, "Sorry, you don't have access to this room."



Authorization

In the Context of Web Applications:

- The rooms are different sections or pages in the application (e.g., an admin panel, settings page).
- The wristband is your role (like “Admin”, “User”, “Guest”) or permission (can you view/edit something).
- The club staff is the **authorization system** that checks if your role or permission allows you to access a specific resource or page.



Servlet Filters

Servlet Filters provide a way to handle common tasks like authentication and authorization in one place, keeping your actual controller code clean and focused on business logic.

We will look into filter's layer a lot in upcoming videos

What are its alternatives?

While Spring Security is the go-to framework for securing Spring apps, an alternative is **Apache Shiro**. It's easy to integrate with Spring, flexible, and works for a variety of applications—from web to mobile. Shiro simplifies security with its own annotations and HTTP filters.

However, it may be **too lightweight for complex needs**, as Spring Security offers a **broad**er set of features and is more tailored to Spring applications with a larger, active community.

Security Implementation - Who's responsibility

Using **Spring Security** in a Spring app means setting up rules to control who can access what. It intercepts requests and checks if the user has the **right permissions**.

Like setting up an alarm system, it's up to the developer to configure it properly—if **something's left unprotected, it's not the system's fault**.

Let's get started !

In this **course**, we'll create our first app with Spring Security. For Spring apps, Spring Security is a great choice for securing them. We'll use **Spring Boot**, explore default configurations, and briefly touch on how to customize them. This will give you a solid introduction to Spring Security and authentication.

Creating first project

Let's start by creating a simple web app with a REST endpoint. You'll see how Spring Security automatically secures the endpoint using HTTP Basic authentication, where the app checks the username and password sent in the HTTP request header.

With the default configuration, the app has two different authentication mechanisms in place: **HTTP Basic** and **Form Login**.

Just by creating the project and adding the correct dependencies, Spring Boot applies default configurations, including a username and a password, when you start the application.

But if you try accessing the URL in a browser, you'll find that your app implements a nice form for user authentication and doesn't display an ugly HTTP Basic

When you use **HTTP Basic** authentication, the browser typically shows a simple pop-up dialog (often called the "HTTP Basic Authentication box") asking for a username and password.

However, if you're seeing a **login form** in the browser instead, it's likely because **Spring Security's default configuration** has enabled Form Login alongside HTTP Basic, as both are supported out-of-the-box. In this case, the form-based login will take precedence when accessing the URL in a browser, providing a more user-friendly interface compared to the basic authentication pop-up.

So, even though **HTTP Basic** is configured, the app might default to showing the form for authentication, especially if you're testing in a web browser. You'll focus on **HTTP Basic** in the upcoming sections, but this form-based login is a part of the default setup.

For our first project, you'll only need two dependencies: **spring-boot-starter-web** and **spring-boot-starter-security**. After creating the project, add these dependencies to your pom.xml file.

The goal is to observe how a default Spring Security configuration behaves and understand the components included in the default setup and their purpose.

Once you run the application, besides the other lines in the console, you should see something that looks like

Using generated security password: 93a01cf0-794b-4b98-86ef-54860f36f7f3

Each time you run the application, it generates a new password and prints this password in the console, as presented in the previous code snippet. You must use this password to call any of the application's endpoints with HTTP Basic authentication

By default, Spring Security expects the default username (user) with the provided password

Accessing a Secured Endpoint with Spring Security (HTTP Basic)

When you try to access a secured endpoint in a Spring Boot app (like `http://localhost:8080/hello`) without providing credentials, the server responds with:

401 Unauthorized

This is expected because Spring Security is enabled by default and protects endpoints using HTTP Basic Authentication unless configured otherwise.

HTTP Basic Auth requires you to send a username and password with each request. These credentials are:

Combined in the format: `User : Password`

Base64 encoded (not encrypted—just encoded for safe transmission).

Sent in the HTTP Authorization header like this:

Basic dXNlcjowNDg5YmUxMi0wMGQ0LTQzNWMtYTQyMy03YWZhYTY0MWJkODA=

Using Postman

To test this easily:

1. Open Postman.
2. Go to the **Authorization** tab.
3. Choose **Basic Auth** from the dropdown.
4. Enter:
 - a. **Username:** user (default username in Spring Security)
 - b. **Password:** (copy the auto-generated password from your app's console)
5. Click **Send**.

You'll now receive a **200 OK** response with Hello! — which means authentication was successful.

What's Really Happening?

When you select Basic Auth in Postman:

- Postman automatically creates an Authorization header behind the scenes.
- You can check that header in the hidden headers in the header tab in postman
- That encoded string is just your username:password, encoded in Base64.

You can decode it using any online tool to confirm it equals something like:

user:93a01cf0-794b-4b98-86ef-54860f36f7f3

Base64 ≠ Encryption

It's important to understand that Base64 is just encoding, not encryption. That means it can be easily decoded and is not secure on its own. It just makes the data safe for transmission in headers—not hidden from attackers.

What happens in browser?

When you enter your **username and password** in the browser (for HTTP Basic Authentication), here's what happens behind the scenes:

- The browser takes your input (e.g., user:password).
- It Base64 encodes that string.
- It sends the request with an Authorization header that looks like this:

Authorization: **Basic dXNlcjpwYXNzd29yZA==**

So yes — the **browser automatically does the Base64 encoding** and sends it to the Spring Boot app, which then uses **Spring Security** to check if the credentials are valid.

You don't see this process, but it's exactly the same as what you do manually

Why 401 ?

NOTE : The HTTP 401 Unauthorized status code is a bit ambiguous. Usually, it's used to represent a failed authentication rather than an authorization. Developers employ it in the design of the application for cases such as missing or incorrect credentials.

For a failed authorization, we'd probably use the 403 Forbidden status. Generally, an HTTP 403 means that the server identified the caller of the request, but they don't have the needed privileges for the call that they are trying to make.

Summary

- Spring Security uses HTTP Basic Auth by default.
- You need to send a Base64-encoded username:password in the Authorization header.
- Tools like Postman automate this, but it's useful to understand what happens under the hood.
- Remember: Base64 is not secure—it's just encoding.

Spring Security: Convention-over-Configuration

In **Spring Security, Convention-over-Configuration** means that the framework takes care of most of the setup for you (like using HTTP Basic authentication, generating default credentials, securing endpoints, etc.), letting you focus on what's truly important for your app. You only need to configure it when you have specific needs that go beyond the default setup.

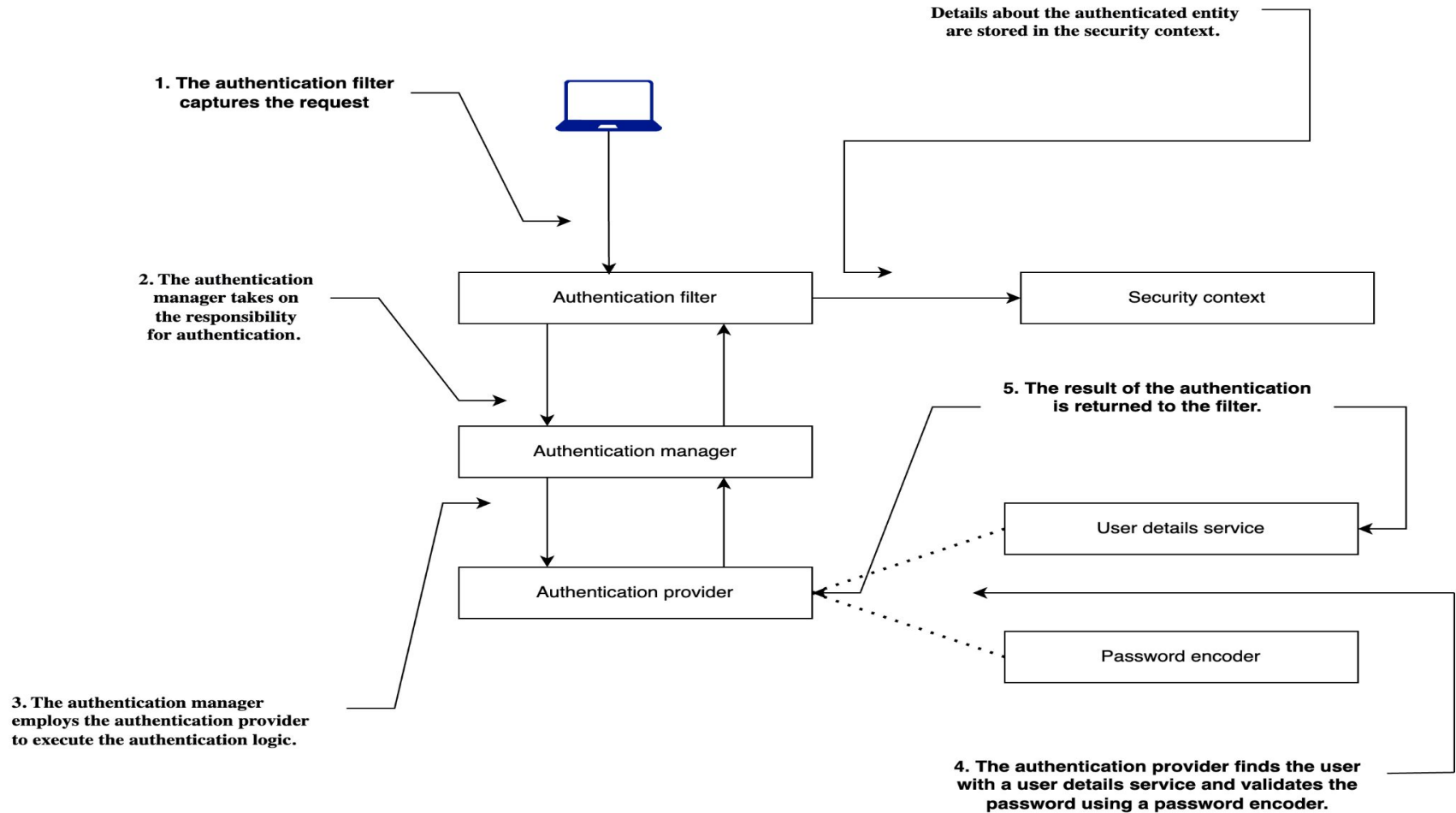
Spring Boot's sensible defaults ensure that **security** is set up for you right out of the box, but you have the flexibility to customize it when necessary.

Key Participants in Authentication Framework

We need to understand this part because we'll need to change some of the **built-in components** to make them work for your application.

This **diagram** gives an overview of the main parts (or components) in the Spring Security setup and how they work together.

These components have a preconfigured implementation in the first project.



Flow of Authentication in Spring Security

- The **authentication filter** forwards the authentication request to the authentication manager and sets up the security context based on the result.
- The **authentication manager** works with authentication providers to process the authentication request.
- **Authentication providers** contain the core logic that determines if authentication succeeds or fails.
- The **user details service** handles user information retrieval, which authentication providers use during their authentication process.

Flow of Authentication in Spring Security

- The **password encoder** manages password-related operations like hashing and verification, which authentication providers utilize during authentication.
- After successful authentication, the **security context** stores the authentication information for the duration of the request. In applications that handle one thread per request, this persists until the client receives the response.

Spring Security Auto-configured Beans

Lets first discuss these autoconfigured beans:

- UserDetailsService
- PasswordEncoder

UserDetailsService

- The **UserDetailsService** interface is responsible for managing user-related data within Spring Security.
- By default, Spring Boot provides a basic **in-memory** implementation of this interface.
- This default setup:
 - Registers a single user with the username "**user**"
 - Generates a random **UUID-based** password every time the application starts
 - Stores these credentials in the application's internal memory

PasswordEncoder

- The **PasswordEncoder** handles password-related operations in Spring Security. It has two main responsibilities:
 - **Encoding passwords** (typically using hashing or encryption algorithms)
 - **Verifying passwords** by comparing raw input with the stored, encoded value
- A PasswordEncoder is mandatory when using Basic Authentication.
- The simplest (and insecure) implementation stores passwords in plain text and performs no encoding.
- When you replace the default **UserDetailsService** with your own custom implementation (e.g., loading users from a database)
- You must also configure a **PasswordEncoder**, as the default one is no longer applied automatically.

Introduction to POC 2

Overriding Default Configurations

Spring Security comes with built-in components, but most real-world apps need customization.

In this POC, you'll learn how to:

- Replace the default **UserDetailsService** with your own implementation
- Add a custom **PasswordEncoder** bean

This gives you full control over user authentication and password handling, allowing you to apply security that fits your application.

Customizing Spring Security Configuration

- We'll override Spring Security's default setup using a configuration class.
- By annotating the class with **@Configuration** and using **@Bean**, we register custom beans in the Spring context.
- After this change, the autogenerated password will no longer appear in the console.
- Spring will use your custom **UserDetailsService** instead of the default.

Why Authentication Fails Now

- No users are defined.
- No PasswordEncoder is present.

Fixing Authentication Step by Step

- To restore functionality, we need to:
- Define a user with credentials (username & password).
- Register the user with our custom **UserDetailsService**.
- Add a **PasswordEncoder** bean to handle password validation.

Define User Credentials

- Use **InMemoryUserDetailsManager** to manage user details.
- Create a UserDetails instance using the builder from User (found in **org.springframework.security.core.userdetails.User**).
- Specify:
 - Username
 - Password
 - At least one authority (can be any string for now)
- **Note:** The authority represents permissions but is not critical at this point.

Adding User to InMemoryUserDetailsManager

- Once the user is created, register it using **InMemoryUserDetailsManager**.
- This replaces Spring's **default user storage** with your own.
- Spring no longer uses the autogenerated user.

Defining a PasswordEncoder Bean

- Since we override UserDetailsService, the default PasswordEncoder is no longer available.
- Add a bean to handle password encoding:
- **NoOpPasswordEncoder** stores passwords as plain text. Never use this in production.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}
```

Defining a PasswordEncoder Bean

...

@Configuration

public class ProjectConfig {

@Bean

UserDetailsService userDetailsService() {

UserDetails user = User.withUsername("john")

.password("12345")

.authorities("read")

.build();

return new InMemoryUserDetailsManager(user);

}

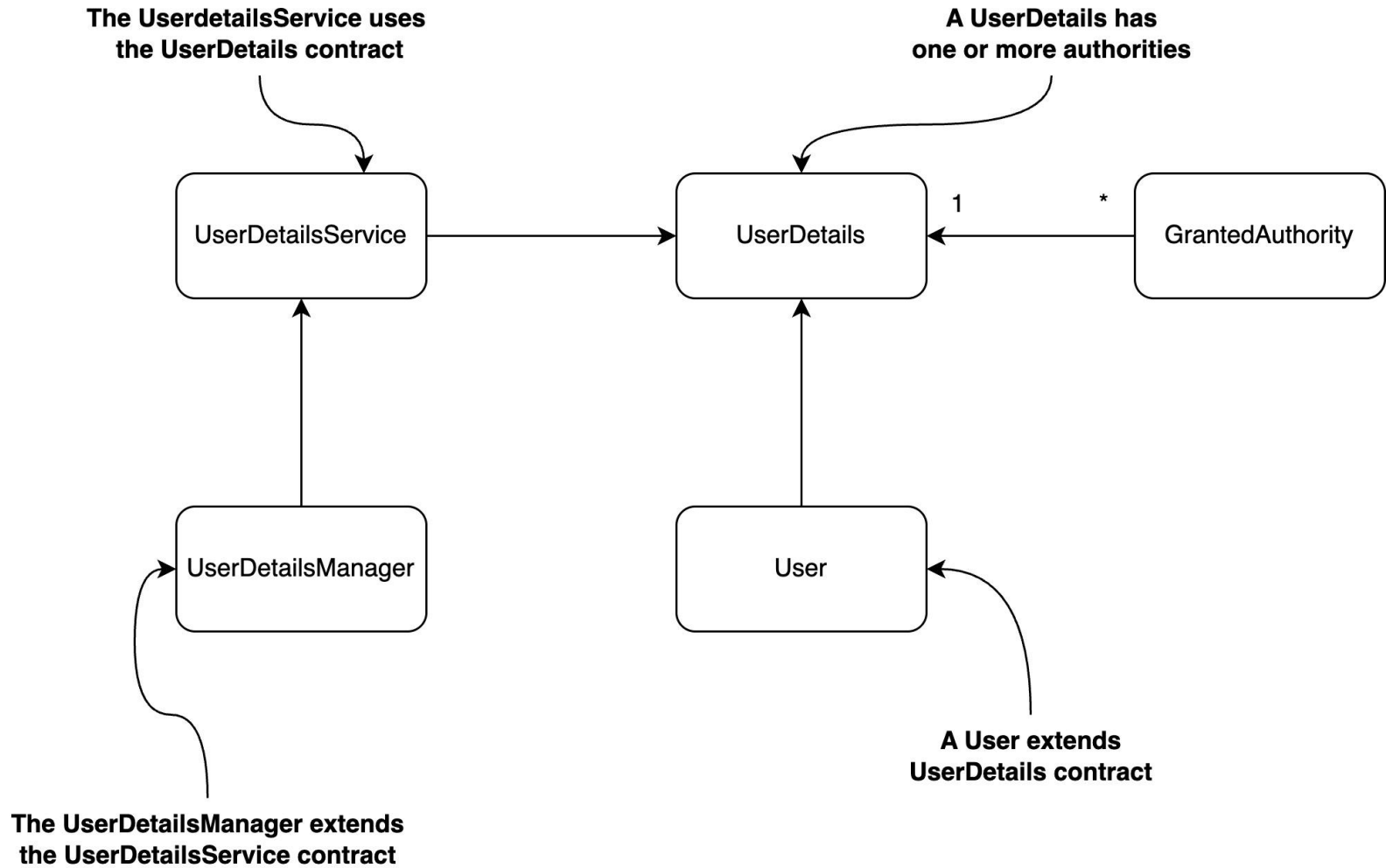
}

...

Why Avoid HTTP Basic Authentication?

- **HTTP Basic Authentication** lacks credential confidentiality.
- It uses **Base64 encoding**, which is merely a way to format data for transmission—not encryption or hashing.
- Because of this, credentials are easily **decodable if intercepted** during transit.
- This makes Basic Auth vulnerable when used **without HTTPS**, as sensitive data like usernames and passwords can be exposed.
- **Best Practice:** If HTTP Basic authentication must be used, it should always be secured over HTTPS to protect the transmitted credentials.

User Management



User Management Components

- **UserDetailsService** is used to load user-specific data by searching for the username.
- It returns a user object that adheres to the **UserDetails** contract.
- The **UserDetails** interface defines core user information such as username, password, and granted authorities.
- **Authorities** represent user permissions and are defined using the GrantedAuthority interface.
- **UserDetailsManager** extends UserDetailsService to provide user management operations like creating, deleting, and modifying passwords.
- The **User** class implements the UserDetails interface.
- The **User** class provides a builder to simplify the creation of UserDetails objects.

UserDetails

- In Spring Security, defining users is essential for handling authentication and controlling access to application features.
- The application must implement the **UserDetails** interface to represent users in a way the framework can understand.
- This interface includes methods like **getUsername()** and **getPassword()** for authentication, and **getAuthorities()** to define user permissions.
- It also includes methods to check account status such as **isAccountNonExpired()**, **isAccountNonLocked()**, **isCredentialsNonExpired()**, and **isEnabled()**.
- These methods help determine if a user account is valid; even though the method names might seem awkward, they follow a clear true/false logic for access control.

UserDetailsManager

- UserDetailsManager extends UserDetailsService by adding capabilities to manage user data beyond just retrieval.
- While UserDetailsService only retrieves a user by username for authentication, UserDetailsManager enables creating, updating, deleting users, and changing passwords.
- This separation aligns with the interface segregation principle, allowing apps to implement only what's necessary.
- Implementing UserDetailsService is sufficient for basic authentication, but most real-world applications also need user management, making UserDetailsManager more practical.
- The UserDetailsManager interface includes methods like createUser(), updateUser(), deleteUser(), changePassword(), and userExists() to support full user lifecycle operations.

User

- Spring Security provides a built-in `User` class to simplify creating instances of `UserDetails` without needing a custom implementation.
- You can use the static builder method `User.withUsername()` to start building a user instance by setting the username, password, and authorities.
- The builder allows chaining additional configurations like account expiration, disabled status, and password encoding.
- The final user instance is created using the `build()` method, which returns an immutable `UserDetails` object.
- This approach helps avoid cluttering your codebase with extra classes when a simple user representation is enough for the application.

Summary

- **AuthenticationProvider** verifies username and password using a **UserDetailsService** and a **PasswordEncoder**.
- **UserDetailsService** is used to retrieve user details based on the username.
- **UserDetailsManager** extends **UserDetailsService** to support user creation, modification, and deletion.
- **UserDetailsService** is focused solely on user retrieval, which is sufficient for authentication purposes.
- **UserDetailsManager** provides full user management functionality, commonly needed in applications.

- This design follows the **Interface Segregation Principle**, keeping responsibilities well-separated.
- **UserDetails** is a core interface in Spring Security that must be implemented to represent user information in a way the framework understands.

Customising User Details Service

- Till now we were saving all users in In memory
- But as the user base grows , can we store all of them by this hardcoding n in memory?
- Answer is No
- Then Can we store all of them in our DB?
- If we want that we have to customise UDS and write our logic to pull user info from db and store in User details Object



POC 3

Creating User & Authority Table

-

Mapping User & Authorities table

-

Why Authorities are eagerly fetched

-

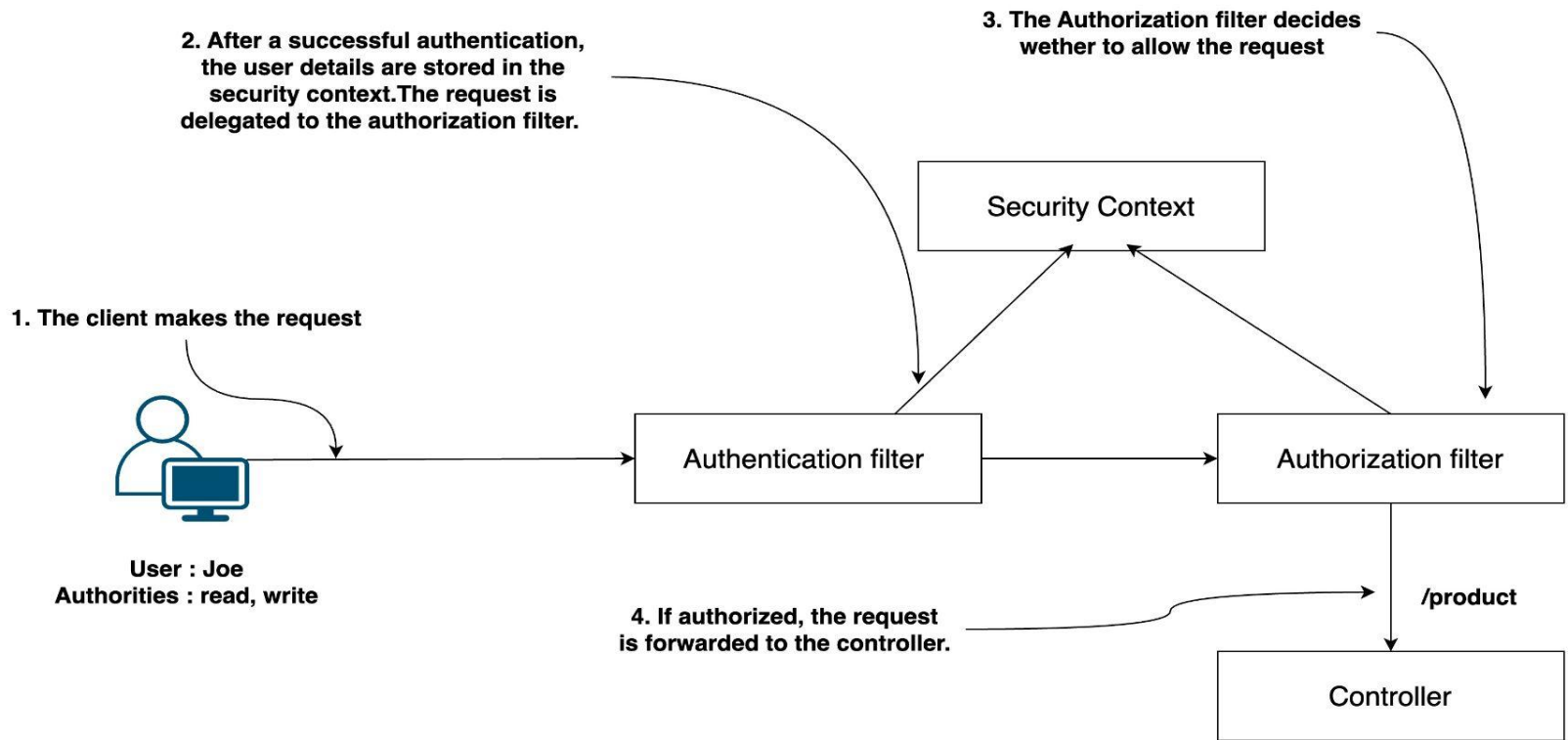
Fetch saved Authorities from SecurityContext

Authorization

Authorization

- **Authentication** is the process of verifying the identity of the caller accessing a resource. It ensures the system knows who the user is. Previous implementations only checked if the user was recognized
- No rules were applied to control access after identifying the user. In real-world applications, not all authenticated users can access all resources
- **Authorization** is the process of deciding whether an authenticated user has permission to access a specific resource
- It controls access based on the user's identity and assigned permissions
- **Authorization** ensures that users can only access what they are allowed to

- In Spring Security, **authorization** follows the completion of authentication
- Spring Security uses an **authorization** filter to handle access control
- The filter evaluates the request against the configured **authorization** rules
- Based on these rules, the filter either allows or denies the request

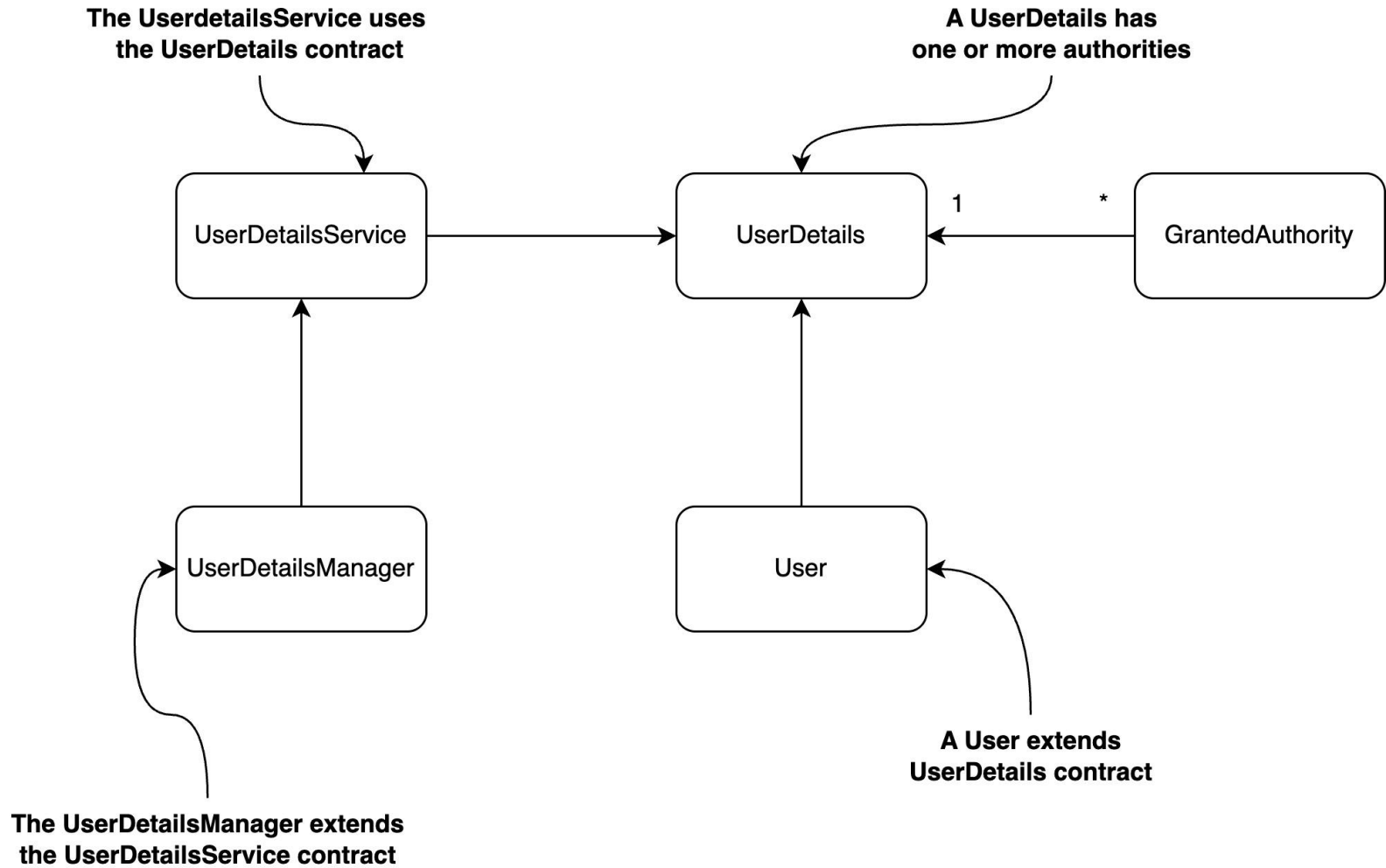


How Authorization works

- When a client initiates a request, the **authentication filter** verifies the user's identity
- After successful authentication, the user's details are stored in the **security context**
- The request is then forwarded to the **authorization filter**
- The **authorization filter** evaluates whether the request should be permitted
- This decision is based on the user information stored in the **security context**

What are we going to learn

- Understand the concept of an authority and how to enforce access control on all endpoints based on user authorities
- Learn how to organize authorities into roles and implement authorization rules using those roles

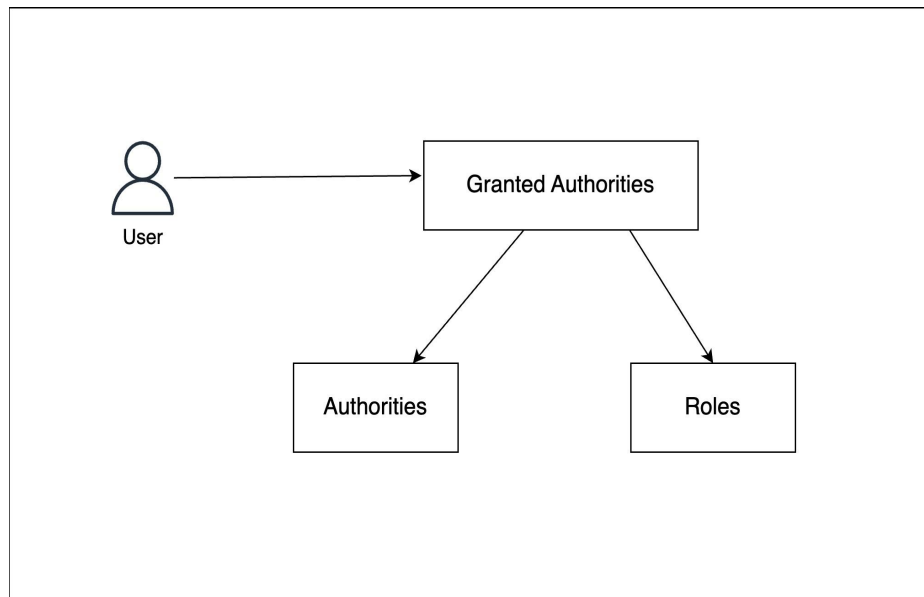


GrantedAuthority

- A user is assigned one or more authorities, which represent the actions they are allowed to perform
- During authentication, the UserDetailsService fetches complete user information, including their authorities
- Once authentication is successful, the application uses these authorities to make authorization decisions
- Authorities are represented in Spring Security through the GrantedAuthority interface

Difference between Authorities and Roles

- Granted authorities can be categorized as either authorities or roles
- Both are implemented using the same interface: `GrantedAuthority`
- The key difference between them is conceptual, not technical
- An authority represents a specific action, such as read, write, or delete — it's a verb
- A role represents a broader identity or responsibility, like Admin, Manager, or Guest — it's like a badge
- Roles are typically composed of multiple authorities



Authorization implementations level

- Authorization can be implemented at two levels within an application
- One level is at the endpoint level, where access to specific URLs or routes is controlled
- The other level is at the method or use case level, where access is restricted within the business logic itself

Endpoint Level Authorization

Security Filter Chain

- By default, all endpoints assume the user is valid and authenticated, granting access automatically
- Not every endpoint in an application requires security
- For those that need protection, different authentication methods and authorization rules may be necessary
- To customize authentication and authorization, we must define a **SecurityFilterChain** bean

Defining a Filter Chain

```
@Configuration no usages ⓘ codedecode25 *  
public class SecurityConfig {  
    @Bean no usages new *  
    SecurityFilterChain configure(HttpSecurity http)  
        throws Exception {  
        return http.build();  
    }  
}
```

Modifying Filter chain

```
@Bean no usages new *
SecurityFilterChain configure(HttpSecurity http)
    throws Exception {
    return http.authorizeHttpRequests((request) -> request
        .anyRequest().authenticated())
        .build();
}
```

Why still 403 ?

- Even after giving proper username password why it still fails?
- Reason is Security filter chain
- SecurityFilterChain is simply a collection of filters that sits between your HTTP requests and your application code.
- These filters work in a strict order.
- Because authorization depends on authentication. You have to add both filters in your filter chain

Modifying Filter chain

- The format used is `.anyRequest().authenticated`, which follows the structure: `(Matcher){rule}`
- This means that any request which is authenticated is also authorized to access the endpoint
- Both the matcher and the rule can be modified to customize the authorization
- Let's explore the available options for adjusting the matcher and rule

anyRequest().authenticated()

- To test this POC, follow these two steps:
- Step 1: Provide the correct username and password, send the request, and you should receive a 200 OK response
- Step 2: Switch to the Auth tab and select No Auth, then send the request — you should get a 401 Unauthorized
- If you still receive 200 OK, open the Cookies section (bottom right), remove the JSESSIONID, and resend the request
- You should now receive the expected 401 Unauthorized response

anyRequest().permitAll()

- The permitAll() rule means access to the endpoint is allowed, regardless of whether the user is authenticated or not
- There are three possible scenarios when testing this:
 - You are **not authenticated** (e.g., selecting "No Auth" in Postman) — you receive 200 OK
 - You are **authenticated** with valid username and password — you receive 200 OK
 - You are **not in the system** or not present in the security context — this is the tricky case
- In the third case, although permitAll() allows access, a 401 Unauthorized is returned

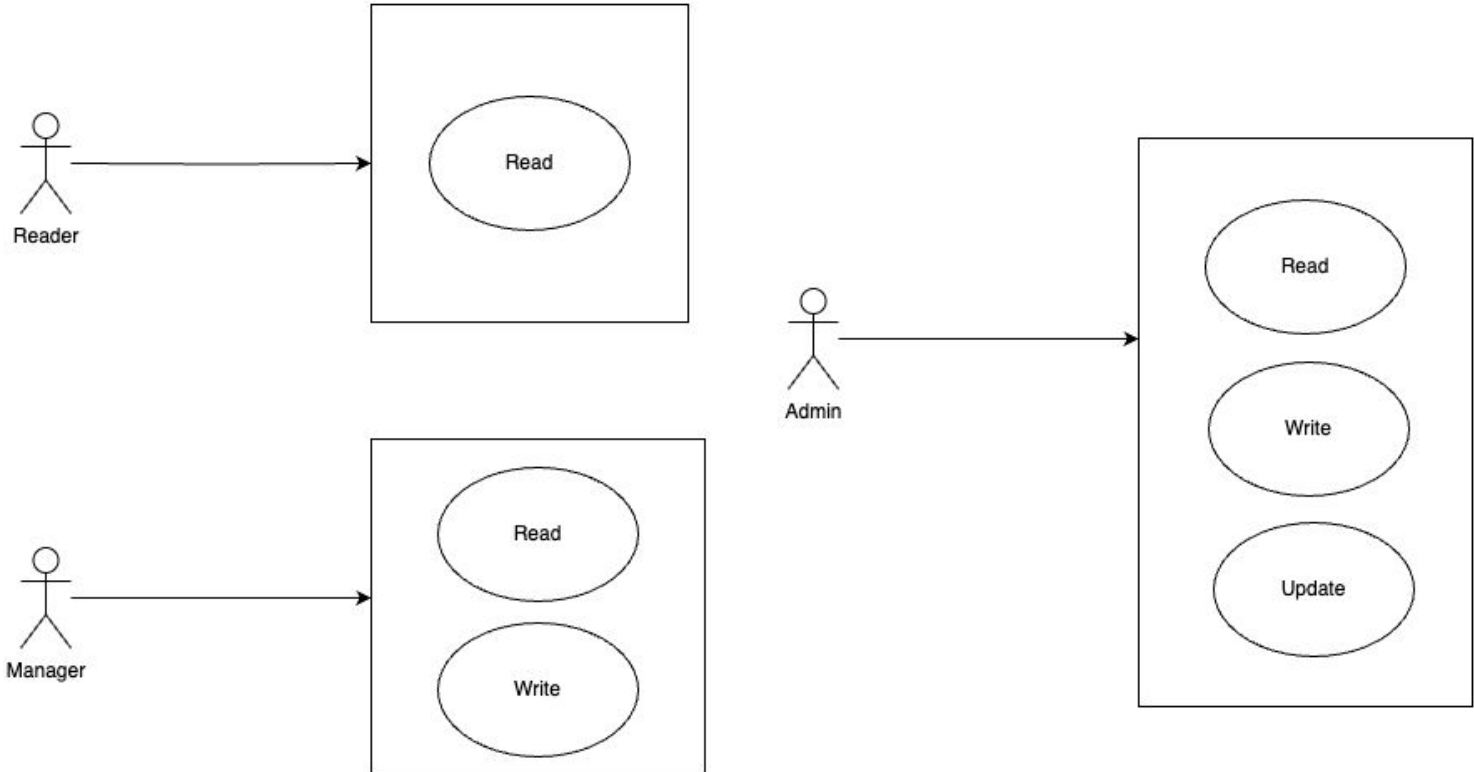
anyRequest().hasAuthority()

- We want to ensure that only users with READ or WRITE authority can access all endpoints. This can be enforced by restricting access based on user authorities.
- To do this, you pass the required authority name as a parameter to the hasAuthority() method.
- The application first authenticates the request, and then determines access based on the user's granted authorities.

anyRequest().hasAnyAuthority()

- We can use the `hasAnyAuthority()` method, which accepts a variable number of authority names (varargs).
- This allows us to specify multiple authorities at once.
- The application grants access if the user possesses at least one of the specified authorities.
- For example, you can replace `hasAuthority()` with `hasAnyAuthority("WRITE")`—this behaves the same as before.
- However, if you use `hasAnyAuthority("WRITE", "READ")`, then users with either authority will be allowed access.

Role



anyRequest().hasRole()

- Spring Security treats authorities as specific, detailed permissions that you use to restrict actions. Roles are like Proper Nouns that represent a set of these permissions grouped together.
- In many applications, users often get the same group of authorities. For example, a user might have only READ authority, or they might have all authorities like READ and WRITE,. In this case, it's easier to think of users with only read permission as having the READER role, and users with all permissions as having the ADMIN role.
- Having the ADMIN role means the application lets you read, write, update, and delete data. You can also create more roles if needed. For example, if you want a user who can only read and write, you can make a MANAGER role.
- Note: When using roles, you don't need to define individual authorities separately. Authorities still exist as a concept and can be part of requirements, but in your app, you just define roles that cover one or more actions.

anyRequest().hasRole()

- The names you choose for roles are up to you, just like authorities. Roles are considered coarse-grained compared to authorities because they group several permissions. Under the hood, roles use the same system as authorities in Spring Security, specifically the `GrantedAuthority` interface.
- When defining a role, its name should start with the prefix `ROLE_`. This prefix helps distinguish roles from individual authorities in the implementation.
- `.authorities("ROLE_ADMIN")`: Having the `ROLE_` prefix, `GrantedAuthority` now represents a role.
- `.hasRole("ADMIN")`: The `hasRole()` method now specifies the roles for which access to the endpoint is permitted. The `ROLE_` prefix does not apply here

anyRequest().hasRole()

- You specify roles by using the roles() method. This method automatically creates the GrantedAuthority objects and adds the ROLE_ prefix to the role names you provide.
- Make sure not to include the ROLE_ prefix in the parameter you pass to the roles() method. If you accidentally include the ROLE_ prefix in the roles() parameter, the method will throw an exception.
- In summary:
- When using the authorities() method, include the ROLE_ prefix.
- When using the roles() method, do not include the ROLE_ prefix.

`anyRequest().hasAnyRole()`

- .

401 VS 403

- 401 Unauthorised - happens when Authentication fails.
- 403 Forbidden - Authorization failed

The HTTP 401 status code is officially labeled “Unauthorized”, but in practice, it means “Unauthenticated” — that is, the client has not provided valid authentication credentials.

The name “Unauthorized” for the 401 error is a bit old-fashioned and confusing (historic naming quirk). It really means the user is not logged in yet. Even though the name sounds like it’s about permission, it’s actually about needing to prove who you are first.

Because of this confusing name, many websites and tools explain it clearly in their guides so people don’t get mixed up.

anyRequest().access()

- The 1st method but a lot more powerful but difficult to debug
- Here you can use SpEL - Spring expression language to implement the authorization rules
- Say access is granted only if user is authenticated and hasAuthority('read')
- It can also call a method from dto or bean also, it in turn should return a boolean
- Situation when we can use spel -
 - there are two users, code and code1, who have different authorities.
 - The user code has only read authority, while code1 has write authorities.
 - The endpoint should be accessible to those users who have read authority, but not to those that have write authority.

```
String expression = "isAuthenticated() and hasAuthority('read')"  
.access(new WebExpressionAuthorizationManager(expression));
```

Advantage of `anyRequest().access()`

- **Powerful and Flexible Logic**

You can write complex, fine-grained access control expressions, combining multiple conditions:

```
hasAuthority('read') and !hasAuthority('delete')
```

- **Bean/DTO Support**

You can invoke methods from beans or request DTOs:

```
@PreAuthorize("@myService.canAccessResource(#dto)")
```

- **Context-Aware Decisions**

Expressions can access authentication details, request details, or even custom beans, making them very context-sensitive.

- **Declarative Style**

Keeps security logic separate from business logic, making the controller/service methods cleaner.

Advantage of anyRequest().access()

- **User-specific Rules**

Only allow users with 'read' but not 'delete' access:

```
.access("hasAuthority('read') and !hasAuthority('delete')")
```

Disadvantage of `anyRequest().access()`

- **Difficult to Read and Maintain**

Complex SpEL expressions can quickly become hard to understand for teams unfamiliar with the syntax.

- **Harder to Debug**

Errors in expressions (e.g. typo in authority name) won't always show clear messages. You'll only get generic "Access Denied".

- **No IDE Support or Compilation Check**

SpEL is just a string, so you don't get compile-time safety or refactoring support from the IDE.

- **Tight Coupling to Authorities or Roles**

Logic tied directly to string-based authority names. A simple rename of a role requires a string update across multiple places.

- **Misuse Risk**

Misconfigured SpEL might accidentally allow or deny access incorrectly.

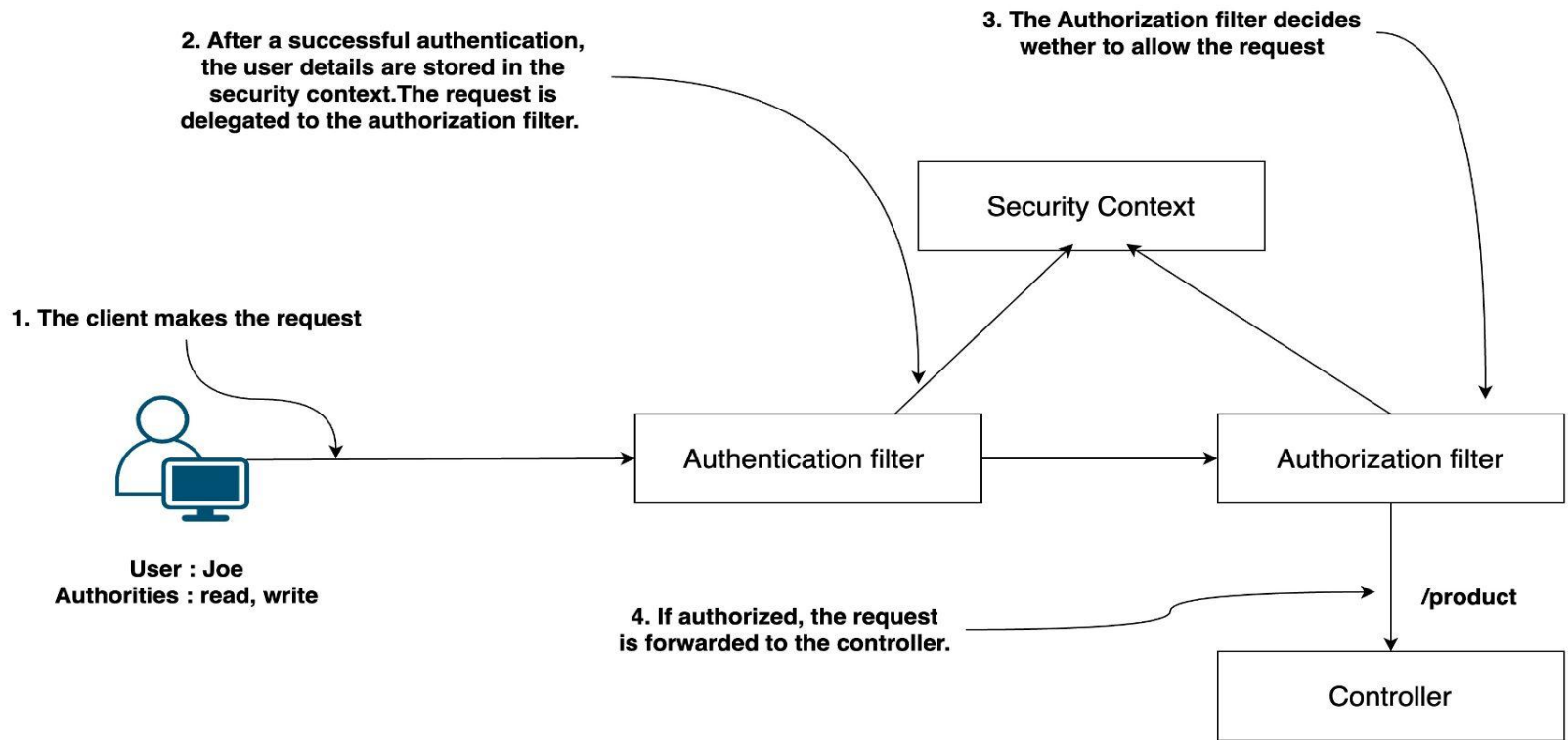
anyRequest().denyAll()

- During site maintenance, only /status should be available. All other requests should be denied.

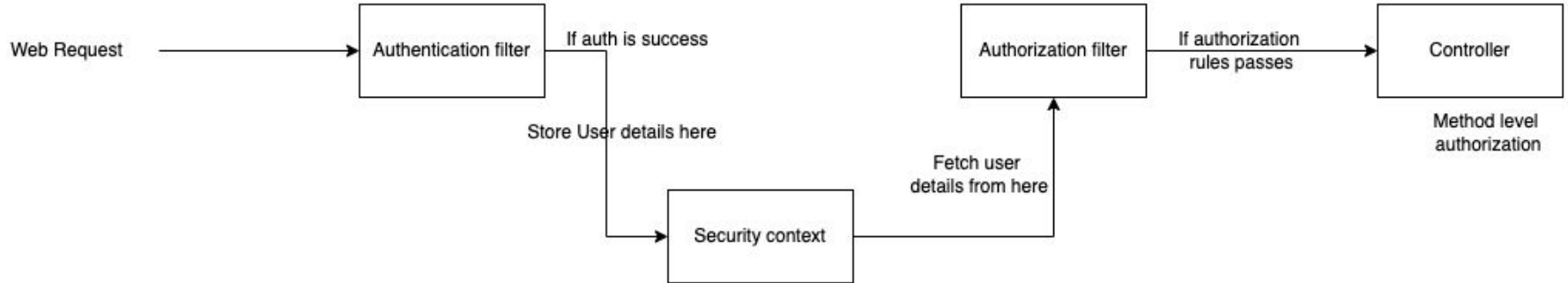
http

```
.authorizeHttpRequests(auth -> auth  
    .requestMatchers("/status").permitAll()  
    .anyRequest().denyAll()  
);
```

Matcher Methods



Matcher Methods



Matcher Methods

- In production applications, it's unlikely that the same rules will apply to all requests. Certain endpoints may be accessible only to specific users, while others can be open to everyone. Based on the business requirements, each application generally defines its own custom authorization configuration.
- **anyRequest()** - it refers to all requests, regardless of the path or HTTP method. It is the way to say "any request" or, sometimes, "all requests."

List of All Matcher Methods

- Request Matcher - The All-Rounder (Preferred)
- antMatcher - The Old Default - Deprecated
- mvcMatcher - The MVC-Aware Matcher - Deprecated
- regex matchers - The Pattern Master
- DispatcherTypeMatchers – The Gatekeeper for Internals
- Servlet Path Matcher - Servlet Path Specialist

Request Matcher

Request Matcher Methods

- request-Matchers() method is a common approach to refer to requests for applying authorization configuration..
- This matcher uses the standard ANT syntax for referring to paths.
- Purpose:
 - Match incoming requests using:
 - URL patterns (/admin/**)
 - HTTP methods (GET, POST)
 - Both at the same time
- Why it's preferred:
 - It replaces the old antMatchers() and mvcMatchers() (both deprecated).
 - Supports Ant-style patterns (/path/**).
 - Can match multiple patterns in one go.

Request Matcher Methods

- The syntax is the same as the one you use when writing endpoint mappings with annotations such as `@RequestMapping`, `@GetMapping`, `@PostMapping`, and so forth.
- The two methods you can use to declare MVC matchers are
 - **`requestMatchers(HttpMethod method, String... patterns)`**—
 - Lets you specify both the HTTP method to which the restrictions apply and the paths.
 - This method is useful if you want to apply different restrictions for different HTTP methods for the same path.
 - **`requestMatchers(String... patterns)`**—Simpler and easier to use if you only need to apply authorization restrictions based on paths. The restrictions can automatically apply to any HTTP method used with the path.

Real-life analogy

Restaurant Buffet Analogy — requestMatchers()

Imagine you run a huge buffet restaurant with many sections:

Salad bar

Main course counter

VIP lounge

Kitchen

Complaint desk

Your waiters are like Spring Security — they check what a guest is trying to access and apply rules.

How requestMatchers() works in this setting

You give your waiters rules like:

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/vip/**").hasRole("VIP")           // Only VIP guests in VIP lounge
    .requestMatchers(HttpMethod.GET, "/menu/**").permitAll() // Anyone can *see* the menu
    .requestMatchers("/kitchen/**").denyAll()           // No guest in the kitchen
    .anyRequest().authenticated()
);
```

In the buffet, you don't check everyone for everything.

You match the section they're going to (and sometimes how they're getting there) — that's exactly what requestMatchers() does in Spring Security.

So whenever a guest (HTTP request) tries to go somewhere (access a URL), Spring Security checks against all your requestMatchers() rules to decide what to do.

Code Block

```
http
    .authorizeHttpRequests(auth -> auth
        // Match by URL only
        .requestMatchers("/public/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/admin/**").hasRole("ADMIN")
        .requestMatchers(HttpMethod.POST, "/admin/**").hasRole("SUPER_ADMIN")
        .anyRequest().authenticated()
    )
    .formLogin(withDefaults());

return http.build();
}
```

Ant Matcher

ANT Matcher Methods

- Match HTTP requests using Ant-style path patterns (like file wildcards in build tools).
- These patterns come from Apache Ant (a Java build tool), which used wildcards to match file names — Spring Security reused this same style to match URL paths.
- Example patterns:
 - `/foo/*` → Matches `/foo/bar` but not `/foo/bar/baz`
 - `/foo/**` → Matches `/foo/bar`, `/foo/bar/baz`, `/foo/anything`
 - `/foo/*.html` → Matches `/foo/index.html`, `/foo/help.html`

Example

. /foo/* — One Level Deep

/foo

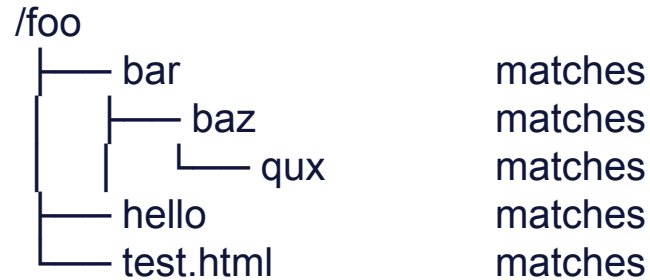
— bar	matches
— hello	matches
— bar/baz	too deep
— test.html	matches

Rule:

- * matches any file/folder name exactly one level under /foo/.
- It will not go deeper than 1 folder.

Example

/foo/** — Any Depth



Rule:

- ** matches anything at any depth inside /foo/.
- Think of it like “give me the whole tree under /foo/”.

Example

/foo/*.html — One Level, Specific Extension

/foo

— index.html	matches
— help.html	matches
— about.html	matches
— bar/index.html	too deep
— image.png	wrong extension

/foo/?*.html — Single Character Name

/foo

— a.html	matches
— b.html	matches
— ab.html	too many characters
— 1.html	matches

Why it was popular

- Simple and easy to read.
- Covered most use cases for securing REST endpoints

Example in Spring Security 5.x

@Configuration

@EnableWebSecurity

public class SecurityConfig extends WebSecurityConfigurerAdapter {

 @Override

 protected void configure(HttpSecurity http) throws Exception {

 http

 .authorizeRequests()

 .antMatchers("/public/**").permitAll()

 .antMatchers("/admin/**").hasRole("ADMIN")

 .antMatchers(HttpMethod.POST, "/api/**").hasRole("API_USER")

 .anyRequest().authenticated()

 .and()

 .formLogin();

 }

}

Real-life analogy:

- Post Office Address Matching
- Think of your URLs like postal addresses.
 - `/foo/*` → “Deliver to any house number on Foo Street, but not to houses inside gated communities.”
 - `/foo/**` → “Deliver to any address inside Foo City, no matter how many neighborhoods deep.”
 - `/foo/*.html` → “Deliver only to houses on Foo Street whose name ends with `.html` (like houses painted red).”

Why Deprecated in Spring Security 6+:

- Limited flexibility (only Ant-style patterns, not regex).
- Inconsistent behavior with Spring MVC path matching rules.
- Replaced by `requestMatchers()` which is more general.

MVC Matcher

MVC Matcher Methods

- `mvcMatchers()` was introduced so Spring Security could match URLs the exact same way that Spring MVC matches them in your `@RequestMapping` annotations.
- This means:
 - It understands path variables (`/user/{id}`)
 - It respects servlet context paths (important if your app is deployed under something like `/myapp` instead of `/`)
 - It uses the same underlying path matching strategy that MVC uses (used to be `AntPathMatcher`, now `PathPatternParser` in Spring 6).

If you used `antMatchers("/api/user/{id}")`, it would not understand `{id}` — it would literally look for a path containing `{id}`.

`mvcMatchers("/api/user/{id}")` would understand `{id}` as “match anything in that position” because that’s how MVC does it.

Why it was used

- Better alignment between your Spring MVC `@RequestMapping` and your security config.
- Worked with servlet mapping offsets (important in WAR deployments).

Example in Spring Security 5.x

@Configuration

@EnableWebSecurity

public class SecurityConfig extends WebSecurityConfigurerAdapter {

 @Override

 protected void configure(HttpSecurity http) throws Exception {

 http

 .authorizeRequests()

 .mvcMatchers("/").permitAll()

 .mvcMatchers("/admin").hasRole("ADMIN")

 .mvcMatchers(HttpMethod.POST, "/posts/**").hasRole("EDITOR")

 .anyRequest().authenticated()

 .and()

 .formLogin();

 }

}

Real-life analogy

Think of `MvcMatchers()` like the GPS system inside the delivery van.

- The controller's `@RequestMapping` is the delivery address.
- MVC knows exactly how to get there, even if there's a "wildcard" in the address like:
- 123 Main Street, Apt {apartmentNumber} → It knows to match any apartment number.
- `MvcMatchers()` talks to the same GPS system so security checks happen on the exact same rules as the delivery route.

Without it, you'd be using a paper map (`antMatchers`) that doesn't understand your GPS's shortcuts, toll avoidance, or variable routes.

Why Deprecated in Spring Security 6+:

- Increased complexity in deciding whether to use Ant or MVC matching.
- `requestMatchers()` can now work with custom matchers to replicate MVC matching if needed.
- The Spring team wanted one unified matcher API.

How They Map to Spring Security 6

Old Method	New Equivalent in Spring Security 6
<code>antMatchers()</code>	<code>requestMatchers("/path/**")</code>
<code>mvcMatchers()</code>	<code>requestMatchers(new MvcRequestMatcher(...))</code> if you want MVC-like behavior

Regex Matcher

regexMatchers()

- Purpose: Match requests with regular expressions instead of simple wildcards.

Why use it

- When Ant patterns (/**) are not enough.
- When you need complex matching logic like v1, v2 in API paths.

Code Example

@Bean

public SecurityFilterChain regexMatcherExample(HttpSecurity http) throws Exception {

```
    httpSecurity
        .csrf(
            c -> c.disable()
        )
        .httpBasic(Customizer.withDefaults())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(new RegexRequestMatcher("/api/v[0-9]+", null))
            .hasRole("USER")
            .anyRequest().authenticated()
        ).build();
```

}

Real-life analogy

`antMatchers("/foo/*")` → Like searching only for people whose name is exactly Foo and then only 1 thing.

`regexMatchers("/api/v[0-9]+/users")` → Like saying “find any /api/ path where v is followed by one or more digits, then /users” → very precise and works even if /v12/users shows up tomorrow.

Dispatcher Type Matcher

Purpose - What is DispatcherType

In a Java web application (Servlet-based, like Spring MVC), every HTTP request passes through the Servlet container (like Tomcat, Jetty).

The DispatcherType tells why and how the request is being processed.

Think of it like the "purpose tag" attached to the request when it enters your web app.

DispatcherType = the purpose or phase of the request in the servlet lifecycle.

Without it, Spring Security treats all requests the same, even if some are just internal forwards or error handlers.

Purpose - What is DispatcherType

The main types are:

REQUEST → A normal browser request (user clicks a link, submits a form).

FORWARD → Request is internally forwarded to another resource inside the server (browser doesn't know).

INCLUDE → A resource includes another one (like a JSP including a header or footer).

ERROR → Request comes from the error-handling mechanism (e.g., 404 page).

ASYNC → Request being processed in async mode (Servlet 3.0 async support).

Real-life analogy:

DispatcherType	Real-life analogy
REQUEST	A customer comes directly to the hub to drop off a package (normal request).
FORWARD	A package is already in the hub, but a worker says “Oops, wrong section, forward it to the right department” (server forwards internally).
INCLUDE	A customer asks, “Can you add a gift note?” and the hub pulls a card from another section to include it (server includes other resources).
ERROR	Package got damaged or wrong label — it’s rerouted to the complaint desk (error handling).
ASYNC	Customer says, “Process this later, I’ll come back for updates” (async processing).

Why use it

- Some requests are not from the browser but are internal (e.g., forwarded to error pages).
- Useful for error handling, internal redirects, or security exceptions.

Example

@Bean

```
public SecurityFilterChain dispatcherMatcherExample(HttpSecurity http) throws Exception {
```

```
    http
```

```
        .authorizeHttpRequests(auth -> auth
```

```
            // Allow access to error pages (otherwise 403 loop can happen)
```

```
            .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
```

```
            // For normal browser requests
```

```
            .requestMatchers("/secure/**").authenticated()
```

```
            .anyRequest().permitAll()
```

```
        )
```

```
        .formLogin(withDefaults());
```

```
    return http.build();
```

```
}
```

Servlet Path Matcher

Purpose

When a request comes into your app, the server breaks the URL into pieces:

Example

`http://localhost:8080/app/api/users?id=5`

The servlet container (Tomcat, Jetty, etc.) splits it like this:

Part	Value
Context Path	<code>/app</code>
Servlet Path	<code>/api</code>
Path Info	<code>/users</code>
Query String	<code>id=5</code>

Servlet Path Matcher means:

"Match requests using only the servlet path portion (`/api` in this case), ignoring everything after it (path info, query params)."

Real-life analogy:

Imagine a shopping mall:

Context Path = City where the mall is (/app)

Servlet Path = The main floor (/api)

Path Info = Specific shop inside (/users)

Query String = Details about your visit (id=5)

If the mall's security guard is told:

“Only check people entering the main floor (/api), don't worry about which shop they go into,”
— that's servlet path matching.

Why use it

- When you have servlet mappings or filters that modify paths.
- Rare but helpful in legacy systems or multi-servlet apps.

Is it any relevant in spring boot app?

In most Spring Boot apps:

- Servlet Path defaults to "" (empty string) because Spring MVC uses the DispatcherServlet mapped to / (everything).
- That means the whole URL path (/testMe/v1) is usually considered path info, not the servlet path.
- So if you try `.servletPathMatchers("/testMe")`, it won't match anything — because /testMe is not your servlet path.

When its relevant

`.servletPathMatchers()` is only useful when you:

- Customize servlet mapping in `application.properties`:

```
server.servlet.path=/api
```

Then `/api` becomes the servlet path, and `.servletPathMatchers("/api")` will match.

- Have multiple servlets in the same app and you want security rules per servlet.

If your app is just a normal Spring Boot REST API and you haven't changed:

```
server.servlet.path
```

then **`.requestMatchers("/testMe/**")`** is what you want — not `.servletPathMatchers()`.

Example

@Bean

```
public SecurityFilterChain servletPathMatcherExample(HttpSecurity http) throws Exception {
```

```
    http
```

```
        .authorizeHttpRequests(auth -> auth
```

```
            // If servlet path is /internal-api/**, deny all
```

```
            .servletPathMatchers("/internal-api/**").denyAll()
```

```
            // Other requests
```

```
            .anyRequest().permitAll()
```

```
        )
```

```
        .formLogin(withDefaults());
```

```
    return http.build();
```

```
}
```

Combining all Matcher methods

http

```
.authorizeHttpRequests(auth -> auth
    // 1. requestMatchers (Preferred)
    .requestMatchers("/public/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/admin/**").hasRole("ADMIN")

    // 2. regexMatchers (For complex patterns)
    .regexMatchers("/api/v[0-9]+/users").hasRole("USER")

    // 3. dispatcherTypeMatchers (For special dispatch cases)
    .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()

    // 4. servletPathMatchers (Servlet path only)
    .servletPathMatchers("/internal-api/**").denyAll()
```

Combining all Matcher methods

```
        // Default
        .anyRequest().authenticated()
    )
    .formLogin(withDefaults());

return http.build();
}
}
```

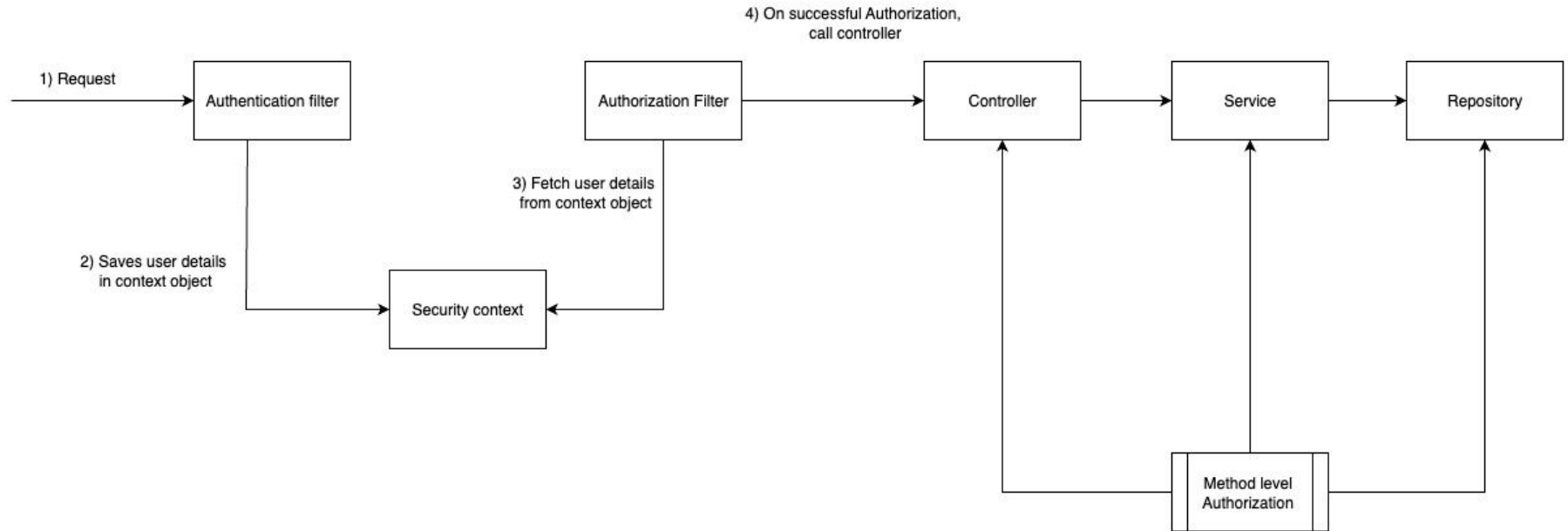
Summary

Matcher Type	Matching Style	Deprecated	Spring Security 6+ Support*
requestMatchers()	Flexible & preferred	NO	YES
antMatchers()	Ant-style (e.g., /admin/**)	YES	NO
mvcMatchers()	Spring MVC-style matching	YES	NO
regexMatchers()	Java Regex pattern matching	YES	NO
servletPathMatchers()	Matches only the Servlet Path part of the URL (ignores path info).	NO	YES
dispatcherTypeMatchers()	Matches based on DispatcherType (REQUEST, FORWARD, INCLUDE, ERROR, ASYNC).	NO	YES



Authorization at the method level

Where do we stand now?



Can Spring Security Be Used in Non-Web Applications?

So far, we've mostly looked at HTTP filter-based solutions in Spring Security. Those work perfectly for web applications, where every request comes through HTTP endpoints and can be intercepted by filters.

But what if our application is not web-based?

Can we still use Spring Security for authentication and authorization?

The answer is Yes. Spring Security isn't limited to web apps.

For non-web applications, Spring Security provides method-level security, which allows us to enforce authorization rules even when there are no HTTP endpoints at all.

This means you can protect access to your application's logic directly at the method level.

Where Can You Apply Method Security?

With Spring Security's method security annotations (`@PreAuthorize`, `@PostAuthorize`, `@Secured`, `@RolesAllowed`), you can apply fine-grained authorization to any Spring bean method, such as:

- Controller methods – to secure endpoints in a REST or MVC controller.
- Service methods – to protect business logic directly, regardless of how it's called.
- Repository methods – to restrict access to certain database operations.

Why Use Method Security?

- Works in both web and non-web applications.
- Adds an extra layer of protection, not just at the boundary (HTTP filters) but also deep inside your app.
- Provides flexibility: you can secure different layers (controller, service, repository) based on your requirements.

In short, Spring Security is not only for web filters. By enabling method-level security, you can apply authentication and authorization rules across your entire application—web-based or not.

Role of Authentication in Enabling Method Security

Method-level authorization still depends on the SecurityContext.

The SecurityContext holds the authenticated user's details (username, roles, authorities).

Hence, authentication is always a **prerequisite** for authorization to work.

In other words:

Authentication = Who you are

Authorization = What you can do

Method-level rules check authorization, but only after authentication has populated the SecurityContext.

Why Not Use `permitAll()` with Method Security

When configuring Spring Security, it's important to understand how request authorization impacts authentication and the `SecurityContext`.

Using `permitAll()`

```
http.authorizeHttpRequests()  
    .anyRequest().permitAll();
```

This configuration allows all requests to pass through without authentication.

As a result, the `SecurityContext` remains empty because no user is authenticated.

Since method-level authorization (e.g., `@PreAuthorize`, `@RolesAllowed`) relies on user details in the `SecurityContext`, it will fail in this case.

Why Not Use `permitAll()` with Method Security

Using `authenticated()`

```
http.authorizeHttpRequests()  
    .anyRequest().authenticated();
```

This configuration ensures that every request must be authenticated.

After successful authentication, the `SecurityContext` is populated with the user's identity and roles.

Now, method-level authorization can use this information to enforce rules correctly.

Code snippet

```
@RestController
@RequestMapping("/products")
public class ProductController {

    // Only ADMIN role can delete a product
    @PreAuthorize("hasRole('ADMIN')")
    @DeleteMapping("/{id}")
    public String deleteProduct(@PathVariable Long id) {
        return "Product " + id + " deleted!";
    }
}
```

Enabling method security

By default, method security is disabled, so if you want to use this functionality, you first need to enable it.

In older versions (pre–Spring Security 6), we used:

```
@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true)
```

This is deprecated now.

New way of enabling Method level Authorization

Use **@EnableMethodSecurity** instead:

@Configuration

@EnableMethodSecurity // enables @PreAuthorize, @PostAuthorize, @Secured, @RolesAllowed
public class SecurityConfig {

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {  
    return http  
        .authorizeHttpRequests(auth -> auth  
            .anyRequest().authenticated()  
        )  
        .formLogin(withDefaults())  
        .build();  
}
```

What Happens Behind the Scenes

When you use `@EnableMethodSecurity`, Spring registers AOP proxies (aspects) around your beans. These proxies intercept method calls and evaluate security expressions before or after the method runs.

When you call `deleteProduct(1L)` → you're not calling the method directly.

You're calling the proxy generated by Spring Security.

That proxy first evaluates the expression `(hasRole('ADMIN'))` against the `SecurityContext`.

- If the check passes → the actual method executes.
- If the check fails → a `403 AccessDeniedException` is thrown.

Why Called “Aspect Behind the Scene”?

- Because Spring uses method interception (AOP) under the hood:
- `@PreAuthorize` → Before advice
- `@PostAuthorize` → After advice
- `@PreFilter` / `@PostFilter` → Around advice (modifies inputs/outputs)
- So you can think of it as Spring automatically “wrapping” your methods with security aspects that enforce rules.

Key Points

@EnableMethodSecurity is the replacement for @EnableGlobalMethodSecurity.

By default, it enables:

@PreAuthorize / @PostAuthorize / @PreFilter / @PostFilter - so now these annotations are evaluated by aspect behind the scene

If you want to enable other 2 , you can pass flags:

```
@EnableMethodSecurity(  
    prePostEnabled = true,    // for @PreAuthorize / @PostAuthorize - by default true  
    securedEnabled = false,  // for @Secured - by default false - disable @Secured  
    jsr250Enabled = false    // for @RolesAllowed - by default false - disable @RolesAllowed  
)
```

Prevent GOD class with Method level Authorization?

So why we use method level security so extensively in enterprise level applications

Suppose you have a monolith or a very large microservice then we will have 100s of urls.

Then u will end up configuring all 100 urls in 1 file , making the security file a GOD Class

GOD Class in Java - It refers to an anti-pattern, where:

- A single Java file (class) becomes too big.
- Everything is dumped into one place, making it a “God” because it knows too much and does too much.

Best Practice

So if you have more than 10 endpoints in a service and u need to apply rules on them then

- 1) Don't add rules and paths in security config file
- 2) Just add Authenticated rule on all endpoints
- 3) Apply authorization rules on endpoints or methods where u need the rule to be applied directly

Priority of Rules: Security Config vs Method-Level Authorization

If you apply rules both in Security Config and at the method level:

Security filter rules (from config) take priority.

This is because filters are evaluated before the method call in the Spring Security chain

Flow

- Request enters the filter chain (configured in SecurityConfig).
- If the rule fails → Spring immediately returns 403 Forbidden.
- The method is never called.
- If the request passes the filter → then method-level annotations (@PreAuthorize, @Secured, etc.) are evaluated.

Summary

- Filter rules (config) = first line of defense.
- Method-level annotations = second layer, applied only if the request passes through filters.

Performance Consideration: Method-Level vs Filter-Level Authorization

Filter-Level Authorization

- Applied in the security filter chain (before controller/method invocation).
- Lightweight and faster since it directly checks authorization without additional overhead.

Method-Level Authorization

- Implemented using AOP (aspects) in Spring.
- Adds an extra layer of processing (proxy + aspect evaluation).
- Slightly slower compared to filter-level checks.

Filter-level rules are **more performant**.

Method-level rules **provide finer-grained control** but come with a small **performance cost**.

But much **more cleaner and maintainable** hence developers prefer method level authorization

How Method-Level Security Goes Beyond Filters

- Method-level authorization runs when your controller/service method is called. At this point, it can see the method arguments (like path variables, query params, or request body).
- They operates closer to the business logic, meaning it can directly access method arguments such as path variables, request parameters, or request bodies. This allows developers to write fine-grained security rules — for example, checking whether the logged-in user's ID matches the userId passed in the request before allowing the operation.
- Filter-level authorization runs **earlier in the request lifecycle** (in SecurityConfig). It only knows about the user's authentication details (username, roles, authorities) from the SecurityContext.
- **It cannot see method arguments**, Since it does not know about the actual method call or its arguments, it can only enforce coarse-grained, role-based access control — like “only users with ADMIN role can access this endpoint.”

Code Snippet

```
@PreAuthorize("#id == authentication.principal.username")
@GetMapping("testMe/{id}")
public String testGetV1DemoEndpoint(@PathVariable String id){

    SecurityContextHolder.getContext().getAuthentication().getAuthorities().stream().f
    orEach(a -> System.out.println(a));
    return "Access granted to user: " + id;
}
```

In short

- Filters = identity-based, generic access control.
- Method-level security = identity + request data-based, fine-grained access control.

Multi-line @PreAuthorize for Complex Security Rules

- When access control requires multiple conditions, @PreAuthorize supports multi-line SpEL expressions.
- This improves readability for complex rules (e.g., combining user roles, ownership checks, and request arguments).
- For very complex logic, you should extract it into a custom security bean.

Code Snippet

```
@PreAuthorize("""
    hasRole('Admin')
    and #id == authentication.name
    """)
@GetMapping("testMultiLine/{id}")
public String testMultiLine(@PathVariable String id) {
    return "Access granted to user: " + id;
}
```

Disadvantages of Multi-line rules

- Less Readable when rules lines increases beyond 2 - 3 rules
- Difficult to debug if anything fails in any of the rule
- Your life becomes miserable as the rules keeps on growing as u cannot accommodate everything inside 1 annotation

Moving Beyond SpEL: Bean-Based Security Checks

- When authorization rules go beyond simple SpEL conditions, the recommended approach is to encapsulate the logic in a dedicated Spring bean (e.g., CustomSecurityService) and then invoke that bean's method inside `@PreAuthorize`.

Code Snippet

```
@Component("customSecurity")
public class CustomSecurityService {

    public boolean canAccessUser(String userId, Authentication authentication) {
        String loggedInUser = authentication.getName(); // current user
        // Complex logic: allow if same user OR if user has ADMIN role
        return loggedInUser.equals(userId) ||
            authentication.getAuthorities().stream()
                .anyMatch(auth -> auth.getAuthority().equals("ROLE_ADMIN"));
    }
}
```



Code Snippet

```
@RestController
@RequestMapping("/users")
public class UserController {

    // Complex logic delegated to customSecurity.canAccessUser
    @PreAuthorize("@customSecurity.canAccessUser(#id, authentication)")
    @GetMapping("/{id}")
    public String getUser(@PathVariable String id) {
        return "Access granted to user: " + id;
    }
}
```

Post Authorize

Execution point: Runs after the method executes.

Use case: When you need to check both the input + the returned object before deciding if the caller should see the response.

Useful for scenarios like:

A user fetches an entity, but you want to allow access only if the returned entity actually belongs to them.

When to use?

Use `@PostAuthorize` only when the authorization decision depends on the returned object.

Code Snippet

```
@PostAuthorize("""
    returnObject == 'Access Granted'
    and #id == authentication.name
    """)

@GetMapping("testPostMultiLine/{id}")
public String testPostMultiLine(@PathVariable String id) {
    return "Access granted";
}
```

Difference Between @PreAuthorize and @PostAuthorize

Aspect	@PreAuthorize	@PostAuthorize
Execution Timing	Runs before the method is invoked	Runs after the method executes
Checks Applied On	Method arguments, authentication object	Method return object, authentication object
Use Case	Prevent method execution if user is not authorized	Validate if the returned object can be accessed by the user
Performance	More efficient (blocks before execution)	Less efficient (method executes first, then checks)
Typical Example	Allow user to update their own profile only (#id == authentication.name)	Allow access to returned User object only if owner == authentication.name
Common Usage	More frequently used in practice	Rarely used, only when decision depends on returned data

Post Authorize

Execution point: Runs after the method executes.

Use case: When you need to check both the input + the returned object before deciding if the caller should see the response.

Useful for scenarios like:

A user fetches an entity, but you want to allow access only if the returned entity actually belongs to them.

When to use?

Use `@PostAuthorize` only when the authorization decision depends on the returned object.

Code Snippet - Post Authorize

```
@RestController
@RequestMapping("/users")
public class UserController {

    // Method executes first, result is checked afterwards
    @PostAuthorize("returnObject.owner == authentication.name or hasRole('ADMIN')")
    @GetMapping("/{id}")
    public User getUser(@PathVariable String id) {
        // Suppose this fetches user from DB
        return userService.findUserById(id);
    }
}
```

returnObject -> Special Spring Security expression that refers to the object returned by the method. Here, the method returns a User. So returnObject is the User instance returned by userService.findUserById(id).

owner → A field/property inside the returned User object.

Pre filter

Pre/post Authorize - These restricts the access completely to the endpoint or the method

Pre /post Filter - They don't restrict complete access but limit the access to specific values being passed to the method.

Having said that it's very clear that these work on only lists, arrays or collections which pre /post filter can filter and restrict access to those elements don't adhere to the filtering logic

Code snippet

```
@PreFilter("!filterObject.contains('hack')")
    @GetMapping("testPreFilter")
public String testPreFilter(@RequestBody List<String> listToBeFiltered) {
    return "filtered list is: " + listToBeFiltered;
}
```

Pre filter - Key Pointers

Purpose:

Filters method input parameters (like lists or arrays) before the method executes — removing unauthorized elements.

Timing:

Runs before method invocation, unlike `@PostFilter`` (which runs after).

Expression Language (SpEL):

Uses Spring Expression Language (SpEL) for filtering conditions — e.g.:

```
@PreFilter("filterObject.owner == authentication.name")
```

Keyword `filterObject``:

Represents the current element being evaluated in the input collection.

Pre filter - Key Pointers

Requires Method Security:

You must enable method-level security:

```
@EnableMethodSecurity(prePostEnabled = true)
```

Works on Collections or Arrays Only:

Filtering only applies to method parameters of type:

- Collection` (e.g., `List`, `Set`)
- Array (e.g., `User[]`)

Single Objects Not Filtered:

If your method takes a single object (not a list), `@PreFilter` has no effect.

Pre filter - Key Pointers

Accessing Authentication Data:

You can use:

- ``authentication.name`` → current username
- ``hasRole('ROLE_ADMIN')``, ``hasAuthority('SCOPE_write')``, etc.

Multiple Parameters:

If the method has multiple parameters, use ``filterTarget`` to specify which one to filter:

```
@PreFilter(value = "filterObject.status == 'PENDING'", filterTarget = "timesheets")  
public void approve(List<Timesheet> timesheets, String managerId) { ... }
```

Pre filter - Key Pointers

Best Practices:

- Use `@PreFilter` for data sanitization at entry point.
- Combine with `@PreAuthorize` for additional access control.
- Always validate inputs server-side, even if filtered.

Postfilter - Key Pointers

Purpose

Filters the method return value (collections/arrays) after the method executes — removes elements the caller is not allowed to see.

When to use

Use when you return a collection (or array) and want Spring Security to remove items the caller shouldn't see, e.g., listing documents, tasks, messages, etc.

Works only on collections/arrays

@PostFilter only applies to Collection or array return types. It does not work for single objects or paged results (Page<T>) directly.

Postfilter - Key Pointers

SpEL basics

filterObject = each element in the returned collection.

authentication = current Authentication object.

Example expression:

```
@PostFilter("filterObject.owner == authentication.name or hasRole('ADMIN')")
```

No filterTarget (unlike @PreFilter)

@PostFilter filters the returned collection; you don't specify a target parameter. For pre-method parameter filtering use @PreFilter.

Performance caveat

@PostFilter executes after the method runs — the method may fetch the full dataset from DB and then items are removed. If the dataset is large, prefer filtering at the query/DAO level (SQL WHERE, repository method) for efficiency and to avoid exposing data to the service layer before filtering.

Postfilter - Key Pointers

Use bean methods for complex checks

If ownership logic requires DB or complex rules, delegate to a bean:

```
@PostFilter("@securityService.canView(filterObject)")  
public List<Document> getAllDocuments() { ... }
```

Combine with other annotations

Use `@PostFilter` with `@PreAuthorize` for layered rules:

```
@PreAuthorize("hasAuthority('DOC_READ')")  
@PostFilter("filterObject.tenantId == authentication.details.tenantId")  
public List<Document> getAll() { ... }
```


Postfilter - Key Pointers

Defensive checks still recommended

@PostFilter is convenient, but for critical ops double-check permissions within business logic (for example before performing sensitive transformations or revealing computed fields).

Security-first design

Prefer to enforce data visibility at the repository query level when possible. Use @PostFilter for additional defensive enforcement, cross-cutting rules, or when returning heterogeneous collections assembled from multiple sources.

Post Filter Pitfalls

If you return immutable object that postfilter cannot modify then it will throw exception

Code snippet for PostFilter

PreFilter VS PostFilter

Aspect	@PreFilter	@PostFilter
Execution Timing	Runs before the method executes	Runs after the method executes
Filters	Method input parameters	Method return value
Typical Use Case	Filter incoming data before processing (e.g., delete/update)	Filter outgoing data before returning to caller (e.g., list/view)
Applied On	Collection or array method parameters	Collection or array method return value
Purpose	Prevent unauthorized data from being processed	Prevent unauthorized data from being exposed
SpEL Variable	filterObject → each element of input collection	filterObject → each element of returned collection

PreFilter VS PostFilter

Aspect	@PreFilter	@PostFilter
Can Specify Target Parameter	Yes — <code>@PreFilter(value="...", filterTarget="paramName")</code>	No — applies only to return value
Example Use	<code>@PreFilter("filterObject.owner == authentication.name or hasRole('ADMIN')")</code>	<code>@PostFilter("filterObject.owner == authentication.name or hasRole('ADMIN')")</code>
When It's Helpful	Bulk deletes, batch updates, or multi-input operations	Fetch-all, search, reporting APIs
Performance Impact	Efficient (filters before logic executes)	Potentially expensive (filters after fetching all data)
Data Flow Direction	Inbound (request/parameters)	Outbound (response/return)

PreFilter VS PostFilter

Aspect	@PreFilter	@PostFilter
Affects Business Logic Execution	Yes — method sees only filtered input	No — method executes fully, then filtering applies
Security Goal	Integrity → don't act on disallowed data	Confidentiality → don't leak disallowed data
Use Case	Delete multiple users	Fetch all documents
Usage	Use @PreFilter to clean inputs before they hit your service logic.	Use @PostFilter to clean outputs after your logic, before exposing them to clients.

@Pre/@PostAuthorize VS @Pre/@PostFilter

Aspect	@PreAuthorize / @PostAuthorize	@PreFilter / @PostFilter
Primary Purpose	Authorize access — allow or deny method execution or result	Filter data — remove unauthorized elements from collections
Action Type	Decision-based (permit or block)	Data-based (filter elements)
Execution Effect	Either executes method or throws <code>AccessDeniedException</code>	Method still executes, but collection inputs/outputs are pruned
When It Runs	@PreAuthorize: before method call @PostAuthorize: after method returns	@PreFilter: before method call @PostFilter: after method returns

@Pre/@PostAuthorize VS @Pre/@PostFilter

Aspect	@PreAuthorize / @PostAuthorize	@PreFilter / @PostFilter
Applies To	Entire method call	Collection/array parameters (@PreFilter) or return value (@PostFilter)
Return Type Requirement	Works with any return type	Works only with Collection or array
Expression Type	Boolean condition (e.g. "hasRole('ADMIN')" or "#id == authentication.name")	Filtering expression evaluated per element (e.g. "filterObject.owner == authentication.name")
Effect of Failure	Access denied — method not executed or result not returned	Unauthorized elements are removed silently
Granularity	Coarse-grained (entire method)	Fine-grained (individual elements)

@Pre/@PostAuthorize VS @Pre/@PostFilter

Aspect	@PreAuthorize / @PostAuthorize	@PreFilter / @PostFilter
Typical Use Cases	- Restrict access to a method based on roles/ownership- Enforce permission on single object	- Sanitize incoming bulk data- Sanitize outgoing list results
SpEL Variable Used	#paramName, authentication, principal, returnObject	filterObject (represents each element)
Example Usage	@PreAuthorize("hasRole('ADMIN') or #id == authentication.name")	@PreFilter("filterObject.owner == authentication.name")
Goal	Ensure caller is authorized for the operation	Ensure data passed or returned complies with security rules
Security Type	Access Control (Authorization)	Data Filtering (Visibility / Integrity)

OAuth 2 & OIDC

Basics

Suppose you work in an MNC
Each MNC has

- Microsoft Outlook
- Microsoft Teams
- Microsoft Onenote

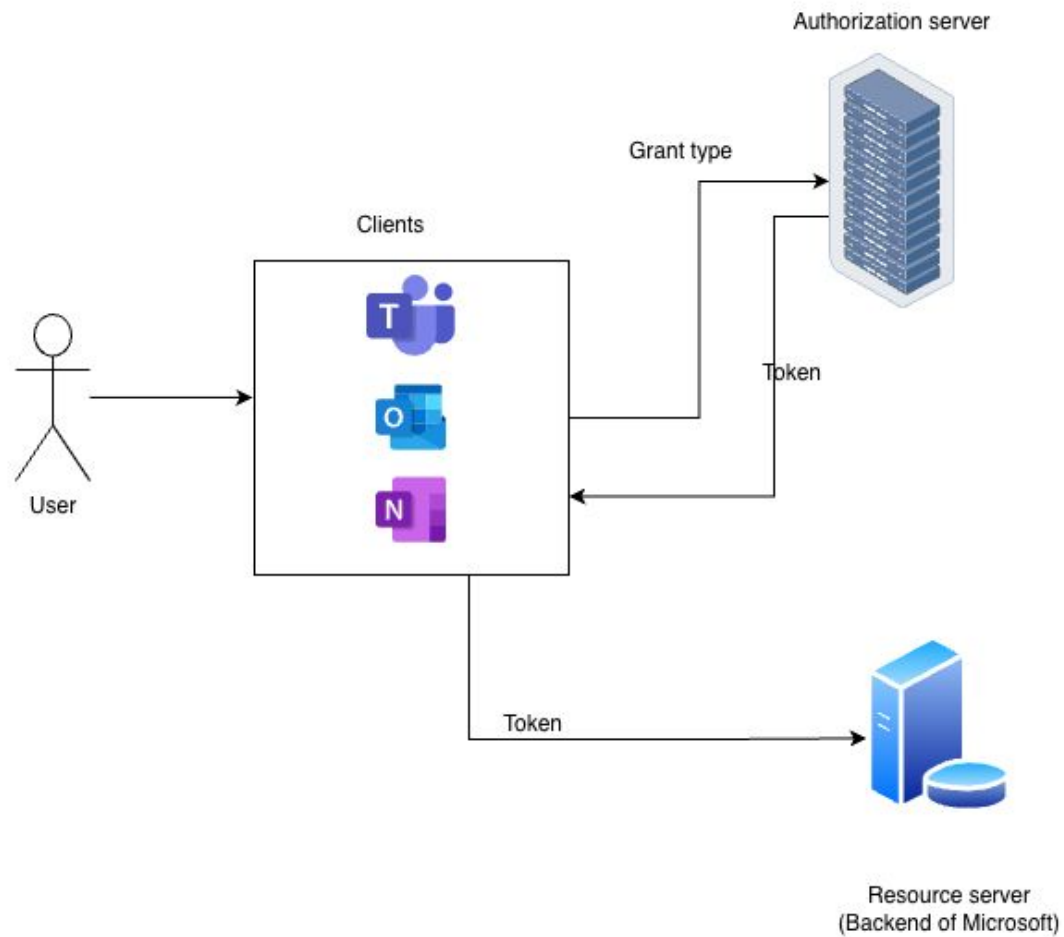
What if u start the day n u have to login to teams, outlook and onenote and all of them have different passwords - don't make sense bcz of of them are from microsoft and for all 3 you are the same user/ consumer / customer - then why for 1 user u maintain 3 different usernames and passwords?

Actors/Roles in OAuth2

- User
- Clients
- Authorization server
- Resource Server - (Backend)

What if u start the day n u have to login to teams, outlook and onenote and all of them have different passwords - don't make sense bcz of of them are from microsoft and for all 3 you are the same user/ consumer / customer - then why for 1 user u maintain 3 different usernames and passwords?

OAuth 2 Flow



The 4 Actors with microsoft example in OAuth 2.0

Actor	In Microsoft Example	Description
Resource Owner	You (the employee/user)	The person who owns the data (emails, notes, chats).
Client Application	Microsoft Teams / Outlook / OneNote	The app that wants to access your data.
Authorization Server	Microsoft Login Service (login.microsoftonline.com)	The system that verifies your identity and issues tokens.
Resource Server	Microsoft APIs (Graph API, Outlook API, OneNote API)	Where your actual data lives (emails, chats, notes).

What will happen without Oauth2

If each app — Teams, Outlook, OneNote — asked for your username and password separately, you'd have to:

- Log In 3 times
- Maintain 3 different sessions
- Risk security leaks (since each app now knows your credentials)

Clearly, that's painful and unsafe.

The OAuth 2.0 Solution

OAuth fixes this by centralizing authentication via the Authorization Server — so all Microsoft apps trust one login authority.

Step-by-Step Flow (Microsoft Example)

Step 1 — You open Microsoft Teams

Teams = Client Application

You click “Sign in”.

Teams redirects you to the **Authorization Server (Microsoft login page)**.

Teams → redirect → <https://login.microsoftonline.com>

The OAuth 2.0 Solution

Step 2 — You log in at Microsoft Login Page

Now you're on the **Authorization Server's domain — not inside Teams.**

You enter your credentials (username, password, maybe MFA).

Microsoft validates your identity.

You're now authenticated.

Teams (Client) never sees your password.

The OAuth 2.0 Solution

Step 3 — Authorization Server issues a code

Once you're verified, the Authorization Server redirects you back to Teams with a short-lived authorization code.

<https://teams.microsoft.com/callback?code=abc123>

Step 4 — Teams exchanges that code for a token

Now Teams calls the Authorization Server (token endpoint) to exchange the code for:

An Access Token (short-lived, used to access data)

A Refresh Token (to get a new access token without logging in again)

Teams never needed your password — only this secure token exchange.

The OAuth 2.0 Solution

Step 5 — Teams uses token to call Resource Server

Now Teams uses the Access Token to fetch your chats, user profile, or OneDrive files from Microsoft Graph API.

GET <https://graph.microsoft.com/me/messages>

Authorization: Bearer eyJhbGciOi...

The Resource Server (Graph API) verifies the token's signature and scope.
If valid → data is returned.

The OAuth 2.0 Solution

Step 6 — Outlook and OneNote join in

When you later open Outlook or OneNote, those apps also redirect you to the same Authorization Server.

But — since you're already logged in (SSO session exists):

You don't re-enter your password.

The Authorization Server immediately issues new tokens for each app.

Result:

One login → access to all Microsoft services.

One user → one identity → many apps.

MFA, SSO, and conditional access enforced centrally.

Why this is powerful

Benefit	Explanation
Single Sign-On (SSO)	Login once → access Teams, Outlook, OneNote
Security	Password handled only by Authorization Server
Central Control	MFA, access policies, and token expiration managed by Microsoft
Scalability	Any new app (like SharePoint, OneDrive) can use the same OAuth mechanism

Steps in OAuth 2

- User makes a request to the client to get data / hit backend api
- Client then requests the authorization server to do authentication and provide the token through the grant type. (The frontend or another backend calling your API with a token)
- Authorization server provides the client the token.(Keycloak, Okta, Auth0, Spring Authorization Server)
- Now client which is authenticated and have the proper token will now send this token to Resource Server - (Backend/ your Spring Boot app) , which will in turn implement authorization rules

Till date what all authorization rules we have seen through web filters or through method level authorization will be used here at Resource server level

Authorization servers tasks will be just to manage the users and clients who wants to access backend apis and provide the token in case the authentication succeeded

Token should contain all information so that resource server can authorise that token

How to get the token?

Imagine you enter an MNC as a guest for a meeting or interview.
At the reception, you're verified (maybe through an email invite) and given a temporary access card that's valid for a day or two.

Here, the **reception acts as the Authorization Server** — it identifies you and issues the ID card (token).

With this **temporary card/token**, you can access certain areas of the building, but not all — for example, you can't open the admin doors.

Similarly, the token contains specific privileges, and based on those, you can access certain resources on the Resource Server — just like specific areas in the building.

Heart of how OAuth2 + Spring Security works

- 1) Now since u get the token, u as a client give this token to Resource server and expects that u will get the exact data from database that u are asking for. So How did u manage to get the perfect token with proper claims / details
- 2) How Authorization server knows that the token is valid? And How does it authorise that token against authorization rules that you have written in filter or pre/post authorize annotation?

Grant types

Shortest answer to what is grant type = Grant type refers to the different methods used to obtain an access token.

A Grant Type defines the way (or method) in which a client application can obtain an access token from the Authorization Server.

Types of Grant types

Grant Type	Used When	Typical Scenario / Example
1. Authorization Code Grant	For web & mobile apps where the user logs in through a browser redirect.	Standard flow used by apps like Gmail, Facebook login, or any app that redirects to a login page.
2. Client Credentials Grant	When an app (no user) needs to access resources on its own behalf.	Server-to-server communication (e.g., cron jobs, backend microservices).
3. Resource Owner Password Credentials (Password Grant)	When the app directly collects the user's credentials (deprecated now).	Legacy mobile/desktop apps — not recommended due to security risks.

Types of Grant types

Grant Type	Used When	Typical Scenario / Example
4. Refresh Token Grant	To get a new access token without re-authenticating the user.	Mobile/web apps use refresh tokens to keep the session alive.
5. Implicit Grant Implicit Grant	When a client (usually SPA) can't securely store a secret (deprecated).	Single Page Apps (old approach; replaced by Authorization Code with PKCE).
6. Device Code (Device Flow)	Smart TVs, IoT devices	User logs in from another device to authorize the TV/device.

Deprecated Grant types

Both Implicit and Password grant types were part of the original OAuth 2.0 (RFC 6749) spec (2012). They worked fine for that time, but modern security standards (OAuth 2.1, OIDC, PKCE) exposed serious flaws in both.

So, the OAuth community and IETF formally deprecated them in OAuth 2.1 (RFC 6749 → RFC 6749-bis).

OAuth's Main Security Principle

Never trust the client with secrets.

In OAuth, the client is the application trying to access a resource (like user data in your API) on behalf of a user. Usually a frontend

And the secret means any sensitive credential that can be used to authenticate or authorize access:

- Username / password
- Client secret
- Access tokens / refresh tokens
- API keys

The client application is often untrusted — especially when it's running on devices or browsers you don't control.

OAuth's Main Security Principle

Concept	Description
Principle	Never trust client apps with sensitive credentials.
Why	Client apps (especially mobile, SPAs) are easily inspected or hacked.
Result	Tokens, passwords, and client secrets must be handled only by trusted servers.
How enforced	By using secure OAuth flows like Authorization Code with PKCE, not Password or Implicit.

OAuth's Main Security Principle

In OAuth, the client should only ever receive temporary, limited-use tokens — never passwords, secrets, or master credentials.

Because anything the client can see or store can eventually be stolen or copied.

Approach	Who collects the username/password?	Security outcome
Old (Password Grant)	The front-end app's own login form	App receives your password → can be stolen, logged, or leaked
Modern OAuth 2.0 / OIDC (Authorization Code + PKCE)	The Authorization Server login page	App never sees your password at all

Why Password Grant Type Is Deprecated

What it was:

The Resource Owner Password Credentials (ROPC) flow allowed the client app to collect the user's username and password directly and exchange them for an access token.

Example flow:

Client → Authorization Server

POST /token

grant_type=password&username=alice&password=secret

In the Password Grant, the client app itself asks the user for their username and password (e.g., via its own login screen).

Then it sends those credentials directly to the Authorization Server to get a token.

Why Password Grant Type Is Deprecated

Imagine you're visiting a big corporate office.

You walk in and a person sitting near the door says:

“Hi, I'm from security — please hand me your ID/aadhar card so I can get you a visitor pass.”

You hand it over — but that person wasn't the real security guard.

They just looked official.

Now your ID (your credentials) is in the wrong hands.

In contrast, in modern OAuth flows (Authorization Code with PKCE):

You're always redirected to the official security desk (the Authorization Server), with badges, cameras, and company branding — so you know it's safe.

You never hand your ID (password) to anyone else.

That's why modern OAuth forces you to log in on the trusted identity provider's domain (e.g., login.google.com, login.microsoftonline.com) so apps can't fake it.

Why it's bad

Problem	Description
1. App handles user credentials directly	The client app (not the authorization server) gets access to the user's password — which breaks OAuth's core principle ("never share credentials with clients").
2. No user consent screen	The user can't see or approve what permissions the app is requesting.
3. Phishing risk	Because the user is typing their credentials into the client app, not the trusted login page (like Google, Okta, etc.).
4. Hard to support MFA or SSO	Since it bypasses the real login flow, So it can't enforce company login rules like MFA (OTP, biometric) or SSO.

Modern Replacement

Use Authorization Code Flow with PKCE — it delegates login to the Authorization Server (e.g., Google, Okta, Azure AD) instead of giving credentials to the client.

Password Grant is like trusting a stranger outside the office to handle your security check.

Modern OAuth (Authorization Code + PKCE) makes sure you go through the official reception, where your identity, MFA, and access rights are all verified safely.

Why Implicit Grant Type Is Deprecated

What it was:

The Implicit Grant flow was designed for browser-based apps (SPAs) that couldn't securely store client secrets.

It returned the access token directly in the URL fragment after login — no backend exchange step.

Example:

`https://app.example.com/#access_token=eyJhbGci...`

Why it's bad

Problem	Description
1. Token exposed in browser URL	Access tokens appeared in the address bar, browser history, and logs — easy to leak.
2. No refresh token support	You had to re-login frequently — causing poor UX or insecure token reuse.
3. No token binding	Anyone who got the token could reuse it (no proof it belongs to the right app).
4. Susceptible to interception attacks	Man-in-the-middle or malicious scripts could grab the token during redirect.

Modern Replacement:

Use Authorization Code Flow with PKCE (Proof Key for Code Exchange) — PKCE adds a dynamic secret per request, making it secure for SPAs and mobile apps even without storing a client secret.

Summary

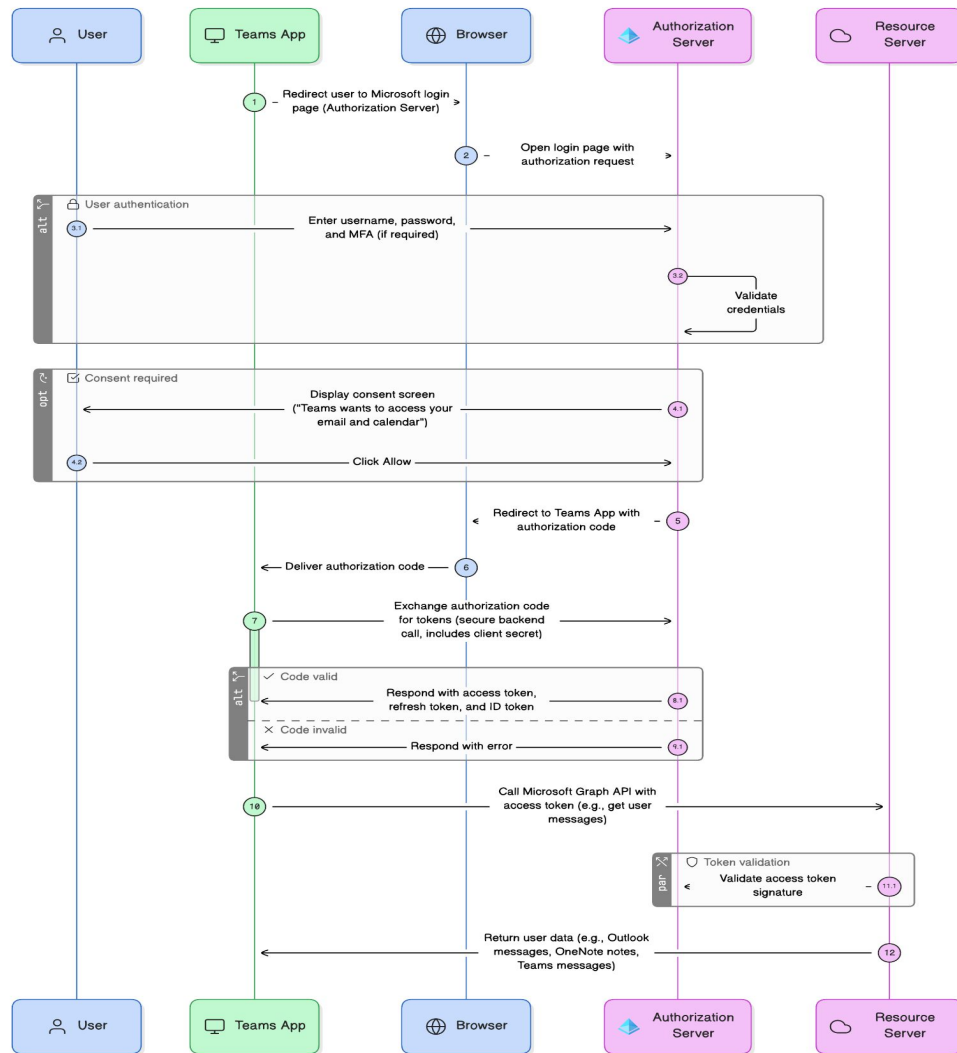
Old Grant Type	Deprecated Because	Modern Replacement
Password Grant	App directly handles user credentials → unsafe	Authorization Code with PKCE
Implicit Grant	Token returned in browser URL → easily leaked	Authorization Code with PKCE



Authorization Code Flow

What Is the Authorization Code Grant Type?

It's a secure way for a client application (like a web or mobile app) to get an access token on behalf of a user without ever handling their password directly.



Step-by-Step Flow

Step 1: The client redirects the user to the Authorization Server

The app (e.g., Teams) wants to access your Microsoft data, so it redirects you to Microsoft's login page:

```
GET https://login.microsoftonline.com/authorize?  
    response_type=code  
    &client_id=teams-client-id  
    &redirect_uri=https://teams.microsoft.com/callback  
    &scope=openid email profile
```

Goal:

The app requests authorization to act on your behalf.

Step-by-Step Flow

Step 2: User logs in and grants permission

You're now on Microsoft's official login page (Authorization Server).

You enter your username, password, and maybe MFA.

Microsoft authenticates you.

You might see a consent screen (e.g., "Teams wants to access your email and calendar").

You click Allow.

Security win:

Teams never sees your password — you log in only at Microsoft's domain.

Step-by-Step Flow

Step 3:

Authorization Server issues an Authorization Code

Once you're verified, the Authorization Server redirects your browser back to the app (Teams) with a temporary code:

<https://teams.microsoft.com/callback?code=SpIxlOBcZQQYbYS6WxSbIA>

This authorization code:

- Is short-lived (usually ~1–5 minutes)

- Can only be used once

- Represents your consent to allow Teams to act on your behalf

Step-by-Step Flow

Step 4: The client exchanges the code for a token

Now Teams makes a server-to-server call to the Authorization Server:

POST <https://login.microsoftonline.com/token>

Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code

code=SpIxlOBeZQQYbYS6WxSbIA

redirect_uri=https://teams.microsoft.com/callback

client_id=teams-client-id

client_secret=teams-client-secret

This step is very secure for server-side web apps, but it becomes dangerous for mobile apps or SPAs — and that's exactly where PKCE comes in. if the `client_secret` is stored in frontend code → instant security breach.

That's why PKCE removes the need for a `client_secret` completely.

Step-by-Step Flow

The authorization code risk = code interception attack

The client_secret risk = secret leakage / exposure

Risk Source	When It's a Problem	Why It's Risky
Authorization Code	If intercepted during redirect (browser URL, proxy, logs)	An attacker could use it to get tokens.
Client Secret	If used in a public app (SPA/mobile)	It's embedded in app code → can be stolen.

Step-by-Step Flow

Step 5: Authorization Server responds with tokens

If everything checks out, the Authorization Server responds:

```
{  
  "access_token": "eyJhbGciOi...",  
  "refresh_token": "8xLOxBtZp8",  
  "id_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "expires_in": 3600  
}
```

Now Teams has:

Access Token — to call APIs (short-lived, e.g. 1 hour)

Refresh Token — to get new access tokens silently

ID Token — identifies the user (OpenID Connect)

Step-by-Step Flow

Step 6: Client uses Access Token to access Resource Server

Teams uses the `access_token` to call APIs like Microsoft Graph:

GET <https://graph.microsoft.com/me/messages>

Authorization: Bearer `eyJhbGciOi...`

The Resource Server validates the token signature using Microsoft's public key. If valid, it returns your Outlook data, OneNote notes, or Teams messages.

Done — all without exposing your password to Teams.

Summary of Authorization code grant type flow

Step	Action	Who Does It	Purpose
1	Redirect user to login	Client App → Authorization Server	Request user consent
2	User logs in & approves	Resource Owner → Authorization Server	Authenticate user
3	Issue Authorization Code	Authorization Server → Client	Temporary proof of consent
4	Exchange code for token	Client → Authorization Server	Obtain Access & Refresh Tokens
5	Access Resource	Client → Resource Server	Use Access Token to fetch data

Advantages

Feature	Description
Security	Very secure, since credentials and tokens are never exposed to the front-end.
Requires Client Secret?	Yes (in traditional Authorization Code flow)
Used For	Web apps with secure backends (server-side apps).
Can use Refresh Tokens?	Yes
Supports MFA, SSO	Yes — since login happens at Authorization Server.

Disadvantages

When used by mobile apps or SPAs, there's a catch

They cannot securely store the `client_secret`.

If you embed a secret in JavaScript or an APK, anyone can extract it.

That's why PKCE (Proof Key for Code Exchange) was added —
to secure the Authorization Code flow for public clients (those without backend secrets).

Authorization Code Flow with PKCE (Proof Key for Code Exchange)

What is PKCE (pronounced “pixy”)

PKCE stands for **Proof Key for Code Exchange**.

We use something else to identify the client app instead of the client credentials. That's challenge and verifier

It's a mechanism that prevents attackers from stealing and reusing the authorization code during the OAuth flow.

The idea:

Since public clients (like mobile or SPA apps) can't store a client secret, PKCE adds a temporary, dynamic secret that's created per login request.

This ensures that only the real app that started the login flow can exchange the code for a token — even if someone intercepts the code.

Why PKCE was introduced

In the traditional Authorization Code flow, if an attacker intercepts the authorization code (via redirect URI or network sniffing), they can exchange it for an access token.

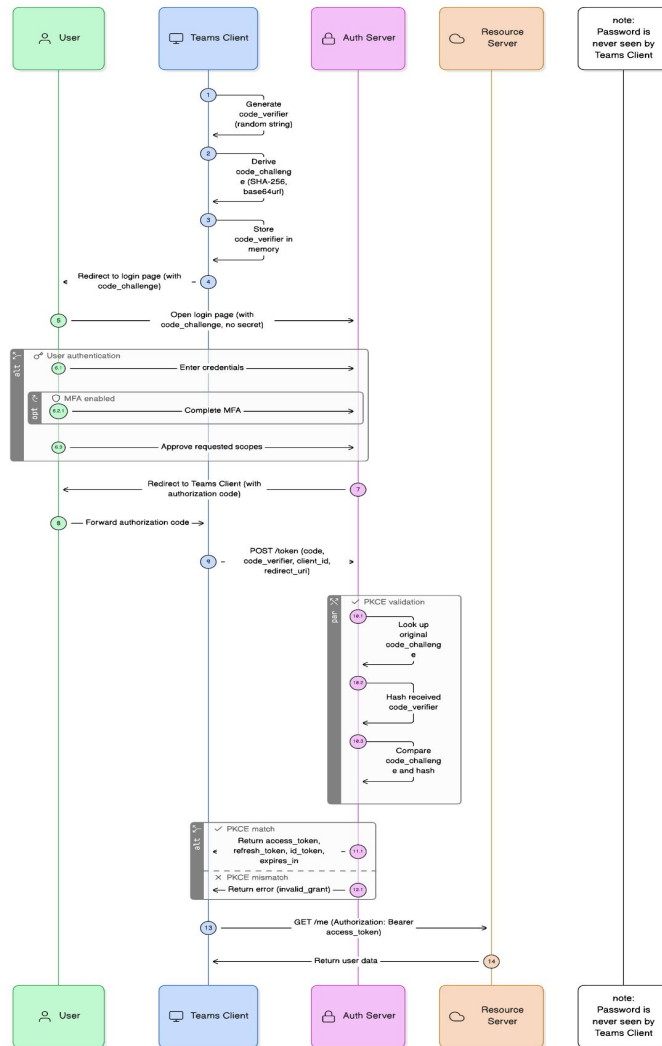
But with PKCE:

That intercepted code is useless without the matching verifier.

So PKCE gives public apps the same security that confidential (server-side) apps had with a client secret.

The Players (same as before)

Actor	Role	Example
Resource Owner	The user	You (Microsoft user)
Client	The app requesting access	Microsoft Teams (SPA or mobile app)
Authorization Server	Issues tokens	Microsoft Login (login.microsoftonline.com)
Resource Server	Hosts resources	Microsoft Graph API



Authorization Code Flow with PKCE — Step by Step

Step 1 — Client generates PKCE pair

Before redirecting the user,
the client (e.g., Teams front-end app) generates:

- A random code_verifier (like a one-time password)
- A hashed code_challenge derived from that verifier (using SHA-256)

Example:

```
code_verifier = "mU7wz9xRzV6tZyqH1pP1iA8sK"  
code_challenge = BASE64URL-ENCODE(SHA256(code_verifier))
```

This is stored only inside the client's memory.

Authorization Code Flow with PKCE — Step by Step

Step 2 — Redirect to Authorization Server

Now the client redirects the user to the Authorization Server's login page:

GET [https://login.microsoftonline.com/authorize?](https://login.microsoftonline.com/authorize?response_type=code&client_id=teams-client-id&redirect_uri=https://teams.microsoft.com/callback&scope=openid%20profile%20email&code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM&code_challenge_method=S256)

`response_type=code`

`&client_id=teams-client-id`

`&redirect_uri=https://teams.microsoft.com/callback`

`&scope=openid profile email`

`&code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM`

`&code_challenge_method=S256`

The `code_challenge` goes along with the request.

No secret or password is sent.

Authorization Code Flow with PKCE — Step by Step

Step 3 — User logs in (as usual)

You log in on the Authorization Server's official page (not the app):

- Enter credentials
- Pass MFA (if configured)
- Approve the requested scopes

Password is never seen by the app.

Authorization Code Flow with PKCE — Step by Step

Step 4 — Authorization Server returns Authorization Code

After successful login, the Authorization Server redirects you back to the client app's redirect URI:

`https://teams.microsoft.com/callback?code=SpIxIOBeZQQYbYS6WxSbIA`

The app now has a short-lived authorization code, but it's useless without the original code_verifier.

Authorization Code Flow with PKCE — Step by Step

Step 5 — Client exchanges code for tokens

The client now makes a POST request to the token endpoint, using the `code_verifier` instead of a `client_secret`:

POST <https://login.microsoftonline.com/token>

Content-Type: application/x-www-form-urlencoded

`grant_type=authorization_code`

`code=SpIxlOBeZQQYbYS6WxSbIA`

`redirect_uri=https://teams.microsoft.com/callback`

`client_id=teams-client-id`

`code_verifier=mU7wz9xRzV6tZyqH1pP1iA8sK`

Authorization Code Flow with PKCE — Step by Step

Step 6 — Authorization Server validates PKCE pair

The Authorization Server:

Looks up the original code_challenge stored in Step 2.

Hashes the received code_verifier.

Compares both values.

- If they match → request is legit (same app that started the login).
- If they don't → token request fails (possible hijack attempt).

Authorization Code Flow with PKCE — Step by Step

Step 7 — Tokens issued

Server returns tokens:

```
{  
  "access_token": "eyJhbGciOi...",  
  "refresh_token": "8xLOxBtZp8",  
  "id_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "expires_in": 3600  
}
```

Now the app can call APIs securely.

Authorization Code Flow with PKCE — Step by Step

Step 8 — Use token to call Resource Server

Teams calls Microsoft Graph:

GET <https://graph.microsoft.com/me>

Authorization: Bearer eyJhbGciOi...

The Resource Server validates the token and returns your data.

How PKCE Prevents Attacks

Attack	What Happens	How PKCE Stops It
Authorization Code Interception	Hacker intercepts the code from redirect URL	Code is useless without code_verifier
Token Replay	Hacker tries to reuse intercepted code	Code + verifier combo can only be used once
Man-in-the-Middle	Attacker modifies code in transit	Hash verification fails at Authorization Server

Real Time Analogy

In Microsoft Example (Easy Analogy)

1. You open Teams → Teams sends you to Microsoft Login.
2. Teams secretly creates a one-time handshake (code_challenge).
3. You log in → Microsoft Login gives Teams a claim slip (code).
4. Teams goes back to Microsoft Login with the claim slip and the secret handshake (code_verifier).
5. Microsoft Login checks that handshake matches → issues access badge (token).
6. Teams uses that badge to enter other departments like Outlook or OneNote.
7. Badge expires → Teams silently gets a new one using refresh token.

Secure login , No password exposure , MFA works , SSO supported

How Verifier & Challenge Work

Instead of relying on a client secret (which public apps can't store safely), PKCE creates a temporary one-time secret for each login attempt.

It's like a handshake known only to your app and the Authorization Server.

This is done using:

- **code_verifier** (the app's secret handshake) (the code_verifier in PKCE is effectively a temporary, one-time replacement for the client_secret.)
code_verifier = One-Time Client Secret
- **code_challenge** (a scrambled version of that handshake)

PKCE adds a one-time proof that only the real client can produce.

Step-by-Step: How Verifier & Challenge Work

Let's say your app (Teams) starts a login flow.

Step 1 — Generate a Code Verifier

Your app creates a random, unique string:

```
code_verifier = "U9bP3Qxk7T3XrDq9L2f5Z8vY"
```

It's just a random sequence of characters (like a one-time password).

It's stored only inside your Teams app's memory — not shared yet.

Step-by-Step: How Verifier & Challenge Work

Step 2 — Create a Code Challenge

Now your app hashes that verifier to create a “challenge”.

```
code_challenge = SHA256(code_verifier)
```

SHA takes any input (like your `code_verifier` string)
and produces a unique, fixed-length digest that cannot be reversed back to the original.

Example (simplified):

```
code_challenge = "qOa8BxD73x0pQ7sHnE6TbZ1jYH..."
```

So `code_challenge` is derived from the verifier using SHA-256,
but cannot be reversed — meaning nobody can find the original verifier just from the challenge.
SHA (Secure Hash Algorithm) is a one-way cryptographic hash function, not encryption and not encoding rather its Hashing.

Step-by-Step: How Verifier & Challenge Work

Step 3 — Send the Challenge in the Authorization Request

Your app redirects the user to the Authorization Server (login page), including the `code_challenge`:

```
GET https://login.microsoftonline.com/authorize?  
  response_type=code  
  &client_id=teams-client-id  
  &redirect_uri=https://teams.microsoft.com/callback  
  &scope=openid email  
  &code_challenge=qOa8BxD73x0pQ7sHnE6TbZ1jYH...  
  &code_challenge_method=S256
```

So:

The Authorization Server stores this `code_challenge` temporarily.

Your app keeps the `code_verifier` safe.

Step-by-Step: How Verifier & Challenge Work

So internally, something like this is saved in authorization Server -

```
{  
  "authorization_code": "SplxIOBeZQQYbYS6WxSbIA",  
  "client_id": "teams-client-id",  
  "code_challenge": "E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM",  
  "code_challenge_method": "S256"  
}
```

Step-by-Step: How Verifier & Challenge Work

Step 4 — User Logs In Normally

You log in → password, MFA, etc.

The Authorization Server verifies you,
then sends a short-lived authorization code back to your app:

<https://teams.microsoft.com/callback?code=abc123>

Remember - SHA-256 is one-way → **you can easily go from verifier → challenge (hash), but not challenge → verifier.**

Step-by-Step: How Verifier & Challenge Work

Step 5 — App Exchanges the Code for a Token

Now your app calls the token endpoint like this:

```
POST https://login.microsoftonline.com/token
grant_type=authorization_code
code=abc123
client_id=teams-client-id
redirect_uri=https://teams.microsoft.com/callback
code_verifier=U9bP3Qxk7T3XrDq9L2f5Z8vY
```

It sends the original code_verifier (not the hash) this time.

Step-by-Step: How Verifier & Challenge Work

Step 6 — The Authorization Server:

- Takes the `code_verifier` you just sent.
- Hashes it again with SHA-256 → gets `qOa8BxD73x0pQ7sHnE6TbZ1jYH...` (The Authorization Server recreates the challenge from the verifier.)
- Compares it to the stored `code_challenge` from Step 3. it just recomputes the hash and checks equality.

If they match → same app = valid request

If they don't → someone tried to fake it = reject token request

Then it issues:

```
{  
  "access_token": "eyJhbGciOi...",  
  "refresh_token": "abc...",  
  "id_token": "eyJ0eXAiOiJKV1Qi..."  
}
```

Real-World Analogy: “The Locker & Key”

Imagine you go to a gym and rent a locker.

- You put your valuables inside and keep the key (`code_verifier`).
- The gym records the locker number and the shape of your key (the hash, `code_challenge`).
- Later, to claim your stuff, you hand over the key.
- The gym checks the key shape — if it fits, it's yours.

If someone steals your locker number (`authorization_code`), they can't open the locker without the real key (`code_verifier`).

Summary of PKCE Flow

Step	Action	Purpose
1	Client generates code_verifier + code_challenge	Creates a one-time secret
2	Redirects to Authorization Server with challenge	Starts secure login
3	User logs in	Authentication by Auth Server
4	Server returns authorization code	Temporary proof
5	Client exchanges code + verifier	Proves legitimacy
6	Server validates challenge/verifier	Prevents interception
7	Tokens issued	Access & refresh tokens granted
8	App uses token to call API	Resource access

Authorization Code vs Authorization Code + PKCE

Feature	Authorization Code	Authorization Code with PKCE
Client Secret Required	Yes	No
Suited For	Backend web apps	SPAs / Mobile apps
Protection Against Code Interception	No	Yes
Token Request Authentication	Client secret	PKCE verifier
Security Level	High	Very High (for public clients)

Points to remember

- The login page is always from Authorization server and not from teams/client app else we will go back to password grant type thats deprecated.
- How authorization server know that where to redirect in clients app? which page? which url? This is registered in authorization server by client and client while redirecting to auth server tell after authentication, redirect to me to this url. That's the redirect URI provided by the client and needs to be know to the authorization server.
- To get the access token , all the requests are post requests, be in with or without PKCE, just that the client to get the access token need to validate / authenticated himself as legitimate client app (Teams needs to verify itself with microsoft authorization server).
- There is a difference between client credentials and user credentials - client id client secrets and user name password, both credentials (client and users) are managed by authorization server.



Client Credentials Grant Type

What is Client Credentials grant

Client Credentials is an OAuth2 grant where a client (machine/service) obtains an access token in its own name (no user involved) by authenticating to the Authorization Server using its client credentials.

When to use it — typical use-cases

- Machine-to-machine communication (microservices → backend APIs).
- Cron jobs, background workers, daemons.
- Service accounts (CI/CD systems, system integrations).
- Non-interactive backend API calls where no user context is required.

The Actors

Actor	Description
Client (confidential)	The service that requests a token (must be able to safely store credentials)
Authorization Server (AS)	issues the token
Resource Server (RS)	the API that requires the token for access
Resource Owner (user)	No Resource Owner (user) participates here.

Flow (step-by-step)

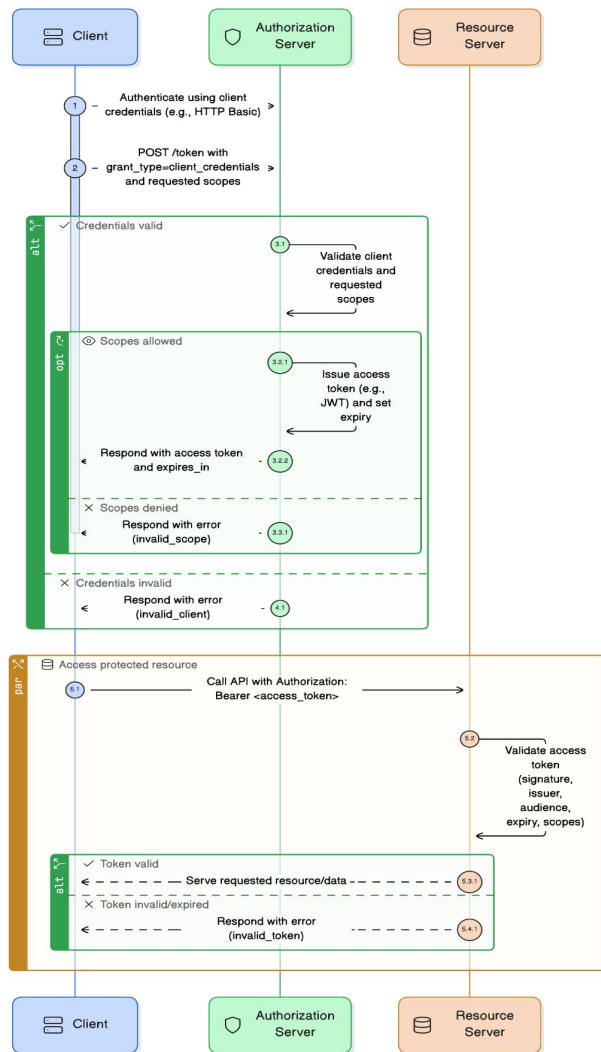
1. Client authenticates to AS (usually via HTTP Basic or another client authentication method) and POSTS to token endpoint:

POST /token

grant_type=client_credentials

scope=read:books write:orders

2. AS validates client credentials and (optionally) requested scopes.
3. AS responds with an access token (often a JWT) and expires_in.
4. Client calls Resource Server with:
Authorization: Bearer <access_token>
5. Resource Server validates token (signature/iss/aud/exp/scopes) and serves request.



Typical token response

```
{  
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "token_type": "Bearer",  
  "expires_in": 3600,  
  "scope": "read:books write:orders"  
}
```

Note: Refresh tokens are typically not issued for client_credentials flows (tokens are short-lived and the client can request a new one).

Client authentication methods with AS

- HTTP Basic (Authorization: Basic base64(client_id:client_secret)) — common.
- POST body (client_id + client_secret in body) — less recommended.
- private_key_jwt — client signs a JWT with its private key and uses it as client assertion (stronger, recommended for high-security).
- mutual TLS (mTLS) — TLS client certificate authenticates client (very strong).
- TLS with client certificate bound tokens — tokens are bound to client certificate.

Use private_key_jwt or mTLS in high-security environments where client_secret leakage is unacceptable.

How Scopes → Authorities Mapping Works

Scopes define what an app can do (declared in the token),

Authorities decide where in your code that's allowed (checked by Spring Security).

Layer	Who Uses It	Example
Authorization Server (Auth0, Keycloak, Okta)	Issues scopes in token	"scope": "read write"
Resource Server (your API)	Converts scopes → authorities	GrantedAuthority: [SCOPE_read, SCOPE_write]
Spring Security	Checks authorities to authorize method/endpoint	@PreAuthorize("hasAuthority('SCOPE_read')")

Scopes & authorities

- Client requests scopes; the AS issues token with scope claim.
- Map token scopes to GrantedAuthority or roles on the Resource Server.
- Keep scopes minimal — principle of least privilege.

Tokens: JWT vs opaque

- **JWT (self-contained):** RS can validate locally using AS public key (via JWKS); fast and scalable.
 - Verify: signature, iss, aud, exp, scope, client_id (if relevant)
- **Opaque tokens:** RS calls AS token introspection endpoint to validate token.
 - Use introspection when server wants central control or immediate revocation.

Always check aud (audience) and scope on RS.

Security considerations / best practices

- **Treat client credentials like secrets:** store in secure vault/keystore, not in source code or container images.
- **Use short token lifetimes** and rotate secrets frequently.
- **Prefer mTLS or private_key_jwt** for high-value clients (avoid static client_secret if possible).
- **Limit scopes** and permissions allocated to the client.
- **Use token binding** features if available (DPoP, mTLS-bound tokens) when risk of token theft is high.
- **Log carefully** — never log client_secret or tokens.

Security considerations / best practices

- **Protect token exchange endpoints** with HTTPS/TLS only.
- **Use RBAC/ABAC within RS** — do not rely solely on scopes if fine-grained control needed.
- **Revoke/rotate** credentials on compromise and rotate tokens (reissue new tokens quickly).

Pitfalls & gotchas

- **Using client_credentials for user actions** — don't use this grant to perform actions on behalf of an end user. Use Authorization Code/OIDC.
- **Over-privileged tokens** — issuing admin scope when only read is needed.
- **Storing secrets in client code or public repos** — immediate compromise risk.
- **Not validating audience** — RS must ensure token was intended for it (aud claim).
- **Assuming refresh token will be issued** — client_credentials commonly doesn't return refresh tokens

Refresh Token Grant Type

What is a Refresh Token?

A refresh token is a long-lived credential issued along with an access token, which the client can later use to get a new access token — without asking the user to log in again.

In short:

“Access token expires soon? Use refresh token to get a new one quietly.”

Why Refresh Tokens Exist

Access tokens are intentionally short-lived (e.g. 5–60 minutes) to reduce damage if they're stolen.

Without refresh tokens:

- Users would have to re-login every hour.
- Apps couldn't keep sessions alive automatically.

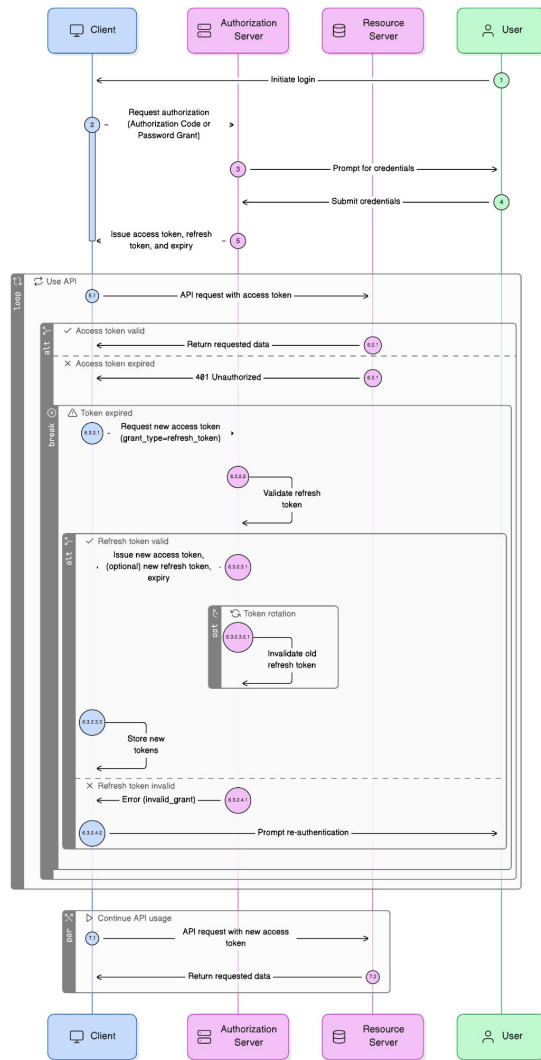
With refresh tokens:

- Apps can silently renew the access token.
- Users stay logged in for longer without re-entering credentials.

Improves user experience + maintains security.

Who uses the Refresh Token flow?

Client Type	Can Use Refresh Token?	Notes
Confidential Client (Backend app)	Yes	Securely stores refresh token on server
Public Client (Mobile app)	Yes, with care	Store in encrypted OS keystore
SPA / Browser app	Sometimes	Only with PKCE + Secure HTTP-only cookies
Machine-to-Machine (client_credentials)	No	No user context — not applicable



Refresh Token Grant Type Flow (Step-by-Step)

Step 1 – User Login (Normal Authorization Flow)

The client performs an Authorization Code (or Password Grant, legacy) flow.
The Authorization Server issues:

```
{  
  "access_token": "eyJhbGciOiJSUzI1...",  
  "refresh_token": "tGzv3JOkF0XG5Qx2TIKWIA",  
  "expires_in": 3600  
}
```

Access token expires in 1 hour, refresh token lasts longer (days/weeks).

Step 2 – Access Token Expires

The client notices the access token is expired (401 from API or timer).

Refresh Token Grant Type Flow (Step-by-Step)

Step 3 – Use Refresh Token to Get New Access Token

When the client first completes an interactive login (e.g. Authorization Code ± PKCE), the Authorization Server may return both:

```
{  
  "access_token": "...",  
  "refresh_token": "tGzv3JOkF0XG5Qx2TIKWIA",  
  "expires_in": 3600  
}
```

Client makes this request to the token endpoint:

POST /token

Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token

refresh_token=tGzv3JOkF0XG5Qx2TIKWIA

client_id=app-client-id

client_secret=app-client-secret

Refresh Token Grant Type Flow (Step-by-Step)

Step 3 – Use Refresh Token to Get New Access Token

Authorization Server verifies the refresh token, and if valid, responds with:

```
{  
  "access_token": "newAccessToken123", (yes a new access token !!)  
  "refresh_token": "newRefreshToken456", (// maybe optional (rotation))  
  "expires_in": 3600  
}
```

The client stores the new tokens and continues using the API.

Refresh Token Grant Type Flow (Step-by-Step)

Analogy

Think of it like a movie theater

- Access token = your movie ticket (valid for one show).
- Refresh token = your membership card.

When your ticket expires, you show your membership card at the counter → they give you a new ticket (new access token).

The refresh token itself is never used to enter the movie — it's only used to request a fresh ticket from the counter.

Refresh Token Grant Type Flow (Step-by-Step)

Step 4 – (Optional) Token Rotation

- Sometimes the AS issues a new refresh token each time.
- The old one becomes invalid (rotation).
- This prevents reuse of stolen refresh tokens.

Same or new refresh token ?

When I use a refresh token to get a new access token — does the Authorization Server send back the same refresh token again, or a new one each time?

It depends on the Authorization Server's policy.

There are two common behaviors:

- Static (reusable) refresh tokens, and
- Rotating (one-time) refresh tokens

Static (Reusable) Refresh Tokens

What happens

The Authorization Server issues one refresh token and keeps it valid until it expires or is revoked.

- You can use the same refresh token multiple times.
- Each time you use it, you get a new access token, but the refresh token stays the same.

Example

```
{  
  "access_token": "accessToken_1",  
  "refresh_token": "refreshToken_X"  
}
```

You call /token again with the same refresh token:

```
grant_type=refresh_token  
refresh_token=refreshToken_X
```

Static (Reusable) Refresh Tokens

Server responds with:

```
{  
  "access_token": "accessToken_2",  
  "refresh_token": "refreshToken_X"  
}
```

Pros

- Simple to implement.
- Works well for trusted clients.

Cons

- If refresh_token is ever stolen, attacker can keep getting new access tokens indefinitely (until revoked or expired).
- Harder to detect misuse.

Rotating (One-time) Refresh Tokens

What happens

The Authorization Server issues a new refresh token every time you use one — and invalidates the old one immediately.

- So each refresh token is one-time use only.
- If someone tries to reuse an old token → server detects it → triggers security alert (token theft suspected).

Example

Initial token response:

```
{  
  "access_token": "accessToken_1",  
  "refresh_token": "refreshToken_ABC"  
}
```

Rotating (One-time) Refresh Tokens

Next refresh request:

```
grant_type=refresh_token  
refresh_token=refreshToken_ABC
```

Response:

```
{  
  "access_token": "accessToken_2",  
  "refresh_token": "refreshToken_DEF"  
}
```

If attacker tries to reuse refreshToken_ABC → rejected (“invalid_grant”).

Rotating (One-time) Refresh Tokens

Pros

- More secure — stolen refresh token can only be used once.
- Detects suspicious reuse patterns (helps identify breaches).

Cons

- Slightly more complex to implement.
- Client must always store the latest refresh token (replacing old one).

How OAuth2 servers decide

Modern providers prefer rotation for security.

Authorization Server	Behavior by default
Auth0	Rotating by default (configurable)
Azure AD	Rotating refresh tokens
Keycloak	Rotating optional (configurable)
Okta	Rotating refresh tokens (recommended)
Google OAuth2	Static (for simplicity)

What clients must do

Rule	Explanation
Always store the latest refresh token you receive.	The server may rotate it.
Never reuse an old refresh token after using it once.	It may be invalidated.
Detect “invalid_grant” errors on reuse.	Indicates rotation failure or token theft detection.
Store refresh token securely.	Treat it like a password.

Analogy

Think of refresh tokens like parking passes:

- Static token → one pass works all month.
- Rotating token → every time you park, you get a new pass and the old one stops working.

If someone copies your old pass (static case), they can keep parking forever.

But with rotating passes, as soon as you use yours again, the copied one becomes invalid.

Key Token Lifetimes

Token Type	Lifetime	Purpose
Access Token	Short (minutes–hours)	Used for API calls
Refresh Token	Long (days–months)	Used to renew access tokens

Why Refresh Tokens Are Sensitive

Refresh tokens are high-value secrets:

- They can generate new access tokens.
- Often valid for days or weeks.
- Must be stored very securely.

If stolen → attacker can get new access tokens indefinitely.

Security Considerations & Best Practices

Category	Best Practice
Storage	Store refresh token securely (server DB, mobile keystore, HTTP-only cookies)
Transmission	Always over HTTPS
Scope & audience	Don't over-privilege refresh tokens; issue them per client/app
Rotation	Enable refresh token rotation (each use invalidates the previous one)
Revocation	Provide a /revoke endpoint to invalidate refresh tokens
Expiration	Set reasonable lifetime (e.g., 7–30 days)
Detection	Detect reuse of a rotated refresh token → treat as breach
No refresh in M2M	Don't issue refresh tokens for client_credentials flows

Refresh Token Flow vs Access Token Flow

Aspect	Access Token Flow	Refresh Token Flow
Triggered when	User logs in	Access token expires
Requires user?	Yes	No
Returns	Access token (+ refresh token)	New access token (+ optional new refresh token)
Token lifetime	Short	Long
Main benefit	Grants initial access	Keeps session alive

Summary

Concept	Description
What it is	A special token used to get new access tokens without logging in again
Who uses it	Clients acting on behalf of a logged-in user
Why	To maintain long sessions securely
Security	Store securely, rotate often, revoke on logout or reuse
When not used	Machine-to-machine (client_credentials)
Flow	grant_type=refresh_token → new access token (and maybe new refresh token)

What is opaque token?

An opaque token is an access token that has no readable information for the client or resource server — it's just a random string (like a reference ID).

You can't decode it to see who the user is, what scopes it has, or when it expires.

It's called opaque because it's literally a black box — its content is “opaque” (hidden).

Opaque token example:

2YotnFZFEjr1zCsicMWpAA

It's just a random-looking value — not a JWT, no header, payload, or signature.

You can't base64-decode it to find user info or expiration time.

How opaque token Works?

With an opaque token, the Authorization Server keeps all token details internally in its database or cache.

When a Resource Server (API) receives this token, it can't verify it locally — it must ask the Authorization Server to check it.

That happens through the token introspection endpoint (/introspect).

Introspection endpoint - It's a token verification endpoint — your API calls it to ask the Authorization Server:

What it does:

Verifies if the token is active or expired/revoked

Returns token details like user, client, scopes, and expiry time

Example flow (using introspection)

1) Client calls your API:

Authorization: Bearer 2YotnFZFEjr1zCsicMWpAA

2) Resource Server (API) doesn't know what this token means, so it calls:

POST /introspect

Authorization: Basic <api-client-creds>

token=2YotnFZFEjr1zCsicMWpAA

Example flow (using introspection)

3) Authorization Server looks it up in its DB and returns something like:

```
{  
  "active": true,  
  "scope": "read:users write:users",  
  "client_id": "my-app",  
  "username": "vipin",  
  "exp": 1735569600,  
  "iat": 1735566000,  
  "aud": "user-api",  
  "iss": "https://auth.example.com"  
}
```

4) These response from introspection endpoint will be used by resource server to authorize the token

What's inside the introspection response

Field	Meaning
active	Boolean — whether the token is valid.
scope	List of scopes (permissions).
client_id	ID of the client who obtained the token.
username	End-user associated with token (if any).
exp, iat, nbf	Expiry, issued-at, not-before timestamps.
aud, iss	Audience & issuer (for verification).
sub	Subject — usually user or client ID.
token_type	access_token or refresh_token.
jti	Token ID (unique).

What does “non-opaque token” mean?

A non-opaque token is any access token whose structure and content are self-describing and verifiable by the Resource Server, without asking the Authorization Server.

So:

- The token contains the claims or metadata (like user ID, scope, expiry).
- The Resource Server can parse and validate it locally.
- No network call to /introspect is required.

The key idea

Non-opaque = “transparent.”

The token carries its own truth — everything needed to verify it is embedded inside the token itself. Verification happens using cryptographic proof (a signature, MAC, or encryption), not by database lookup.

How it works

When the Authorization Server issues a non-opaque token:

- It encodes claims (subject, client_id, scope, expiry, etc.) into the token payload.
- It signs (and sometimes encrypts) the token using its private key.
- The Resource Server, which has the matching public key or secret, can:

Decode the token,

Verify the signature,

Validate the claims (like exp, aud, scope).

No need to call the Auth Server again.

Common forms of non-opaque tokens

Token type	Format	How verified	Notes
JWT (JSON Web Token)	JSON (Base64URL encoded)	Verify signature (RS256, ES256, HS256)	Most common, standardized, used in OIDC
PASETO (Platform-Agnostic Security Token)	JSON or compact binary	Verify using modern crypto primitives	A newer alternative to JWT — safer defaults
Macaroon token	Layered HMAC-based token	Verify via chained signatures	Used in systems like Google Cloud or Lightning Network
Encrypted token (JWE)	Encrypted JSON token	Decrypt using shared key	Used when confidentiality is required

Why use non-opaque tokens

Advantage	Description
Performance	No need for introspection; validation happens locally.
Scalability	Fewer calls to Authorization Server → less load.
Statelessness	Resource Server doesn't maintain session state or token DB.
Integrity	Signature guarantees token hasn't been tampered with.

Drawbacks

That's why non-opaque tokens often use short lifetimes (minutes) combined with refresh tokens for renewal.

Concern	Explanation
Difficult revocation	Once issued, can't easily revoke before expiry.
Bigger tokens	Self-contained tokens carry all data → longer strings.
Key rotation complexity	Must distribute new public keys (JWKS).
Information exposure	Token payload may reveal metadata (even though signed, not encrypted).

Non-opaque tokens vs opaque tokens

Feature	Opaque token	Non-opaque token
Structure	Random string	Self-contained structure
Who stores token data	Authorization Server	Token itself
Validation method	Introspection API	Local signature verification
Revocation	Easy	Hard
Performance	Slower (extra call)	Faster
Typical forms	UUID, random hash	JWT, PASETO, Macaroon
Visibility	Opaque (cannot read)	Transparent (can read / verify)

Boarding Pass Analogy

Concept	Opaque Token	Non-Opaque Token
Analogy	You show a reference number and the airline staff must check the database before letting you board.	You show a printed QR boarding pass with your flight, gate, and time — the scanner validates it instantly.
Meaning	Reference lookup required.	Encoded, signed, verifiable locally.



JWTs

What is a JWT?

JWT (JSON Web Token) is a compact, URL-safe, digitally signed token used to securely transmit information between two parties — typically between an Authorization Server and a Client or Resource Server.

It's an encoded string that:

- Proves the authenticity of who issued it.
- Contains claims (user info, permissions, etc.).
- Can be verified but not modified by clients.

The basic structure of a JWT

A JWT has three parts, separated by dots .:

HEADER.PAYLOAD.SIGNATURE

Each part is Base64URL-encoded, meaning it's safe to send in URLs or HTTP headers.

Example JWT:

eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.

eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwicm9sZSI6ImlsiVFNFUilzIkFETUIOI19.

TjV0K_pZl0gohsHIMa0y4bLxHk5ZgjA6W5FeYmyC34E

The basic structure of a JWT

. Header

```
{  
  "alg": "RS256",  
  "typ": "JWT"  
}
```

alg → signing algorithm (e.g., HS256, RS256).

typ → type of token (JWT).

Think of it as: “How should this token be verified?”

The basic structure of a JWT

2. Payload

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "roles": ["USER", "ADMIN"],  
  "iat": 1734470000,  
  "exp": 1734473600  
}
```

sub → subject (user ID)

name → user name

roles → custom claim

iat → issued at

exp → expiry time

This is the data/claims section — what the token says about the user.

The basic structure of a JWT

3. Signature

```
HMACSHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  secretKey  
)
```

or (for RSA):

```
RS256(privateKey, data)
```

This is the cryptographic proof that ensures the token hasn't been tampered with.

How JWT works (high-level flow)

Step	Description
1	User logs in with username/password.
2	Authorization Server verifies credentials.
3	Server creates a JWT containing user info (claims) and signs it.
4	Server sends the JWT back to the client.
5	Client sends this JWT with every request (Authorization: Bearer <token>).
6	Resource Server verifies the signature and grants access if valid.

The Resource Server doesn't need to call the Auth Server — it verifies the token locally using the public key.

JWT signing methods

Algorithm	Type	Description
HS256	Symmetric	Shared secret key used for both signing and verifying.
RS256	Asymmetric	Private key signs, public key verifies.
ES256	Asymmetric	Uses elliptic curve cryptography (smaller, faster).

In OAuth2, RS256 is most common because:

- The Authorization Server signs with its private key.
- Resource Servers verify with its public key (published via `/jwks.json`).

Common JWT claims

Claim	Example Value	Meaning / Explanation
iss	"https://login.microsoftonline.com/72f988bf-86f1-41af-91ab-2d7cd011db47/v2.0"	The issuer — Microsoft's Authorization Server (tenant-specific). It shows who created and signed the token.
sub	"a1b2c3d4e5f6g7h8i9"	The subject, i.e., the unique ID of the logged-in user within Azure AD. This is how Microsoft uniquely identifies you.
aud	"https://graph.microsoft.com"	The audience, meaning this token is intended for Microsoft Graph API (used by Teams, Outlook, OneNote, etc.). If an app tries to use this token somewhere else, it'll be rejected.
exp	1734473600	The expiry time (UTC seconds since epoch). E.g., token valid for 1 hour after issuance.

Common JWT claims

Claim	Example Value	Meaning / Explanation
iat	1734470000	The issued at time — when Microsoft issued the token.
nbf	1734470000	The token is not valid before this time (prevents use before issuance).
scope	"User.Read Mail.Read Mail.Send Files.ReadWrite"	The OAuth scopes granted — meaning this token can read your profile, read/send mail, and access OneNote files.
jti	"8e7eebd5-2a3b-44d5-9a09-9d129ce4e42d"	A unique identifier for this token (used for revocation tracking).
roles	["Employee", "TeamsUser"]	Custom roles assigned to the user within the organization (for app-level access control).

Common JWT claims

Claim	Example Value	Meaning / Explanation
email	"abc@codedecode.com"	The user's email address — included as a user claim for personalization and correlation.
azp	"teams-client-id"	Authorized Party — identifies the Teams client app that obtained this token.
tid	"72f988bf-86f1-41af-91ab-2d7cd011db47"	The tenant ID (your company's Azure AD directory).
name	"Code Decode"	User's display name.
preferred_username	"Code.Decode@company.com"	Usually the login username or email used for SSO.

Analogy

Imagine:

- Microsoft = airport security office (Authorization Server)
- JWT = your boarding pass
- Teams, Outlook, OneNote = different gates

Your boarding pass (JWT) includes:

- Who issued it (Microsoft)
- Who you are (sub, name, email)
- Where you're allowed to go (scopes = flights)
- When it expires (exp)
- Verification signature (can't fake the pass)

Each gate (Teams/Outlook/OneNote) checks the pass before letting you through.

How JWTs are verified

- 1 Resource Server receives token → splits into header, payload, signature.
- 2 Uses the public key (from `/jwks.json`) to verify the signature.
- 3 Decodes claims → checks:

- Is exp still valid?
- Is aud correct?
- Is iss trusted?

When the Authorization Server (like Microsoft, Okta, Keycloak, or your Spring Authorization Server) issues a JWT,
it signs it with its private key

That signature ensures that only the issuer could have created this token —
and that it hasn't been modified since issuance.

The two keys: Private and Public

So the Resource Server (your API) needs a way to get the public key of the Authorization Server. That's where `/jwks.json` comes in.

Key Type	Who owns it	What it does	Shared?
Private key	Authorization Server	Used to sign the JWT	Secret (never shared)
Public key	Resource Server (APIs)	Used to verify the JWT's signature	Publicly available

What is /jwks.json?

/jwks.json = JSON Web Key Set — a public endpoint exposed by the Authorization Server that lists all public keys currently valid for verifying JWTs.

Each key inside it corresponds to a private key used to sign tokens.

Example of /jwks.json:

```
{  
  "keys": [  
    {  
      "kty": "RSA", // Uses two keys: private (sign), public (verify). // old-fashioned key lock  
      "use": "sig", // Used for signing/verifying JWTs.  
      "kid": "rsa-key-1",  
      "alg": "RS256",  
      "n": "u89DxYjK3i1n2h... (public modulus)",  
      "e": "AQAB"  
    }  
  ]  
}
```

kty: key type (RSA, EC, etc.)

use: What this key is used for

alg: Which algorithm was used to sign the token.

kid: A unique ID for this key. Helps match JWT → key in JWKS.

n, e: RSA public key components

This is public, so any API (resource server) can fetch it safely.

Analogies

- If **key** is the type of key, then **use** is the job it performs.

So when your API fetches `/jwks.json`, it knows:

“Ah, I need the key meant for signatures, not encryption.”

- If **key** is the type of lock, then **alg** is the brand and mechanism — e.g. “This RSA lock uses SHA-256 for security.”

So your JWT’s header says:

```
{ "alg": "RS256", "kid": "rsa-key-1" }
```

The API sees that and goes:

“I’ll find a key in JWKS with `alg=RS256` and `kid=rsa-key-1`.”

Analogies

When the Authorization Server signs a JWT, it includes in header:

```
{ "alg": "RS256", "kid": "rsa-key-1" }
```

The Resource Server (API) reads that kid and says:

“I’ll pick the public key from /jwks.json with the same kid.”

This is crucial when key rotation happens — multiple keys may exist, and kid tells which one was used.

If you have multiple locks on your office doors, each lock has a serial number. When someone gives you a signed document saying “verified by Lock #5,” you look up the right lock key (**kid** = 5) from your keyring.

Analogies

Think of RSA like a special lock:

n defines the shape of the lock (complex metal pattern).

e defines how the key turns in it.

The public key = (n, e)

In RSA, your public key has two parts:

n = modulus (large number)

e = exponent (small number, usually 65537 → Base64URL = "AQAB")

Your private key has:

n = same modulus

d = private exponent (mathematically related to e)

Analogies

So:

$(n, e) \rightarrow$ used to verify JWT signatures (public key).

$(n, d) \rightarrow$ used to create JWT signatures (private key).

Together, $n + e$ reconstruct the public key

This key can verify JWTs signed by the private half of the pair,
but **cannot be used to generate valid signatures** itself because d (the private exponent) is not known
hence they cant sign JWTs and make a valid sign hence its safe

How they work together

Step	JWT Header Field	Purpose
1	kty	What kind of key is this (RSA, EC)?
2	use	Is it for signing or encryption?
3	alg	What algorithm should be used (RS256)?
4	kid	Which key ID was used to sign this JWT?
5	n, e	The actual public key numbers to verify the signature.

How signature verification actually happens

1) Authorization Server (issuer)

Creates JWT → signs it with private key.

`RS256(privateKey, header + "." + payload)`

2) JWT is sent to client → client uses it in API requests.

3) Resource Server (API)

1. Receives Authorization: Bearer <JWT>.
2. Decodes the JWT header:

```
{ "alg": "RS256", "kid": "rsa-key-1" }
```


How signature verification actually happens

- alg → which algorithm to use.
- kid → which public key to fetch.

3) Fetches the Authorization Server's JWKS:

GET <https://auth-server.com/oauth2/jwks>

4) Finds the key where kid = rsa-key-1.

5) Uses that public key to verify the signature.

Verification math (simplified)

Let's say the JWT =

<encoded header>.<encoded payload>.<signature>

The Resource Server:

- 1) Recomputes the expected signature:

```
verifySignature(publicKey, header + "." + payload)
```

- 2) Compares it with the received signature.
- 3) If they match → token is valid (not tampered).
- 4) If not → reject (someone tried to alter it or use a fake token).

Why /jwks.json is needed

Without /jwks.json, the Resource Server wouldn't know:

- Which key to use,
- Or whether the token was actually signed by your Authorization Server.

JWKS provides a dynamic, trusted list of valid public keys — allowing:

- Key rotation (you can add new keys without breaking verification),
- Multi-tenant validation (multiple keys per issuer),
- Decentralized verification (no need to call the Auth Server for every request).

Example – Microsoft

Microsoft exposes this publicly for all Azure AD tenants:

<https://login.microsoftonline.com/common/discovery/v2.0/keys>

When a Microsoft Graph API gets a JWT access token from Teams, Outlook, etc., it:

1. Reads kid from the token header.
2. Fetches the matching key from /discovery/v2.0/keys.
3. Uses that public key to verify the JWT signature.
4. If valid → trusts the token.

Analogy

Think of this like a notary seal 🕵️ :

- The Authorization Server (notary) stamps your document with a private seal.
- The Resource Server (recipient) checks the seal's authenticity using the public template stored in `/jwks.json`.
- If it matches → signature is valid → document (token) is genuine.

In one line:

`/jwks.json` provides the public key(s) that Resource Servers use to verify the digital signature of JWTs issued by the Authorization Server — ensuring the token is authentic and untampered.

Why JWTs are so popular

Benefit	Description
Stateless	Server doesn't need to store sessions. All info is inside the token.
Fast	Verification is local — no DB or introspection call.
Portable	Can be used across microservices easily.
Integrity-protected	Can't be modified without breaking the signature.
Cross-language	JSON + Base64 = easily parsed by any language.

Limitations / Pitfalls`

Issue	Description
Revocation	Once issued, JWTs can't easily be revoked before expiry.
Sensitive data exposure	Claims are Base64-encoded, not encrypted. Anyone can decode them.
Size	Larger than opaque tokens.
Key rotation	You must manage key rotation securely.
Expiry management	Must combine with refresh tokens to avoid constant re-login.

JWK & JWKS

JWK (JSON Web Key) → JSON representation of a cryptographic key.

Example:

```
{  
  "kty": "RSA",  
  "use": "sig",  
  "kid": "rsa-key",  
  "alg": "RS256",  
  "n": "base64url-modulus",  
  "e": "AQAB"  
}
```

JWKS (JSON Web Key Set) → A collection of JWKs, usually published at:

<https://auth-server.com/oauth2/jwks>

JWK & JWKS

Resource Servers fetch this to verify JWT signatures.

This is exactly why you define:

```
@Bean  
public JWKSource<SecurityContext> jwkSource() { ... }
```

— to make your Authorization Server publish these keys.

JWT usage in OAuth2 & OIDC

Context	Usage
Access Token	Grants access to APIs.
ID Token	Used in OpenID Connect to share user identity info.
Refresh Token	Usually opaque for safety (not JWT).

Analogy

Think of a JWT as a sealed envelope:

- Anyone can open it and read what's inside (because it's only encoded).
- But only the issuer can seal it with a unique wax stamp (the signature).
- If anyone tampers with it, the seal breaks and it's rejected.

Key points to remember

Concept	Example / Purpose
Microsoft issues the JWT	via https://login.microsoftonline.com/.../v2.0
JWT's audience (aud)	Always https://graph.microsoft.com for Microsoft Graph APIs
Scopes (scope)	Define what app can do (Mail.Read, Chat.ReadWrite, etc.)
Roles (roles)	App-specific or org-level roles (TeamsUser, Admin)
Resource Server verifies JWT	Uses Microsoft's JWKS endpoint to verify signature



OIDC

Analogy

Think of a JWT as a sealed envelope:

- Anyone can open it and read what's inside (because it's only encoded).
- But only the issuer can seal it with a unique wax stamp (the signature).
- If anyone tampers with it, the seal breaks and it's rejected.



Authorization server implementation

Basics

The Authorization Server is the central brain of OAuth2 — it's the component that authenticates users and clients, issues tokens (access, refresh, ID), and validates or revokes them.

In short:

It's the “identity & permissions provider” in your OAuth2 setup.

Core Responsibilities

Function	Description
Authenticate users	Verify user credentials (login screen, MFA, SSO, etc.)
Authenticate clients	Verify apps (using client_id & client_secret or mTLS)
Issue tokens	Create access tokens, refresh tokens, ID tokens
Validate tokens	Expose /introspect for opaque token validation
Revoke tokens	Allow /revoke to invalidate tokens
Expose metadata	Provide discovery (/well-known/openid-configuration)
Publish public keys	For JWT verification via /jwks.json endpoint

Key endpoints an Authorization Server provides

Endpoint	Purpose
/authorize	Handles user login and consent (for authorization code flow).
/token	Issues access and refresh tokens.
/introspect	Lets Resource Servers verify opaque tokens.
/revoke	Allows clients to revoke access/refresh tokens.
/jwks.json	Publishes public signing keys for JWT verification.
/.well-known/openid-configuration	(OIDC) Metadata about supported endpoints and claims.

Implementation steps

Step 1 — Add dependency

In your pom.xml:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-authorization-server</artifactId>  
</dependency>
```

Implementation steps

```
@Bean
@Order(1)
public SecurityFilterChain authServerSecurityFilterChain(HttpSecurity http) throws Exception {
    OAuth2AuthorizationServerConfigurer authorizationServerConfigurer = new
    OAuth2AuthorizationServerConfigurer();

    http
        .securityMatcher(authorizationServerConfigurer.getEndpointsMatcher())
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .csrf(csrf -> csrf.ignoringRequestMatchers(authorizationServerConfigurer.getEndpointsMatcher()))
        .with(authorizationServerConfigurer, (authServer) ->
            authServer.oidc(Customizer.withDefaults()) // Enable OpenID Connect (optional)
        )
        .formLogin(Customizer.withDefaults());

    return http.build();
}
```

Implementation steps

@Bean and @Order(1)

You are defining a Spring Security filter chain bean.

Each SecurityFilterChain defines a set of security rules for certain URL patterns.

@Order(1) means:

“This security configuration should be applied before other filter chains.”

That’s because the Authorization Server endpoints (/oauth2/token, /oauth2/authorize, /oauth2/jwks, etc.) must be secured first, before any application endpoints.

In short:

This filter chain handles only OAuth2 Authorization Server–related endpoints.

Implementation steps

```
OAuth2AuthorizationServerConfigurer authorizationServerConfigurer = new  
OAuth2AuthorizationServerConfigurer();
```

This object knows how to register all Authorization Server–specific endpoints and filters into Spring Security.

It internally:

- Registers /oauth2/token, /oauth2/authorize, /oauth2/jwks, /oauth2/introspect, /oauth2/revoke
- Adds filters for:
 - Authorization code issuance
 - Token issuance
 - JWT signing
 - Introspection
 - OIDC endpoints (if enabled)

Implementation steps

So instead of manually declaring all those filters, this one line brings in the entire Authorization Server behavior.

In short:

It's the “module” that converts this app into an Authorization Server.

Implementation steps

`.securityMatcher(authorizationServerConfigurer.getEndpointsMatcher())`

This line says:

“Apply this security configuration only to Authorization Server endpoints.”

Each filter chain in Spring Security needs to know which requests it applies to.
`authorizationServerConfigurer.getEndpointsMatcher()` returns a matcher that includes:

- `/oauth2/authorize`
- `/oauth2/token`
- `/oauth2/jwks`
- `/oauth2/introspect`
- `/oauth2/revoke`

In short:

This filter chain won't interfere with your normal app routes like `/api/**`.

Implementation steps

`.authorizeHttpRequests(auth -> auth.anyRequest().authenticated())`

This ensures:

All Authorization Server endpoints require authentication.

So:

- To access `/oauth2/authorize`, the user must log in.
- To access `/oauth2/token`, the client must authenticate (with client credentials or code verifier).

In short:

No public access — all OAuth endpoints are protected.

Implementation steps

```
.csrf(csrf ->  
csrf.ignoringRequestMatchers(authorizationServerConfigurer.getEndpointsMatcher()))
```

CSRF protection (Cross-Site Request Forgery) is designed for browser forms, not for machine-to-machine token requests.

Token endpoints (like /oauth2/token) use POST requests with credentials and don't include CSRF tokens — so you disable CSRF only for those endpoints.

In short:

Turn off CSRF checks just for OAuth2 endpoints — not globally.

Implementation steps

`.with(authorizationServerConfigurer, (authServer) ->authServer.oidc(Customizer.withDefaults()))`

This is the new Spring Security 6.2+ style of applying a configurer (replacing `.apply()`).

- You are attaching the `authorizationServerConfigurer` to `HttpSecurity`.
- The lambda `(authServer) -> authServer.oidc(...)` lets you customize its behavior.

`authServer.oidc(Customizer.withDefaults())` means:

“Enable OpenID Connect (OIDC) endpoints, with default behavior.”

That includes endpoints like:

`/userinfo`

`/openid-connect/jwks`

`/openid-configuration`

Implementation steps

and issuing ID tokens along with access tokens when requested.

In short:

Enables OIDC support — users can log in and get ID tokens for identity info.

(If you don't need OIDC, you can safely remove this line.)

Implementation steps

.formLogin(Customizer.withDefaults())

Adds a basic login form for user authentication.

So when a user goes to /oauth2/authorize,
Spring Security shows the login page (unless already authenticated).

In short:

Provides a built-in username/password login screen for testing and basic auth.

return http.build();

This finalizes the HttpSecurity configuration and builds the filter chain bean that Spring Security uses at runtime.

In short:

“Activate everything above.”

Summary of the entire method

Line	Purpose
@Order(1)	Run before other filter chains
new OAuth2AuthorizationServerConfigurer()	Registers all OAuth2 endpoints
.securityMatcher(...)	Apply rules only to those endpoints
.authorizeHttpRequests()	Require authentication for them
.csrf(...ignoring...)	Disable CSRF only for token endpoints
.with(...oidc(...))	Enable OpenID Connect (optional)
.formLogin(...)	Add login form for user login
.build()	Construct and register the chain



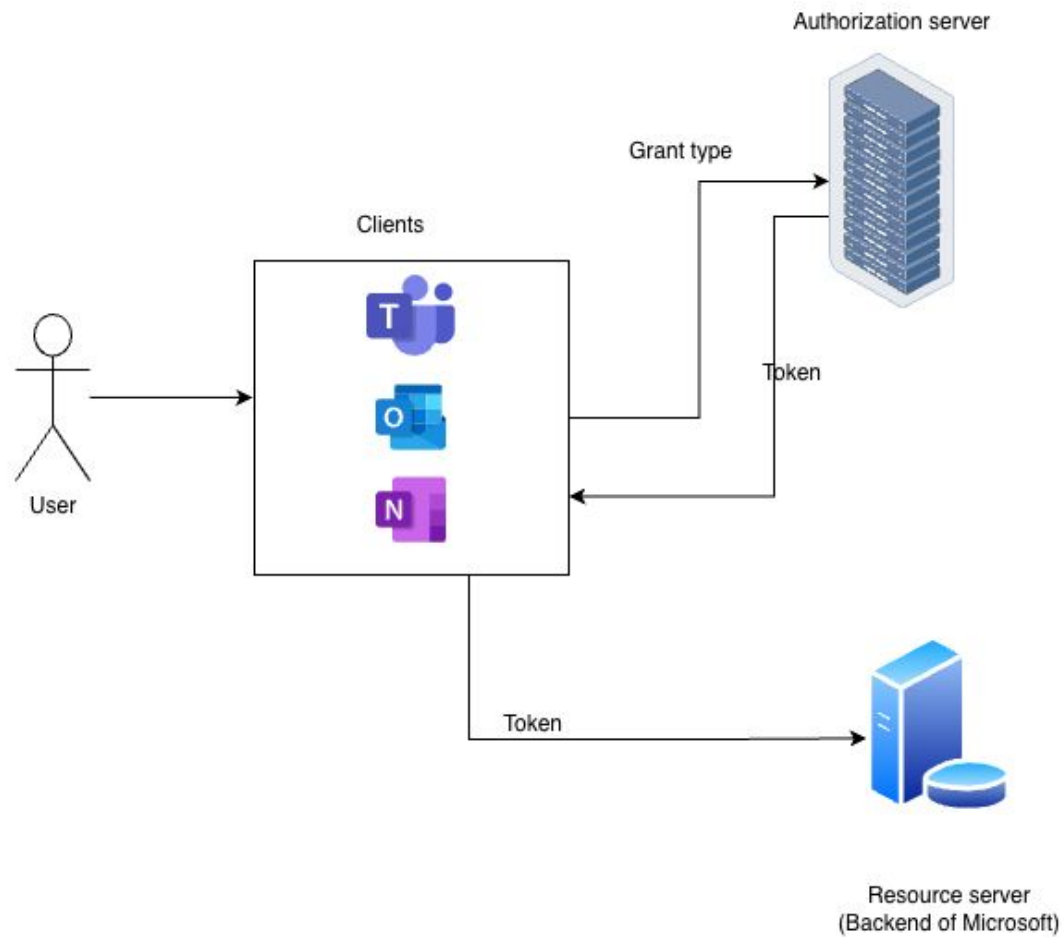
OIDC

What is OIDC

OIDC is “an identity layer on top of OAuth2.”

OIDC uses the OAuth2 flow, but adds extra information for identity.

Layer	Purpose
OAuth2	Grants permission (like access tokens for APIs)
OIDC	Adds user identity info (who logged in) via ID Token



Authorization code flow with PKCE

Request - GET [https://login.microsoftonline.com/authorize?](https://login.microsoftonline.com/authorize?response_type=code&client_id=teams-client-id&redirect_uri=https://teams.microsoft.com/callback&scope=openid%20profile%20email&code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM&code_challenge_method=S256)

`response_type=code`

`&client_id=teams-client-id`

`&redirect_uri=https://teams.microsoft.com/callback`

`&scope=openid profile email`

`&code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM`

`&code_challenge_method=S256`

Server returns tokens in response:

```
{  
  "access_token": "eyJhbGciOi...",  
  "refresh_token": "8xLOxBtZp8",  
  "id_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "expires_in": 3600  
}
```

`scope=openid` tells the OP “I want OIDC (ID Token).” In Authorization Code flow, you get both when you requested `scope=openid`. If you didn’t include `openid`, the OP will usually return only the `access_token`.

Real-world example (Google Login)

When you click “Sign in with Google”:

Your app redirects to Google’s OIDC URL (accounts.google.com).

Google asks you to log in.

Google redirects back to your app with an `id_token`.

Your app checks the token → confirms your identity.

You’re now logged in to your app — without creating a new password.

So in 1 line -

OIDC lets apps authenticate users safely using trusted identity providers (like Google, Microsoft, or your own Authorization Server), and it gives your app a signed token (ID Token) that proves who the user is

Why OIDC, Doesn't Auth server does authentication?

In both OAuth 2.0 and OIDC, the Authorization Server (AS) is the component that:

- Authenticates the user (verifies username/password, MFA, etc.),
- And issues tokens (code, access_token, etc.).

So yes — user authentication happens there and access token contains some user info in claims

The problem is - Client (like Microsoft Teams) doesn't know anything about the user because it redirects to the Authorization Server, where the user enters login details. So Teams never sees the username or password.

OAuth2 alone only gives an access token (to call APIs).

It doesn't say who the user is.

So if your app just wants to show “Hello Alice,

OAuth2 doesn't help — you need OIDC to get user identity info.

That's why OIDC was created — to fill the authentication gap in OAuth2.

OAuth 2.0's perspective

OAuth2 was never designed for authentication (login).

It was designed for authorization (granting access to resources).

Let's take a practical OAuth2 example:

Suppose you connect a 3rd-party app (like “Teams”) to your chats.

- The Authorization Server (Microsoft Entra ID) will authenticate you (the resource owner).
- It will then issue an access token to Teams, so Teams can read your chats from backend server where they are stored.

Yes, authentication happened,
but OAuth2 doesn't tell Teams who you are — only that you granted permission.

Why that's a problem for “login” purposes

The OAuth2 access token:

- Is meant for APIs, not for client apps to know who you are.
- Is often opaque (random string), or even if it's a JWT, it's for the Resource Server, not for the client.

So the client (like Teams) cannot safely rely on that access token to know:

“Who just logged in?”

The access token says:

“Someone who had permission can call this API.”
But not who that person is.

This was the gap OIDC fixed.

OIDC's perspective — making “login” official

OIDC said:

“Let’s take OAuth2, and officially define a safe, verifiable way to identify the user.”

It does this by introducing:

- An ID Token → a signed token specifically for authentication.
- Standard claims (sub, email, name, etc.) inside that ID token.
- Rules for how to validate it (iss, aud, exp, nonce).

So now the app (client) can safely say:

“This user really is Code Decode, and Microsoft authenticated him at 2:01 PM.”

That’s something pure OAuth2 could not promise.

Why not just use the access token for user info?

Issue	Why Access Token Can't Be Used for Login
Audience mismatch	Access tokens are intended for Resource Servers (APIs) — not for the client app. Their aud claim is the API, not the client.
Contents vary	Some are opaque; even JWTs have unpredictable claims, and structure depends on provider.
No guaranteed identity fields	OAuth2 doesn't define sub, email, etc. — those are OIDC's additions.
No signature verification rules	OAuth2 doesn't specify how clients should validate tokens.
No nonce	OAuth2 tokens can't be safely tied to a specific login session.
No authentication time	You don't know when the user actually authenticated (could be days ago).

Real world analogy - why access token can't be used

Type	Analogy	Purpose
Access Token (OAuth2)	Door key	Opens certain rooms (APIs). Doesn't say who you are — only that you have permission.
ID Token (OIDC)	Photo ID card	Proves who you are — with your photo, name, and signature.

Both come from the same building security desk (Authorization Server), but one is for access control, the other for identity verification.

Where authentication really happens

In OAuth2:

- The user is authenticated by the Authorization Server.
- But the token returned (access token) is meant for APIs, not the client.
- So the client app has no safe way to verify “who” logged in.

In OIDC:

- The same Authorization Server performs authentication.
- But now it also returns a signed ID Token specifically for the client (Relying Party) to verify who logged in.

So the authentication process is the same,
but OIDC formalizes the identity communication between the Authorization Server and the client app.

How they differ in token contents

Token	Used By	Contains	Audience	Purpose
Access Token	Resource Server (API)	Scopes, permissions, sometimes sub	API	Grants access to data
ID Token	Client App (RP)	User's identity info	Client ID	Verifies who the user is

Example:

Access token → “Can read user’s chats.”

ID token → “User = Code decode, verified by Microsoft.”

So, final truth:

- The Authorization Server authenticates the user.
- But OAuth2 does not standardize how to pass that identity to clients.
- OIDC does — using the ID Token.

ID Token = OIDC / authentication token. You get it only when OIDC is requested (**i.e. scope=openid**) or when an OIDC flow explicitly returns it. It proves who the user is.

Access Token = OAuth2 / authorization token. You get it when the client needs to call APIs (resource servers). It proves what the client can access.

Which Grant Type flow return IDToken?

Flow / Request	ID Token returned?	Access Token returned?	Notes
Authorization Code (with scope=openid)	Yes returned at token exchange	Yes — returned at token exchange.	Recommended: use PKCE for public clients.
Authorization Code (without openid)	No	Yes	Classic OAuth2 (no identity info).
Client Credentials	No	Yes	Machine-to-machine; no user => no ID token.
Device Authorization Flow	Yes (if scope=openid during device/user auth)	Yes	For devices without browser input.
Refresh Token grant	Maybe — OP may return new ID token if requested	Yes (new access token)	ID Token often returned if scope or OP policy asks.

What Problem OIDC Solves — “Why OIDC Exists

The core issue before OIDC:

OAuth 2.0 was created for **authorization** —

“Can this app access a user’s data?”

But it didn’t handle **authentication** —

“Who is this user?”

For example:

- OAuth2 lets an app get permission to access an API (like Google Calendar).
- But OAuth2 doesn’t define how to confirm who the user is — only that the user granted permission.

What Problem OIDC Solves — “Why OIDC Exists

That created confusion.

Developers started using OAuth2 for login — but in slightly different, insecure ways.

So, the industry needed a standard way to log users in using OAuth2 — safely and consistently.

That standard is OpenID Connect (OIDC).

What OIDC Actually Is

OIDC = OAuth 2.0 + Identity Layer

It uses OAuth2's existing flow and tokens.

But adds:

- ID Token → a secure, signed token that tells “who” the user is.
- Standard endpoints (for discovery, user info, public keys).
- Extra security parameters (nonce, at_hash, etc.).

So now, we can do both:

- Authentication (via OIDC — identify the user),
- Authorization (via OAuth2 — control access to APIs).

Real-world analogy

- Imagine an office building with many office rooms (apps: Teams, Outlook, OneNote).
- The Security Reception Desk = the **Authorization Server** / Identity Provider (OP)
(handles login and issues ID badges).
- Each office room (Teams, Outlook, etc.) = a **Client** / Relying Party (RP)
(relies on the ID badge to identify you).
- The ID badge = **the ID Token** (issued after login).
It says “You are Code Decode, employee #12345, valid for 8 hours.”
- The access key card = **the Access Token** (OAuth2)
It allows you to open specific doors (access certain APIs).
- So OIDC = the identity check part of this system.

Core Components in OIDC

Component	Role	Description
User (Resource Owner)	The person trying to log in	You
Client / Relying Party (RP)	The app the user wants to use	Teams, Outlook, OneNote
Authorization Server / OpenID Provider (OP)	Handles login and issues tokens	Microsoft Entra ID / Google Identity / Auth0
Resource Server	The API that the access token is used for	Microsoft Graph API, Outlook API

Core Components in OIDC

Component	Role	Description
ID Token	Proof of who the user is	Signed JWT issued by OP
Access Token	Proof of what the user can access	Usually another JWT or opaque token
Refresh Token	Used to get new tokens	Long-lived
UserInfo Endpoint	API that gives user details	e.g., /userinfo
JWKS Endpoint	Public keys for verifying token signatures	e.g., /jwks.json
Discovery Endpoint	Lists all endpoints & capabilities	e.g., /.well-known/openid-configuration

OIDC Builds on OAuth 2.0 – What It Adds

Area	OAuth 2.0	OIDC Adds
Goal	Authorization (permissions)	Authentication (identity)
Token	Access Token	ID Token (JWT with user identity)
Flow	Authorization Code, Implicit, etc.	Authorization Code + PKCE (recommended)
Endpoints	/authorize, /token	/userinfo, .well-known/openid-configuration, /jwks
Security	State, HTTPS	Nonce, ID Token signature, claims
Output	Permissions (scopes)	Verified user identity
Audience	APIs	Apps (RPs)

What is an ID Token?

The ID Token is the heart of OIDC.

It's a JWT (JSON Web Token) — a signed, tamper-proof JSON object that says:

“This user has been authenticated by the OpenID Provider.”

Example ID Token:

```
{  
  "iss": "https://login.microsoftonline.com/{tenant-id}/v2.0",  
  "sub": "5c143fd2-13f9-4019-aaa8-4f5d1adf0cce",  
  "aud": "teams-client-id",  
  "exp": 1734405600,  
  "iat": 1734402000,  
  "nonce": "abc123xyz",  
  "name": "Code decode",  
  "email": "code.decode@company.com"  
}
```

What is an ID Token?

Key features:

- Signed by the provider's private key.
- Verified by your app using the public key (JWKS).
- Contains identity info like user ID, email, etc.
- Has expiry time to prevent reuse.
- Has a nonce to prevent replay attacks.

What is Nonce- A nonce is a random, one-time value created by the client (app) and sent to the Authorization Server during login. Later, the same value must come back inside the ID Token. If it doesn't match → the login is rejected.

Standard Claims in an ID Token

Claim	Meaning
iss	Issuer — the OP that created the token
sub	Subject — unique user ID
aud	Audience — your app's client_id
exp	Expiration time
iat	Issued at time
nbf	Not before time
nonce	Random value from the request to prevent replay
auth_time	When user logged in

Standard Claims in an ID Token

Claim	Meaning
acr, amr	How user authenticated (password, MFA, etc.)
email, name	Profile info (if scope allows)
at_hash	Hash of access token (binds ID token to access token)

OIDC Scopes

Scopes tell the OP what you want to know about the user. Eg scope=openid profile email offline_access

Scope	Description
openid	Required — activates OIDC
profile	Get user's name, picture, etc.
email	Get user's email address
address	Get address info
phone	Get phone number
Custom scopes	For app-specific data

OIDC Endpoints (every provider must have these)

Endpoint	Purpose
/authorize	Start login (user redirected here)
/token	Exchange authorization code for tokens
/userinfo	Fetch user profile (with access token)
/jwks.json	Get public keys to verify token signatures
/.well-known/openid-configuration	Discovery document listing all above

The OIDC Flow – Step-by-Step (Authorization Code + PKCE)

1) User opens app (Client / RP)

App (e.g., Teams) says:

“I need to know who this user is → redirect to login.”

It redirects the user's browser to the Authorization Server (OP):

```
GET https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize?  
  response_type=code  
  &client_id=teams-client-id  
  &scope=openid profile email  
  &redirect_uri=https://teams.microsoft.com/callback  
  &state=xyz  
  &nonce=abc  
  &code_challenge=BASE64URL(SHA256(verifier))  
  &code_challenge_method=S256
```

The OIDC Flow – Step-by-Step (Authorization Code + PKCE)

2) User logs in at the OpenID Provider

1. Microsoft (the OP) shows login form.
2. User authenticates (password, MFA, etc.).
3. OP creates an authorization code (temporary one-time code).
4. OP redirects back:

<https://teams.microsoft.com/callback?code=SpIxlOBcZQQYbYS6WxSblA&state=xyz>

The OIDC Flow – Step-by-Step (Authorization Code + PKCE)

3) App (RP) exchanges the code for tokens

Teams sends:

POST /token

grant_type=authorization_code

code=SpIxIOBeZQQYbYS6WxSbIA

redirect_uri=https://teams.microsoft.com/callback

client_id=teams-client-id

code_verifier=<original_verifier>

The OIDC Flow – Step-by-Step (Authorization Code + PKCE)

Response:

```
{  
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOi...",  
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "refresh_token": "tGzv3JOkF0XG5Qx2TIKWIA",  
  "expires_in": 3600,  
  "scope": "openid profile email"  
}
```

The OIDC Flow – Step-by-Step (Authorization Code + PKCE)

4) App verifies ID Token

App (Teams) checks:

- Signature (using OP's public key)
- iss, aud, exp, nonce
- If valid → user is authenticated

Now it knows: “User = Code decode, [code.decode@company.com](#)”

5) App can call Resource Servers

Teams can now use the access token to call Microsoft Graph APIs:

GET <https://graph.microsoft.com/v1.0/me>

Authorization: Bearer eyJ0eXAiOiJKV1Qi...

Where OIDC fits with OAuth2 (side by side)

Step	OAuth2	OIDC
Start	App redirects to /authorize	Adds scope=openid and nonce
Token exchange	Get access token	Adds ID token
Response	Access token only	Access + ID token
Purpose	Authorization	Authentication
Validates	Permissions	User identity

Example with Microsoft (real URLs)

Component	Example URL
Authorize	https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize
Token	https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token
Discovery	https://login.microsoftonline.com/{tenant}/v2.0/.well-known/openid-configuration
JWKS	https://login.microsoftonline.com/common/discovery/v2.0/keys
UserInfo	https://graph.microsoft.com/oidc/userinfo

Why OIDC is Secure

- Uses digital signatures (RS256, ES256) — tokens can't be faked
- Nonce prevents replay attacks.
- PKCE prevents code interception.
- HTTPS protects tokens in transit.
- Scopes limit permissions.
- Short-lived tokens reduce risk.
- Key rotation via JWKS ensures ongoing safety.

Benefits of OIDC

Benefit	Explanation
SSO	Log in once → all apps trust same identity
Passwordless	Apps never see passwords — provider handles login
Standardized	Works with any compliant provider (Google, Microsoft, Okta, Auth0, etc.)
Secure	Uses JWTs, signatures, HTTPS, nonce
Developer-friendly	Simple JSON-based format, discovery endpoints
Extensible	You can add custom claims/scopes easily

Common OIDC Providers (OPs)

Provider	Discovery URL
Microsoft Entra ID	https://login.microsoftonline.com/common/v2.0/.well-known/openid-configuration
Google	https://accounts.google.com/.well-known/openid-configuration
Auth0	https://YOUR_DOMAIN/.well-known/openid-configuration
Okta	https://YOUR_OKTA_DOMAIN/.well-known/openid-configuration
Keycloak	https://your-keycloak-domain/realms/your-realm/.well-known/openid-configuration

Without OIDC — What breaks?

Without OIDC:

- No standard ID Token (no way to confirm identity).
- Apps implement their own login logic.
- SSO becomes inconsistent.
- You might misuse OAuth2 access tokens for identity (bad practice).
- User identity trust breaks across systems.

OIDC fixes all that with one consistent standard.

Common pitfalls

Mistake	Consequence
Not validating ID token signature	Anyone could forge identity
Using implicit flow	Tokens exposed in URL (deprecated)
Not checking nonce	Replay attacks possible
Storing tokens in localStorage	XSS risk
Not validating aud/iss	Could accept token from another provider
Long token lifetimes	Higher security exposure

Summary (OIDC in one slide)

Concept	Purpose	Example
OIDC	Adds authentication to OAuth2	“Who is the user?”
ID Token	Proves user identity	code.decode@company.com
Access Token	Grants API access	“Can read calendar data”
OpenID Provider (OP)	Issues tokens	Microsoft Entra ID
Relying Party (RP)	Uses tokens	Teams / Outlook / OneNote
Scopes	Request user info	openid profile email

Summary (OIDC in one slide)

Concept	Purpose	Example
Endpoints	Standard OIDC URLs	/authorize, /token, /userinfo
Main flow	Authorization Code + PKCE	Most secure
Security	Signatures, Nonce, PKCE	Prevent tampering
SSO	Log in once, access all apps	Microsoft 365 experience

In one simple line:

OpenID Connect (OIDC) is the modern, secure way for apps to authenticate users using OAuth2 — it gives the app a signed ID Token that proves who the user is, enables Single Sign-On, and ensures your app never handles passwords directly.

What is Nonce

A nonce is a random, one-time value created by the client (app) and sent to the Authorization Server during login.

Later, the same value must come back inside the ID Token.

If it doesn't match → the login is rejected.

Why Nonce

Why does nonce exist?

To prevent replay attacks.

The problem without nonce (simple story)

Imagine this happens:

- Yesterday, you logged in and received an ID Token.
- An attacker somehow steals that old ID Token.
- Today, the attacker tries to send that old ID Token to your app.
- Your app sees a valid signature and expiry → might accept it 😱

That's called a replay attack.

How nonce fixes Replay attack

Step 1 Client generates a nonce

The client creates a random value:

```
nonce = "n-0S6_WzA2Mj"
```

Step 2 Client sends it in /authorize

```
GET /authorize?  
  response_type=code  
  &client_id=app123  
  &scope=openid profile  
  &redirect_uri=https://app/callback  
  &nonce=n-0S6_WzA2Mj
```

How nonce fixes Replay attack

Step 3 Authorization Server stores it temporarily

The server remembers:

“For this login request, nonce = n-0S6_WzA2Mj”

Step 4 Server puts nonce into the ID Token

```
{  
  "sub": "12345",  
  "email": "user@company.com",  
  "nonce": "n-0S6_WzA2Mj",  
  "iss": "https://login.example.com",  
  "aud": "app123",  
  "exp": 1734405600  
}
```

How nonce fixes Replay attack

Step 5 Client verifies the nonce

Client checks:

nonce in ID Token == nonce I generated?

Match → login is valid

OIDC spec requires nonce for flows that return an ID Token via the browser.

Summary - Nonce

One-line analogy

Nonce is like a claim ticket.

You give the ticket number when you drop your bag, and you only accept the bag back if the same number is on it.

One-line definition (exam-ready)

Nonce is a client-generated, single-use random value included in the authentication request and echoed in the ID Token to prevent replay attacks.

SS0

What is SSO

You log in once to Microsoft Entra ID - Authorization server, and then you can open Teams, Outlook, and OneNote without logging in again.

That's it.

Everything else is how Microsoft achieves this.

Core SSO rule -

SSO works because Teams, Outlook, and OneNote ALL trust the SAME Authorization Server — Microsoft Entra ID.

If even one app trusted a different auth server → SSO breaks.

Actors in SSO

Component	In Microsoft world
User	You (employee)
Identity Provider / Auth Server	Microsoft Entra ID
Apps (Clients / RPs)	Teams, Outlook, OneNote
Resource Servers	Teams API, Outlook API, OneNote API
Protocol	OAuth2 + OpenID Connect (OIDC)

Steps in SSO

Step 1: You start your day — open Microsoft Teams

What Teams does (internally)

Teams says:

“I don’t know who this user is.
Let me ask Microsoft Entra ID.”

So Teams redirects your browser to:

<https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize>

At this moment:

- ☐ Teams knows nothing about you
- ☐ Teams does not have your password
- ☐ Teams does not authenticate you

Steps in SSO

Step 2: Authentication happens ONLY at Microsoft Entra ID

You now see the Microsoft login page.

You: Enter email, Enter password

Maybe MFA / OTP / biometric

What Entra ID does -

Verifies your credentials

Confirms you are a valid employee

Applies company policies (MFA, device check, etc.)

Authentication is complete

Important:

SSO is NOT about Teams authenticating you

SSO is about Entra ID authenticating you ONCE

Steps in SSO

Step 3: The MOST IMPORTANT SSO STEP (session creation)

After successful login, Microsoft Entra ID creates a session.

Technically:

A session cookie is stored in your browser

Domain: login.microsoftonline.com

Example (conceptual):

Cookie: ENTRA_SESSION = abc123

Domain: login.microsoftonline.com

THIS SESSION IS THE HEART OF SSO

As long as this session exists:

You are considered “logged in” by Microsoft

Steps in SSO

Step 4: Tokens are issued to Teams

Now Entra ID sends Teams:

ID Token → Who you are

Access Token → What Teams APIs you can access

Teams:

Verifies token signature

Reads your identity

Creates a local Teams session

You are now inside Teams.

Up to now: what has happened?

Thing	Where it happened
Authentication	Microsoft Entra ID
Password handling	ONLY Entra ID
Session created	Entra ID
Identity shared	ID Token
App login	Teams

This is NOT SSO yet — this is just login.

Steps in SSO

Step 5: Now you open Outlook (this is where SSO appears)

You open Outlook (web or app).

What Outlook does

Outlook says:

“I need to know who this user is.
I also trust Microsoft Entra ID.”

So Outlook also redirects to:

<https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize>

Steps in SSO

Step 6: Magic moment — Entra ID checks its session

Entra ID checks:

“Do I already have an active session for this user?”

YES — from when you logged into Teams.

So Entra ID:

- ☐ Does NOT show login page
- ☐ Does NOT ask for password
- ☐ Immediately issues tokens

This is SSO IN ACTION.

Steps in SSO

Step 7: Outlook receives tokens and logs you in

Outlook receives:

- ❑ ID Token (who you are)
- ❑ Access Token (API permissions)

Outlook verifies token → creates local session → inbox opens.

You never logged in again.

Steps in SSO

Step 8: Same thing with OneNote

OneNote repeats the same steps:

- ☐ Redirects to Entra ID
- ☐ Entra ID sees active session
- ☐ Issues tokens
- ☐ No login prompt

Key - Why SSO works

Single Identity Provider (Entra ID) + Shared Trust by All Apps + Central Session = Single Sign-On

Common Pitfalls Of SSO

SSO does NOT mean:

- ☐ Same password stored everywhere
- ☐ Apps sharing cookies
- ☐ Apps talking to each other

SSO means:

- ☐ All apps trust one identity authority
- ☐ Identity authority maintains one session
- ☐ Apps only consume tokens

Security benefit of SSO

Without SSO	With SSO
Teams stores password	Teams never sees password
Outlook stores password	Outlook never sees password
Multiple login prompts	One login
Weak security	Central MFA & policies
Hard to revoke access	Disable user in Entra ID

SSO Logout Scenarios

- Logout from Teams only
 - ❑ Teams session destroyed
 - ❑ Entra ID session still alive
 - ❑ Outlook & OneNote still logged in
- Logout from Microsoft account (global)
 - ❑ Entra ID session destroyed
 - ❑ All apps will require login again
- Session expires
 - ❑ Entra ID session expires
 - ❑ Next app access → login required

Why OAuth2 + OIDC are REQUIRED for SSO

Piece	Why needed
OAuth2	Secure token issuance
OIDC	Standard identity sharing (ID Token)
Tokens	Passwordless trust
Redirect flow	Apps never handle credentials
Sessions	Enable SSO

SSO is not a protocol

SSO is the result of using these correctly.

SSO Summary

- In Microsoft SSO, you authenticate once with Microsoft Entra ID, which creates a central session. Teams, Outlook, and OneNote trust that session and receive tokens instead of passwords, so you never log in again.



CSRF

What is CSRF

Lets Understand this with a story

Step 1: You log in to a website

- You log in to:
mybank.com
- Now your browser remembers:
“User is logged in to mybank.com”

Step 2: You visit a bad website

- You open:
xryevill.com
- You are just browsing. You don't click anything suspicious.

What is CSRF

Step 3: Evil website secretly tells your browser:

“Hey browser, load this link ”

`https://mybank.com/sendMoney?to=hacker&amount=10000`

Step 4: Browser obeys (this is the danger)

Your browser says:

“Oh, this is mybank.com — I’ll attach login cookies.”

So it sends:

Request to mybank.com

+ your login cookie

Remember CSRF is possible only with PUT/POST/DELETE Not with GET

What is CRSF

Step 5: Bank gets the request

The bank sees:

- Valid login cookie

- Valid request format

So it thinks:

“The user asked for this.”

Money sent

You never intended it

This is CSRF.

What is CSRF

What you did NOT do

- You did not type your password
- You did not click “Send money”
- You did not approve the action

Your browser was tricked.

CSRF is when a website secretly uses your already-logged-in browser to perform actions on another website.

Core browser behavior

Browsers automatically:

- attach cookies to requests for the target website's domain

So if you are logged in to a site, every request from your browser to that site includes your session cookie — even if the request was triggered by another website.

This is the root of CSRF.

Simple real-life analogy

Imagine:

- You sign a blank cheque
- You leave it on your desk
- Someone else fills the amount and submits it

You didn't intend to pay —
but the system trusts the signature.

That's CSRF.

Authentication proves who you are.

CSRF attacks abuse the fact that you are already authenticated.

Why CSRF is dangerous

Reason	Explanation
Browser auto-sends cookies	Attacker abuses this
User already authenticated	No login needed
No user interaction	Happens silently
Looks legitimate	Server can't tell difference

Common confusion for CSRF

CSRF is NOT:

- Hacking passwords
- Stealing cookies
- Breaking login

CSRF IS:

- Misusing already trusted login
- Fooling the browser
- Acting as you without permission

How websites stop CSRF

They ask:

“Prove this request came from my page, not another site.”

They do this by:

- Adding a secret random token to forms
- Or blocking cross-site cookie sending

Bad sites can't guess that token.

When CSRF happens

CSRF is possible only if ALL are true:

- Application uses cookies for authentication
- Cookie is automatically sent by browser
- Request causes a state change (POST, PUT, DELETE)
- No CSRF protection in place

Why OAuth2 / OIDC are mostly safe from CSRF

CSRF is possible only if ALL are true:

- Browser is involved
- Authentication data is automatically sent by the browser
- Server trusts that automatic data
- Attacker can trigger a request from another site

If any one of these is missing → CSRF fails.

Why OAuth2 / OIDC are mostly safe from CSRF

What authentication data is auto-sent by browsers?

Cookies

That's it.

Browsers do not auto-send anything else.

They do NOT auto-send:

- Authorization headers
- JWT tokens
- OAuth access tokens
- Custom headers

This single fact kills CSRF in JWT-based systems.

Why OAuth2 / OIDC are mostly safe from CSRF

Why CSRF exists in session-based login but NOT JWT

Session-based (CSRF possible)

Browser → Cookie auto-sent → Server trusts it

JWT-based (CSRF impossible)

Browser → JWT must be manually attached → Server trusts only JWT

Attackers cannot:

- Read JWT (same-origin policy)
- Attach JWT (no auto-send)
- Guess JWT (cryptographically signed)

Why OAuth2 / OIDC are mostly safe from CSRF

Now your OAuth2 + JWT setup (Angular + Spring Boot + Okta)

Your flow looks like this:

Angular App —(Bearer JWT)→ Spring Boot API

Every API request contains:

Authorization: Bearer eyJhbGciOi...

Key observation :

The browser sends this header only if Angular explicitly adds it.

The browser will never add it automatically.

Why OAuth2 / OIDC are mostly safe from CSRF

What an attacker tries in CSRF (and why it fails)

Attacker's goal:

“Make the victim's browser send an authenticated request.”

Attacker tries:

```

```

Browser sends:

GET /transfer

What is MISSING?

- No Authorization header
- No JWT

So Spring Security says:

401 Unauthorized - **Attack failed.**

Why grant types do NOT change CSRF factors

Grant types answer:

How does the client obtain the token?

Examples:

- Authorization Code
- Authorization Code + PKCE
- Client Credentials
- Refresh Token

VERY IMPORTANT:

- Grant types happen BEFORE the API call
- CSRF happens DURING the API call

Why grant types do NOT change CSRF factors

Once the JWT is issued:

Spring Security does NOT care how it was obtained

It only checks:

- Is JWT present?
- Is signature valid?
- Is it expired?

So grant types have ZERO impact on CSRF risk.

Why Spring Security enables CSRF by default

Spring Security doesn't know if:

- You are building a web app (cookies)
- Or an API (JWT)

So it says:

“I'll protect cookies by default.”

When you move to JWT:

```
csrf().disable()
```

You are telling Spring:

“I am not using cookies. I know what I'm doing.”

Why Spring Security enables CSRF by default

Let's say you build a backend API protected by OAuth2 JWT.

http

```
.csrf(csrf -> csrf.disable())  
.oauth2ResourceServer(oauth -> oauth.jwt());
```

What happens?

Client (Teams / browser / mobile) sends JWT
Spring Security:

- Verifies signature
- Checks expiry
- Extracts claims

Request allowed or denied

No cookies → no CSRF → safe

Final connection map

User logs in



Authorization Server (Microsoft Entra ID)

↓ issues tokens via grant

Access Token (JWT)



Client sends JWT in Authorization header



Spring Security validates JWT



Request allowed / denied

- No cookies
- No CSRF
- Safe

Summary

In OAuth2 + JWT-based systems, CSRF attacks are not possible because authentication relies on bearer tokens sent explicitly in the Authorization header, which browsers do not automatically include in cross-site requests. Therefore, CSRF protection can be safely disabled.

Ask yourself:

Is authentication done via cookies?

Yes → CSRF risk

No → CSRF impossible

Is JWT sent via Authorization header?

Yes → disable CSRF

No → rethink

Are we stateless?

Yes → CSRF irrelevant

No → CSRF required



CORS

What is CORS

CORS is a browser security rule that decides whether a website is allowed to call an API hosted on another domain.

CORS is enforced by the BROWSER — not by Spring Boot, not by your API, not by OAuth.

Servers only respond with headers.

Browsers decide whether to block or allow.

Why CORS exists

Without CORS:

- Any website could call any API
- Read responses
- Steal private data

So browsers say:

“I will block cross-website requests unless the API explicitly allows it.”

Simple real-world analogy

Imagine an apartment building (API):

Gate security asks:

“Which society are you coming from?”

- If your society name is on the allowed list, you enter.
- If not → blocked.

That allowed list = CORS configuration.

The problem CORS solves

Your setup:

Angular app → `http://localhost:4200`

Spring Boot API → `http://localhost:8080`

These are different origins (port differs).

Browser says:

“This is a cross-origin request.
I must apply CORS rules.”

What is an “origin”

An origin = scheme + domain + port

Examples:

If any one differs → cross-origin.

URL	Scheme	Host	Port	Origin
http://localhost:4200	http	localhost	4200	http://localhost:4200
http://localhost:8080	http	localhost	8080	http://localhost:8080
https://api.example.com	https	api.example.com	443 (default)	https://api.example.com

What happens WITHOUT CORS

Angular calls API:

GET http://localhost:8080/users

Browser sends request

API responds:

200 OK

Browser checks:

“Did server allow localhost:4200?”

No CORS headers → browser blocks response

- API worked
- Browser blocked it

What happens WITH CORS

API responds with headers:

Access-Control-Allow-Origin: http://localhost:4200

Browser checks:

- Origin allowed?
- Then gives response to Angular

Request succeeds

CORS is NOT security for backend

- CORS does NOT stop hackers
- CORS does NOT protect APIs

CORS only protects browser users.

Postman / curl / backend services:

- Ignore CORS completely

CORS Rule

Client	CORS applies?
Browser (Angular, React)	Yes
Mobile app	No
Desktop app (Teams)	No
Backend service	No
Postman	No

How CORS connects to OAuth2 + JWT

Your flow:

Angular → Authorization: Bearer JWT → Spring Boot

Important:

- Authorization header = custom header
- Custom header → preflight happens
- API must allow it

So you must allow:

Access-Control-Allow-Headers: Authorization

Spring Boot CORS config

@Bean

```
CorsConfigurationSource corsConfigurationSource() {  
    CorsConfiguration config = new CorsConfiguration();  
    config.setAllowedOrigins(List.of("http://localhost:4200"));  
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));  
    config.setAllowedHeaders(List.of("Authorization", "Content-Type"));  
    config.setAllowCredentials(false); // JWT, not cookies  
  
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();  
    source.registerCorsConfiguration("/**", config);  
    return source;  
}
```

http.cors(); // in security config:

Common CORS mistakes

- Using * with credentials
- Forgetting Authorization header
- Not handling OPTIONS
- Thinking CORS = backend security
- Confusing CORS with CSRF

CORS vs CSRF

Topic	CORS	CSRF
Protects	Browser users	Authenticated users
Enforced by	Browser	Server
Related to	Origin	Cookies
JWT apps	Required	Not needed
OAuth APIs	Needed	CSRF disabled

How Teams / Desktop apps differ

- Teams Desktop → no browser → no CORS
- Teams Web → browser → CORS applies
- Mobile apps → no CORS

That's why:

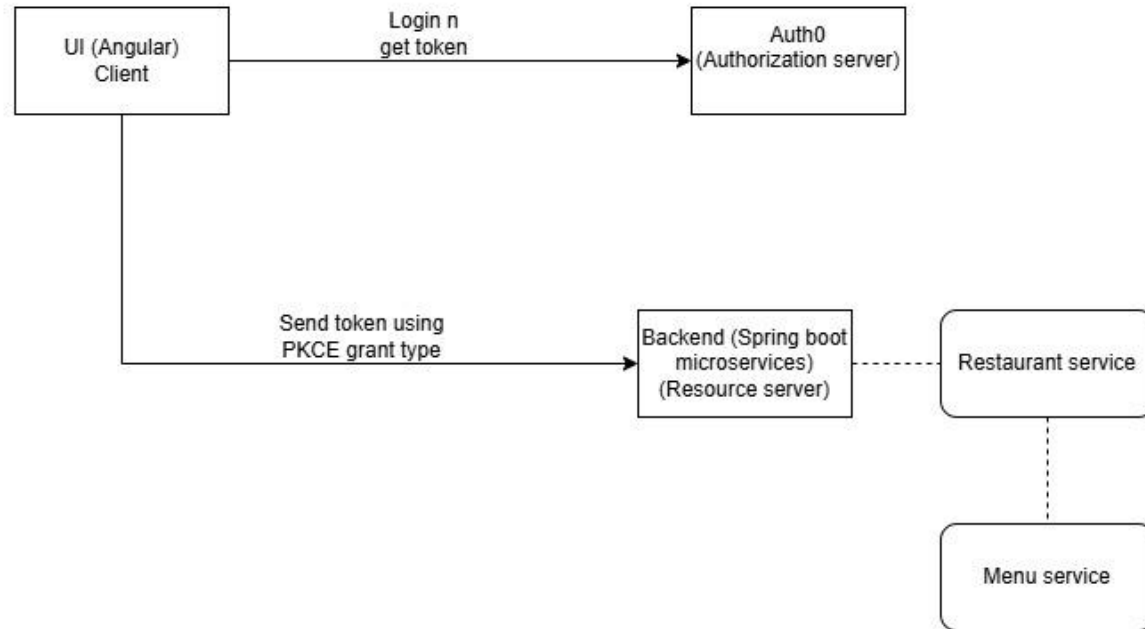
- Browser apps need CORS config
- APIs still secure with JWT

Full stack POC

Intro to Foodify App

- Till Now we have seen small Concepts and their specific POCs but never implement Spring security in an end to end full stack project with
 - Authorization server as Auth0 (Free for developers)
 - Front end in Angular which is the client for us - Foodify UI with A page to login on authorization server (Auth0)
 - BAckend as resource server with Spring n spring boot and microservices - restaurant and menu services
 - Grant type as Authorization code with PKCE
- First we will cover skeleton app with FE and BE configured without SS as happens in Real time enterprise app development process
- Then we will implement SS over it

Foodify (Fullstack POC)



UI Of Foodify App POC

Backend Of Foodify App POC - Restaurant n Menu Services

Auth0 configurations

Spring Security Implementation

`anyRequest().hasAnyRole()`

- .

Why Security for your spring boot app?

Security is needed in a Spring Boot app to:

- Protect sensitive data.
- Prevent unauthorized access to critical features.
- Defend against common attacks (CSRF, XSS, etc.).
- Ensure compliance with industry regulations.

Analogy -

To think about application-level security, you can **consider your home** and the **way you allow access** to it. Do you place the key under the entrance rug? Do you even have a key for your front door? The same concept applies to applications, and Spring Security helps you develop this functionality.

You can choose to leave your house completely unsecured, or you can decide not to allow everyone to enter your home.

The way you configure security can be straightforward like hiding your key under the rug, or it can be more complicated like choosing a variety of alarm systems, video cameras, and locks. In your applications, you have the same options, but as in real life, the more complexity you add, the more expensive it gets. In an application, this cost refers to the way security affects maintainability and performance.

The developer configures components to do precisely what's desired. If you mount an alarm system, it's you who should make sure it's set up for the windows as well as for the doors. If you forget to set it up for the windows, it's not the fault of the alarm system that it doesn't trigger when someone forces a window

Why Spring Security for your spring boot app?

- Like any framework, one of the primary purposes of Spring is to allow you to write less code to implement the desired functionality.
- This is also what Spring Security does. It completes Spring as a framework by helping you write less code to perform one of the most critical aspects of an application—security.
- Spring Security provides predefined functionality to help you avoid writing boilerplate code or repeatedly writing the same logic from app to app.
- However, it also allows you to configure any of its components, thus providing great flexibility.

Key Spring Security Concepts

- Authentication
- Authorization
- Servlet Filters

Authentication

- Imagine you're trying to enter a private club. To get in, you need to prove who you are. Here's how it works:
- The Club Entrance (Your Website/Application): This is the door or gate where you want to get into (e.g., your website or app).
- The Key (Your Username & Password): To enter the club, you need a key. But not just any key! It's a special key that only you should have. In the world of websites, this "key" is like your username (your identity) and password (your secret code).



Authentication

- The Bouncer (Authentication System): At the door, there's a bouncer (the authentication system). The bouncer's job is to check if the key you have matches the one they have on record.
 - If the key is correct (you entered the right username and password), the bouncer says, "Welcome in! You're authenticated!" and you're allowed to enter the club.
 - What Happens if You Don't Have the Right Key?
 - If you try to enter with a wrong key (incorrect username or password), the bouncer stops you and says, "Sorry, you're not on the list." You won't be allowed to enter.

Authentication

In the Context of Web Applications:

- The club entrance is the website or application you're trying to use.
- The key is your username and password (or sometimes even a fingerprint or face scan in more secure cases).
- The bouncer is the system that checks if your credentials are valid (authentication process).
- So, authentication is like the process of proving you are who you say you are using the key (your credentials). If your credentials are correct, you're allowed in. If not, you get stopped at the door.

In short: Authentication = Proving your identity.

Authorization

- ❑ Authorization is about what you're allowed to do or where you're allowed to go once you've proved your identity.
- ❑ Even if you're inside the club (authenticated), the wristband (your role/permissions) decides if you can enter the VIP room or if you're restricted to the general seating area.

Analogy –

Now, let's say you're inside the private club you've just entered after passing the bouncer (authentication). But inside the club, there are different rooms—some are for VIPs, some are for regular guests, and some are for staff only. You can't just wander into any room; you need permission based on who you are.

The Different Rooms in the Club (Resources or Areas of Your Application):

- Inside the club, there are various rooms: VIP lounge, Staff office, General seating area, and so on. Each room represents a resource or section in your application (e.g., an admin page, a regular user page, a dashboard, etc.).

The Wristband (Your Role or Permission):

- After you've entered the club (authenticated), you're given a wristband (this could represent your role or permissions).
- ❖ If you're a VIP, you get a VIP wristband.
- ❖ If you're just a regular guest, you get a guest wristband.
- ❖ If you're a staff member, you get a staff wristband.
- The wristband is a way of telling the club staff which rooms you're allowed to enter.

The Club Staff (Authorization System):

- When you try to enter a particular room, the staff checks your wristband to see if you have permission to enter.
- ◆ **VIP Room:** If you have a VIP wristband, the staff lets you in.
- ◆ **Staff Room:** If you're a regular guest, the staff stops you because your wristband doesn't give you access.
- ◆ **General Seating Area:** If you're a guest or a VIP, you can enter, since it's open to most people.

What Happens if You Don't Have Permission?:

- If you try to enter a restricted room (a room you don't have permission for), the staff will stop you and say, "Sorry, you don't have access to this room."

In the Context of Web Applications:

- The rooms are different sections or pages in the application (e.g., an admin panel, settings page).
- The wristband is your role (like “Admin”, “User”, “Guest”) or permission (can you view/edit something).
- The club staff is the authorization system that checks if your role or permission allows you to access a specific resource or page.

Servlet Filters

- ❑ Servlet Filters provide a way to handle common tasks like authentication and authorization in one place, keeping your actual controller code clean and focused on business logic.

Dispatcher Servlet in Spring:

- ❑ In a typical Spring web application, the main entry point is Spring's DispatcherServlet.
- ❑ The Dispatcher Servlet is responsible for routing incoming HTTP requests to the appropriate controllers (like `@Controller` or `@RestController`).
- ❑ It acts as a central point for handling requests and dispatching them to the correct handler methods.
- ❑ No built-in security: The DispatcherServlet does not have any built-in mechanisms for handling authentication or authorization.
- ❑ This means that security checks (like login verification, role-based access) are not inherently part of the servlet processing.

Problem with Security in Controllers:

- ❑ If you were to handle authentication and authorization directly within the `@Controller` or `@RestController` methods, you would need to manually process things like HTTP Basic Authentication headers, which can be cumbersome.
- ❑ It's not ideal to clutter business logic inside controllers with security-related concerns.

Solution with Servlet Filters:

- ❑ Servlet Filters can be configured to intercept incoming requests before they reach the actual servlet (like the `DispatcherServlet`).
- ❑ Filters can be used to check authentication (whether the user is logged in) and authorization (whether the user has permission to access a resource) in a central location, before requests reach your controllers.
- ❑ Security Filters in Spring Security are a specialized type of servlet filter.
- ❑ This Security Filter is configured to run before the request reaches the `DispatcherServlet` or any specific controller.

Security Filters Tasks

A `SecurityFilter` has roughly 4 tasks:

- ❑ First, the filter needs to extract a username/password from the request. It could be via a Basic Auth HTTP Header, or form fields, or a cookie, etc.
- ❑ Then the filter needs to validate that username/password combination against something, like a database.
- ❑ The filter needs to check, after successful authentication, that the user is authorized to access the requested URI.
- ❑ If the request survives all these checks, then the filter can let the request go through to your `DispatcherServlet`, i.e. your `@Controllers`.

Filter Chain

If we do all these 4 tasks in 1 filter, it would sooner or later lead to one monster filter with a ton of code for various authentication and authorization mechanisms. In the real-world, however, you would split this one filter up into multiple filters, that you then chain together.

For example, an incoming HTTP request would

- ❑ First, go through a LoginMethodFilter (OAuth2LoginAuthenticationFilter: This filter handles OAuth2 login flow.).
- ❑ Then, go through an AuthenticationFilter
- ❑ Then, go through an AuthorizationFilter
- ❑ Finally, hit your servlet.

Security Filters:

- ❑ The filter chain is a sequence of security filters that Spring Security applies to every HTTP request.
- ❑ The filters are executed in a specific order to handle tasks such as authentication, authorization, CSRF protection, and session management.

Filter Chain Configuration:

- ❑ In Spring Security, you configure the filter chain to customize security behavior, such as defining which URLs require authentication or specific roles.
- ❑ Filters are automatically configured in the correct order by Spring Security, but you can customize the filter chain and add your own filters if necessary.

How Filter Chain Works:

- ❑ When a request comes in, it passes through the filter chain, and each filter performs its security-related task.
- ❑ For example, one filter might check if the user is logged in (authentication), while another filter checks if the user has the right permissions (authorization).
- ❑ If any filter rejects the request (e.g., authentication fails), the request is blocked or redirected.

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

 http

 .authorizeRequests()

 .requestMatchers("/admin/**").hasRole("ADMIN")

 .requestMatchers("/user/**").hasRole("USER")

 .anyRequest().authenticated()

 .and()

 .formLogin()

 .loginPage("/login")

 .permitAll()

 .and()

 .logout()

 .permitAll();

 return http.build();

 }

}

Analyzing Spring's FilterChain:

- ❑ `BasicAuthenticationFilter`: Tries to find a Basic Auth HTTP Header on the request and if found, tries to authenticate the user with the header's username and password.
- ❑ `UsernamePasswordAuthenticationFilter`: Tries to find a username/password request parameter/POST body and if found, tries to authenticate the user with those values.
- ❑ `DefaultLoginPageGeneratingFilter`: Generates a login page for you, if you don't explicitly disable that feature. THIS filter is why you get a default login page when enabling Spring Security.
- ❑ `DefaultLogoutPageGeneratingFilter`: Generates a logout page for you, if you don't explicitly disable that feature.
- ❑ `FilterSecurityInterceptor`: Does your authorization.
- ❑ Spring Security provides you a login/logout page, as well as the ability to login with Basic Auth or Form Logins, as well as `CsrfFilter`.
- ❑ Those filters, for a large part, are Spring Security. Not more, not less. They do all the work. What's left for you is to configure how they do their work, i.e. which URLs to protect, which to ignore and what database tables to use for authentication.

Bonus Topic

- What is DSL / Security Configuration DSL

http

```
.authorizeRequests()  
    .antMatchers("/admin/**").hasRole("ADMIN")  
    .anyRequest().authenticated()  
.and()  
.formLogin();
```

- What u see here is DSL, Spring Security's HTTP configuration provides a **fluent API** (part of the core Spring Security library), which is a type of DSL
- The fluent API allows developers to configure security settings in a readable and expressive way, by chaining method calls.

How to configure Spring Security

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {  
    http.csrf(csrf -> csrf.disable())  
        .authorizeHttpRequests(auth -> auth  
            .requestMatchers("/auth/welcome", "/auth/addNewUser", "/auth/generateToken").permitAll()  
            .requestMatchers("/auth/user/**").hasAuthority("ROLE_USER")  
            .requestMatchers("/auth/admin/**").hasAuthority("ROLE_ADMIN")  
            .anyRequest().authenticated() // Protect all other endpoints  
        )  
        .sessionManagement(sess -> sess  
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS) // No sessions  
        )  
        .authenticationProvider(authenticationProvider()) // Custom authentication provider  
        .addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter.class); // Add JWT filter  
  
    return http.build();  
}
```

How to configure Spring Security

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
```

- @Bean: This annotation indicates that the method will return a Spring bean, which will be managed by the Spring container. The SecurityFilterChain bean will be used by Spring Security to configure the security filter chain.
- HttpSecurity http: The HttpSecurity object is passed into the method and is used to configure HTTP security settings (such as authentication, authorization, session management, etc.).

```
.csrf(csrf -> csrf.disable())
```

- CSRF protection is usually used in web applications where users interact with a website using browsers (to prevent unauthorized actions).
- In stateless APIs (like those using JWT for authentication), CSRF protection is typically unnecessary since the application does not maintain a session or rely on cookies for authentication. Hence, CSRF protection is disabled.
- csrf.disable() is a fluent API method to disable CSRF protection.

- Authorization Rules –

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/welcome", "/auth/addNewUser", "/auth/generateToken").permitAll()
    .requestMatchers("/auth/user/**").hasAuthority("ROLE_USER")
    .requestMatchers("/auth/admin/**").hasAuthority("ROLE_ADMIN")
    .anyRequest().authenticated() // Protect all other endpoints
)
```

- `.authorizeHttpRequests(auth -> auth):` This sets up authorization rules for HTTP requests.
- These specific paths (`/auth/welcome`, `/auth/addNewUser`, `/auth/generateToken`) are accessible to everyone, regardless of authentication or authorization. No authentication required.
- `.requestMatchers("/auth/user/**").hasAuthority("ROLE_USER"):` This line specifies that any request to URLs that match `/auth/user/**` requires the user to have the role `ROLE_USER` (meaning the user must be authenticated and have this role).
- `.requestMatchers("/auth/admin/**").hasAuthority("ROLE_ADMIN"):` Similarly, any request to URLs that match `/auth/admin/**` requires the user to have the role `ROLE_ADMIN`.
- `.anyRequest().authenticated():` This rule ensures that any other request (i.e., any URL not explicitly matched above) must be authenticated. Users must be logged in and authenticated to access these endpoints.

- Session Management (Stateless API) -

`.sessionManagement(sess -> sess`

`.sessionCreationPolicy(SessionCreationPolicy.STATELESS))`

□ `.sessionManagement(sess -> sess)`: Configures session management for the application.

□ `.sessionCreationPolicy(SessionCreationPolicy.STATELESS)`: This is key for stateless applications. By setting the session creation policy to STATELESS, you tell Spring Security not to use sessions to store any state about the user.

□ In a stateless API (often with JWT-based authentication), the server does not store any session data. The client must provide the token with each request, and the server will validate it on every request without needing to track sessions.

- Custom Authentication Provider

`.authenticationProvider(authenticationProvider())`

□ This line adds a custom authentication provider to handle user authentication , `authenticationProvider()` is likely a method (not shown in the provided code) that returns a custom implementation of an `AuthenticationProvider` used to authenticate the user. The custom provider is necessary when you're using a custom authentication mechanism, such as JWT-based authentication.

- Adding a JWT Authentication Filter

`.addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter.class); // Add JWT filter`

- `.addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter.class)`: This line adds a custom filter to the Spring Security filter chain.
- `authFilter`: This is likely a JWT authentication filter (again, not shown in the provided code) that intercepts incoming requests, extracts the JWT token from the request, and validates it. The filter authenticates the user based on the token in the request header.
- `UsernamePasswordAuthenticationFilter.class`: This specifies that the `authFilter` should be placed before the `UsernamePasswordAuthenticationFilter` in the filter chain. The `UsernamePasswordAuthenticationFilter` is the default filter used by Spring Security for handling login and authentication using a username and password. By placing the JWT filter before this filter, you ensure that JWT authentication is checked first, before any other form of authentication (like username/password) is attempted.

- Returning the Filter Chain

`return http.build();`

- `http.build()`: Finally, the `http.build()` method is called to build and return the configured `SecurityFilterChain` object.
- This object is then managed by the Spring Security framework and used to apply the security configuration to incoming HTTP requests

- Returning the Filter Chain

`return http.build();`

- `http.build()`: Finally, the `http.build()` method is called to build and return the configured `SecurityFilterChain` object.
- This object is then managed by the Spring Security framework and used to apply the security configuration to incoming HTTP requests

Bonus Topic

- How just by adding a dependency , you get a login logout form
- By-default we have this functionality
- We have a Spring auto configuration as a part of our spring security starter dependency
- This imports `SpringBootWebSecurityConfiguration` class
- At bottom last method we have bean automatically created - `SecurityFilterChain` that you customize later.
- **This default configuration is why your application is on lock-down, as soon as you add Spring Security to it.**

Bonus Topic

@Bean

@Order(2147483642)

SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
 http.authorizeHttpRequests((requests) -> {

((AuthorizeHttpRequestsConfigurer.AuthorizedUrl)requests.anyRequest()).authenticated();
 });
 http.formLogin(Customizer.withDefaults());
 http.httpBasic(Customizer.withDefaults());
 return (SecurityFilterChain)http.build();
 }
}

Bonus Topic

- ❑ `.anyRequest().authenticated()` ensures that all requests are protected and require authentication. This is the default behavior when no other specific URL patterns are defined.
- ❑ `http.formLogin(Customizer.withDefaults())`:
This enables form-based login with the default configuration. When a user tries to access a protected resource without authentication, they will be redirected to a login page.
- ❑ `http.httpBasic(Customizer.withDefaults())`:
This enables HTTP Basic Authentication with the default settings. This is another form of authentication where the username and password are sent in the Authorization header of each HTTP request (base64 encoded).
- ❑ `http.build()`: This finalizes the security configuration and creates the `SecurityFilterChain` object, which Spring Security will use to apply the security settings.

Authentication with Spring Security

When it comes to authentication and Spring Security you have roughly three scenarios:

- ❑ **The default:** You can access the (hashed) password of the user, because you have his details (username, password) saved in e.g. a database table.
- ❑ **Less common:** You cannot access the (hashed) password of the user. This is the case if your users and passwords are stored somewhere else, like in a 3rd party identity management product offering REST services for authentication. Think: Atlassian Crowd.
- ❑ **Also popular:** You want to use OAuth2 or "Login with Google/Twitter/etc." (OpenID), likely in combination with JWT. Then none of the following applies and you should go straight to the OAuth2 chapter.

Default Authentication (Having access to the user's password)

In this case Spring Security needs you to define two beans to get authentication up and running.

- A `UserDetailsService`.
- A `PasswordEncoder`.

UserDetailsService

@Bean

```
public UserDetailsService userDetailsService(UserInfoRepository userInfoRepository) {  
    return new CustomUserDetailsService(userInfoRepository);  
}
```

@Service

```
public class CustomUserDetailsService implements UserDetailsService {  
    @Autowired  
    private final UserInfoRepository userRepository;
```

@Override

```
public UserDetails loadUserByUsername(String username) throws  
UsernameNotFoundException {  
    return userRepository.findByUsername(username)  
        .orElseThrow(() -> new UsernameNotFoundException("User not found"));  
}
```

UserDetailsService

- A UserDetailsService loads UserDetails via the user's username. Note that the method takes only one parameter: username (not the password).
- The UserDetails interface has methods to get the (hashed!) password and one to get the username.
- UserDetails has even more methods, like is the account active or blocked, have the credentials expired or what permissions the user has - but we won't cover them here.

Full UserDetails Workflow: HTTP Basic Authentication

- Now think back to your HTTP Basic Authentication, that means you are securing your application with Spring Security and Basic Auth. This is what happens when you specify a `UserDetailsService` and try to login:
- Extract the username/password combination from the HTTP Basic Auth header in a filter. You don't have to do anything for that, it will happen under the hood.
- Call your **CustomUserDetailsService** to load the corresponding user from the database, wrapped as a `UserDetails` object, which exposes the user's hashed password.
- Take the extracted password from the HTTP Basic Auth header, hash it automatically and compare it with the hashed password from your `UserDetails` object. If both match, the user is successfully authenticated.
- That's all there is to it. But hold on, how does Spring Security hash the password from the client (step 3)? With what algorithm?

PasswordEncoders

- ❑ Spring Security cannot magically guess your preferred password hashing algorithm. That's why you need to specify another `@Bean`, a `PasswordEncoder`.
- ❑ If you want to, say, use the BCrypt password hashing function (Spring Security's default) for all your passwords, you would specify this `@Bean` in your `SecurityConfig`.

`@Bean`

```
public BCryptPasswordEncoder bCryptPasswordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

`@Bean`

```
public PasswordEncoder passwordEncoder() {  
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();  
}
```


PasswordEncoders

- What if you have multiple password hashing algorithms, because you have some legacy users whose passwords were stored with MD5 (don't do this), and newer ones with Bcrypt or even a third algorithm like SHA-256? Then you would use the following encoder:
- How does this delegating encoder work? It will look at the UserDetails's hashed password (coming from e.g. your database table), which now has to start with a {prefix}. That prefix, is your hashing method! Your database table would then look like this:

Username

password

john@doe.com.

{bcrypt}\$2y\$12\$6t86Rpr3lIMANhCUt26oUen2WhvXr/A89Xo9zJion8W7gWgZ/zA0C

my@user.com.

{sha256}5ffa39f5757a0dad5dfada519d02c6b71b61ab1df51b4ed1f3beed6abe0ff5f6

PasswordEncoders

- ❑ Spring Security will:
- ❑ Read in those passwords and strip off the prefix ({bcrypt} or {sha256}).
- ❑ Depending on the prefix value, use the correct PasswordEncoder (i.e. a BCryptEncoder, or a SHA256Encoder)
- ❑ Hash the incoming, raw password with that PasswordEncoder and compare it with the stored one.
- ❑ That's all there is to PasswordEncoders.

Summary

- Summary: Having access to the user's password
- The takeaway for this section is: if you are using Spring Security and have access to the user's password, then:
- Specify a `UserService`. Either a custom implementation or use and configure one that Spring Security offers.
- Specify a `PasswordEncoder`.
- That is Spring Security authentication in a nutshell.

For 2nd type – we don't have access

- We will use Okta for centralized identity management – not Atlassian Crowd
- The takeaway for this section is: if you are using Spring Security and have access to the user's password, then:
 - Specify a UserDetailsService. Either a custom implementation or use and configure one that Spring Security offers.
 - Specify a PasswordEncoder.
 - That is Spring Security authentication in a nutshell.

Custom Authentication

Note

- In real-time scenarios, we typically use OIDC, HTTP Basic, certificates, or similar standard methods—custom authentication is rarely used, but it's still good to be familiar with it.

Custom Authentication Filter

-

Custom Authentication Manager

-

Custom Authentication Provider

-